Forside

Eksamensinformation
EMK110020P - MATØK1 P1 Diskret optimering E22

Besvarelsen afleveres af
Nicolai Kjær Johannesen
njohan22@student.aau.dk

Magnus Toft Høj
mhoj22@student.aau.dk

Niklas Thorgaard Sørensen
ntso22@student.aau.dk

Dennis Legart Rittermann Hansen
dlrh22@student.aau.dk

Eksamensadministratorer
Lise Støvelbæk Hansen
listha@mp.aau.dk
+45 99409835

Bedømmere
Kenn Erik Markholm Andersen
Eksaminator
kema@math.aau.dk

J. Eduardo Vera-Valdés
Intern medbedømmer
eduardo@math.aau.dk
+45 99408845

Besvarelsesinformationer

Projekttitel: Indtast den endelige titel på dit projekt. Titlen vil fremgå af dit eksamensbevis: Diskret Optimering - Gru

Projekttitel, engelsk. Titlen vil fremgå af dit eksamensbevis : Discrete Optimization - Group 5.216B

Indeholder besvarelsen fortroligt materiale: Nej

P1-Project

Mathematics-Economics

1. Semester, Group 5.216B

Magnus Toft Høj
Dennis L. R. Hansen
Niklas Thorgaard Sørensen
Nicolai Kjær Johannesen





AALBORG UNIVERSITET
STUDENTERRAPPORT

Department of Mathematical Sciences
Mathematics-Economics
Skjernvej 4A
9220 Aalborg Øst
http://math.aau.dk

Title:

Gas storage optimization

Subject:

Discrete optimization

Project:

P1-Project

Project period:

Autumn semester 2022

Project group:

5.216B

Group members:

Magnus Toft Høj
Dennis L. R. Hansen
Niklas Thorgaard Sørensen
Nicolai Kjær Johannesen

Supervisor:

Kenn Erik Markholm Andersen

Page number: 110 + front page
Completed 21/12-2022

Abstract:

The purpose of this project is to optimize the profit when controlling a gas storage in a given period of time by buying and selling units of gas. In order to solve the problem posed in the project, induction, graph theory and algorithms are introduced. The problem consists of a basis problem and an expanded problem that takes time and a goal for the amount of units that should be present in the storage unit at the end of the period into account. This optimization problem was constructed using graphs from graph theory to represent the period and the trade of natural gas. The greedy algorithm Dijkstra's Algorithm was then implemented in python, to examine the graphs and find the longest path through this representation of the possible trading strategies during the period. It does this by finding the shortest path through a remodelled version of the graph. Hence, this path would then represent the most optimal trading strategy throughout the period, which gives the maximum profit that can be achieved during the period. In the basis problem the maximum profit is 252.72€, while the maximum profit in the expanded problem is 188.27€.

The contents of the report are freely available, but publication (with reference) may only be made in agreement with the authors.

Preface

This project deals with discrete optimization, and it is written during the 1st semester of Mathematics-Economics by students from the project group 5.216B. The project period started on 10-10-2022 and ended on 21-12-2022.

The group would like to thank Kenn Erik Markholm Andersen for great guidance throughout the period.

Reading guide

References are written in the beginning of every chapter, and refers to the bibliography at the end of the project. References are written using the Vancouver Method. In the case that a reference only applies to one sentence, it will be noted before the full stop. If a reference applies to more than one sentence, it will be noted after the full stop.

Magnus Toft Høj
mhoj22@student.aau.dk

Dennis L. R. Hansen
dlrh22@student.aau.dk

Niklas Thorgaard Sørensen
ntso22@student.aau.dk

Nicolai Kjær Johannesen
njohan22@student.aau.dk

Nomenclature

Sets

$\emptyset$	The empty set
$\mathbb{N}$	Set of natural numbers
$\mathbb{R}$	Set of real numbers
$\mathbb{Z}$	Set of all intergers

Symbols

$\mathbb{1}[x]$	An indicator function with the value 1 if x is true and 0 if x is false
$\|x\|$	Absolute value of x when x is a number and cardinality of x when x is a set
α	The variable alpha
$\blacksquare$	End of proof
$\cup$	Union - Operation through which sets can be combined
Δ	Delta - Denotes a change in a variable
€	Euro
$\exists$	Existential quantifier
$\forall$	Universal quantifier
$\in$	Set operator "in"
∞	Infinity
$\lfloor x \rfloor$	Floor function
$\neg$	Logical operator "negation"
$\notin$	Set Operator "not in"
Ω	Big Omega
$\rightarrow$	Logical operator "if, then"
$\subseteq$	Set operator "is a subset of"
$\sum$	Big Sigma - Denotes the sum of a series
Θ	Big Theta
$\vee$	Logical operator "or"
$\wedge$	Logical operator "and"

I_k	$k \times k$ identity matrix
O	Big-O

Nomenclature

Sets

$\emptyset$	The empty set
$\mathbb{N}$	Set of natural numbers
$\mathbb{R}$	Set of real numbers
$\mathbb{Z}$	Set of all intergers

Symbols

$\mathbb{1}[x]$	An indicator function with the value 1 if x is true and 0 if x is false
$\|x\|$	Absolute value of x when x is a number and cardinality of x when x is a set
α	The variable alpha
$\blacksquare$	End of proof
$\cup$	Union - Operation through which sets can be combined
Δ	Delta - Denotes a change in a variable
€	Euro
$\exists$	Existential quantifier
$\forall$	Universal quantifier
$\in$	Set operator "in"
∞	Infinity
$\lfloor x \rfloor$	Floor function
$\neg$	Logical operator "negation"
$\notin$	Set Operator "not in"
Ω	Big Omega
$\rightarrow$	Logical operator "if, then"
$\subseteq$	Set operator "is a subset of"
$\sum$	Big Sigma - Denotes the sum of a series
Θ	Big Theta
$\vee$	Logical operator "or"
$\wedge$	Logical operator "and"

I_k	$k \times k$ identity matrix
O	Big-O

Contents

Preface **i**

Nomenclature **vi**

1 Introduction **1**
1.1 Problem Analysis . . . 1
1.2 Problem Definition . . . 2

2 Graph Theory **3**
2.1 Graphs and Graph Models . . . 3
2.1.1 Simple Undirected Graphs . . . 3
2.1.2 Simple Directed Graphs . . . 4
2.1.3 Multigraphs . . . 5
2.1.4 Pseudographs . . . 6
2.1.5 Directed Multigraphs . . . 7
2.1.6 Mixed Graph . . . 8
2.1.7 Overview of Properties of Different Types of Graphs . . . 9
2.2 Graph Terminology . . . 9
2.2.1 Degrees . . . 10
2.2.2 Neighbourhoods . . . 11
2.2.3 Adjacency Matrices . . . 12
2.3 Paths in Graphs . . . 13
2.3.1 Paths . . . 13
2.3.2 Connectivity . . . 14
2.3.3 Weighted Graphs . . . 15

3 Methods of Proofs **17**
3.1 Proof by Induction . . . 17
3.1.1 Reasons for Using Mathematical Induction . . . 17
3.1.2 Constructing a Proof by Mathematical Induction . . . 18
3.2 Proof by Contradiction . . . 22

4 Algorithms **25**
4.1 Types of Algorithms . . . 26
4.1.1 Searching Algorithms . . . 26
4.1.2 Sorting Algorithms . . . 28
4.1.3 Greedy Algorithms . . . 30
4.1.4 Brute Force Algorithms . . . 31
4.2 Shortest-Path Algorithms . . . 32
4.2.1 Dijkstra's Algorithm . . . 33

4.2.2 Proof that Dijkstra's Algorithm Finds A Shortest Path 37
4.3 Complexity 39
4.3.1 Big-O Notation 39
4.3.2 Big-Omega Notation 46
4.3.3 Big-Theta Notation 47
4.3.4 Complexity in Algorithms 48

5 Interest Rate Theory **51**
5.1 Simple Interest 51
5.2 Periodic Compounding of Interest 53
5.3 Continuous Compounding of Interest 58
5.4 Discount Factor 62

6 Basis Problem **63**
6.1 Definition of The Basis Problem 63
6.2 Simple Version of The Basis Problem 64
6.3 Solving The Basis Problem 68
6.4 Using Python to Solve The Basis Problem 70
6.5 Solution to The Basis Problem 76

7 Expanded Problem **81**
7.1 Definition of The Expanded Problem 81
7.2 Using Python to Solve The Expanded Problem 83
7.3 Solution to The Expanded Problem 85

8 Discussion **91**
8.1 Choice of Algorithm 91
8.2 Natural Gas Prices 92

9 Perspectivation **93**

10 Conclusion **95**

11 Bibliography **97**

A Appendix **99**
A.1 The Triangle Inequality 99
A.2 The Binomial Theorem 102
A.3 Source Code Used to Solve The Basis Problem 102
A.4 Source Code Used to solve The Expanded Problem 105
A.5 Source Code for Finding an Insignificant Penalty Factor 108

1 Introduction

1.1 Problem Analysis

The problem in this project concerns the control of a gas-storage during a period of one year. The storage unit is controlled but not owned, since a deal with the owner to rent the unit has been made. The goal in the period is to maximize the profit or minimize the losses achieved by buying, selling or holding gas in the period of control. It is assumed that a forward-curve on the price of gas is given, which means a range of contracts with the counterparts have been made. These contracts provide the prices on gas at all time points in the period. At every time point of the given period it is possible to buy or sell a limited amount of gas. There exists a lower and upper limit, on the amount of gas which can be stored in the unit. The owner of the unit has agreed to buy the final amount of gas present in the storage unit at the end of the period. The problem looks apart from any costs associated with such a leasing contract.

Moreover, the basis problem can be expanded with more parameters to construct an expanded problem. In the expanded problem, the goal in the period is to maximize the profit or minimize the losses achieved by buying, selling or holding gas in the period of control. At some time points of the given period, it is not possible to keep the number of gas units constant, this is due to the minimum and maximum capacity varying throughout the period. Additionally a clause in the contract has been made causing a penalty factor to be applied to the sale of the remaining units of gas, if the clause on the number of units of gas at the end of the period is not met.

Initial Wondering

The initial wonder that arose on the basis of the problem was how a sale of gas with a limit on trade could be optimized. Furthermore, it was wondered how the given future prices could be used to optimize the profits on the trade of gas. Likewise it was wondered which mathematical theories and economical theories could be used to construct a model for solving the problem. In our research and by working with our supervisor, we found that graph theory was relevant for representing the given stock of gas and prices at a given time. It was found that algorithms could be used to solve optimization problems, which likewise was relevant for this problem. We then settled on using graph theory to represent the gas storage and the in and outflow of units of gas. Furthermore we settled on using an algorithm to find the most optimal trade of the gas to maximize the profit. Therefore the initial wondering: how the given future prices could be used to optimize the profits on the trade of gas within the problem got narrowed down to another set of questions, which would need to be answered in order to solve the problem.

Questions for The Research

- What is graph theory?
- What is an algorithm?
- How can we optimize sales of gas based on a limit on trade and forecast on future prices?

1.2 Problem Definition

How can graph theory and algorithms be used to determine how much gas should be bought or sold at a given time to maximize the profit or minimize the losses during a given period of time?

2 | Graph Theory

The definitions used in this chapter can be found in [1, p. 673-682].

2.1 Graphs and Graph Models

Graphs are discrete structures that consists of vertices and edges. Vertices are the points in a graph, and edges are the lines that connect these vertices. There exists different types of graphs depending on which type of edges that are present in the graph. A general definition of a graph is given by 2.1.

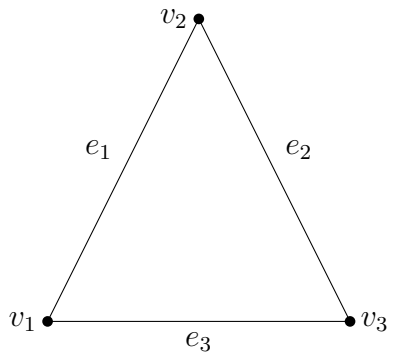
Definition 2.1.
A graph $G = (V, E)$ consists of V, a nonempty set of vertices, and a set of edges E. Each edge has either one or two vertices associated with it, called its endpoints. An edge is said to connect its endpoints.

2.1.1 Simple Undirected Graphs

Definition 2.2.
A graph $G = (V, E)$ is a simple graph, when each pair of vertices (u, v) where $u, v \in V$ is connected by at most one undirected edge, and no edge starts and ends at the same vertex. An undirected edge is the line connecting an unordered pair of vertices (u, v) which means that $\{u, v\} = \{v, u\}$.

A simple undirected graph, is a graph which does not contain any edges that connects a vertex to itself, this is also called a loop which will be introduced in 2.1.4. Furthermore for a graph to be simple, it must not contain any two edges that connect the same pair of vertices. Another requirement for a simple undirected graph is that each edge in the graph must not have any arrows or other ways of identifying which direction the edges point. Since no two edges are allowed to connect the same pair of vertices for the graph to be a simple undirected graph, it is possible to say that an edge in a simple undirected graph that connects the pair of vertices u and v, must be the only edge that connect the vertices u and v. An example of a simple undirected graph is given in 2.3:

Example 2.3. Simple undirected graph



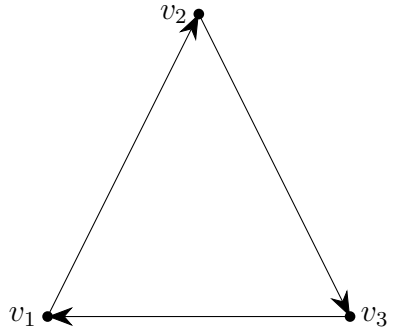
This graph is a simple undirected graph, because it contains a set of vertices $V = \{v_1, v_2, v_3\}$ and a set of undirected edges $E = \{e_1 = \{v_1, v_2\}, e_2 = \{v_2, v_3\}, e_3 = \{v_1, v_3\}\}$. Since no two vertices are connected by more than one edge, it is apparent that the graph is a simple undirected graph. Furthermore the directions of the edges are not specified, and hence the graph is undirected.

2.1.2 Simple Directed Graphs

Definition 2.4.
A directed graph $G = (V, E)$, consists of a set of vertices $V \neq \emptyset$, and a set of directed edges E. A directed edge $e \in E$ is associated with an ordered pair (u, v) where $u, v \in V$, is said to start at u and end at v.

When a directed graph is constructed, the edges are represented by an arrow pointing from the initial vertex u to the terminal vertex v, meaning that the edge starts in u and ends in v. A simple graph with directed edges is said to be a simple directed graph. An example of a simple directed graph can be seen in example 2.5

Example 2.5. Simple directed graphs



This graph is a simple directed graph, because it contains a set of vertices $V = \{v_1, v_2, v_3\}$, and a set of 3 directed edges $E = \{\{v_1, v_2\}, \{v_2, v_3\}, \{v_3, v_1\}\}$. This graph

is simple, because no pair of edges has exactly the same initial vertex and with exactly the same terminal vertex, additionally no edge has the same initial vertex as it's own terminal vertex. The graph is said to be directed, because the ordering of the vertices representing the edges cannot be interchanged.

2.1.3 Multigraphs

Sometimes a graph G has multiple edges $\{e_1, e_2, \cdots, e_m\} \subseteq E$ that connect an unordered pair of vertices (u, v). Here m denotes the amount of edges $\{u, v\}$ that connect the pair of vertices (u, v), the edge is then said to be of multiplicity m. A graph with multiple edges connecting a pair of vertices is called a multigraph.

Definition 2.6.
A graph $G = (V, E)$ contains a set of vertices $V \neq \emptyset$ and a set of edges E for which,

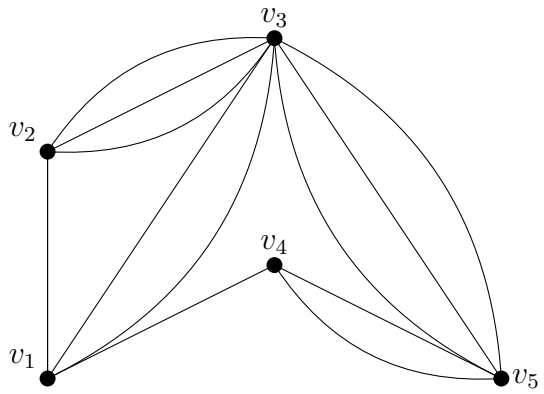
$$E \subseteq \{\{u, v\} | u, v \in V, u \neq v\}$$

This means that the set of edges E in G is a subset of all possible combinations of vertices in V without loops. For G to be a multigraph, the following proposition must be true:

$$\exists e_i, e_j \in E : e_i = e_j \wedge i \neq j$$

This means that there exists two edges in the set of edges that connects the same two vertices, where the edges are not the same edge.

Example 2.7. Multigraph



The graph illustrated above is a multigraph, because all edges in the graph are undirected and it contains multiple edges connecting the same pair of unordered vertices.

This can be seen by the fact that the graph contains three different edges that connect the unordered pair of vertices (v_3, v_5), namely $\{v_3, v_5\}_1$, $\{v_3, v_5\}_2$ and $\{v_3, v_5\}_3$, therefore the edge $\{v_3, v_5\}$ is said be an edge of multiplicity 3. The same argument could be applied to show that any other pair of vertices that are connected by at least two edges makes the graph a multigraph.

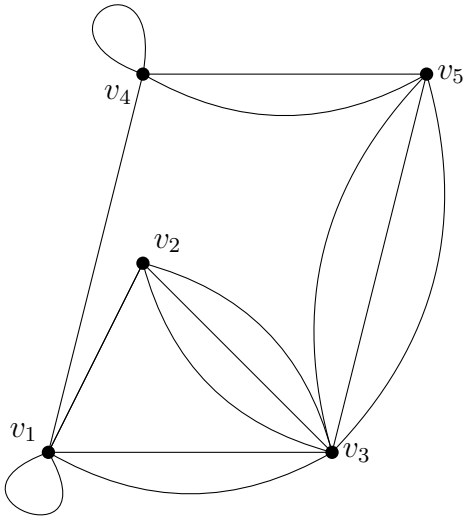
2.1.4 Pseudographs

While multigraphs have multiple edges connecting at least a pair of vertices, they cannot contain an edge connecting a vertex to itself. These edges are called loops and indicate that the graph is a pseudograph. This means that pseudographs are defined as being multigraphs that contain at least one loop.

Definition 2.8.
An undirected graph $G = (V, E)$ with some vertex $v \in V$ for which there exists an edge $\{v, v\}$ is called a pseudograph.

To illustrate how a pseudograph can be constructed, an example of a pseudograph can be seen in example 2.9. In this case the pseudograph contains multiple edges between some pairs of vertices. This is however not necessary for the graph to be a pseudograph, because a undirected graph not containing at least one loop is either a simple graph or a multigraph. None of these types of graphs are allowed to contain a loop. Hence, if a loop is present in a graph it must be a pseudograph even if no multiple edges are present between an unordered pair of vertices. Therefore it is not a requirement that a graph contains multiple edges for the graph to be a pseudograph.

Example 2.9. Pseudograph



This graph can be described as being a pseudograph, because it meets the requirements that all edges must be undirected, and that it must contain at least one loop. These are present as the edges $\{v_4, v_4\}$ and $\{v_1, v_1\}$ are loops.

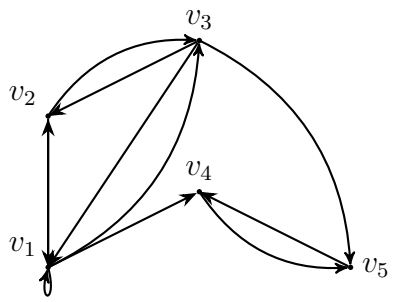
2.1.5 Directed Multigraphs

In the definition of a multigraph in 2.1.3, a multigraph was defined as a graph that only contains undirected edges. As described earlier in 2.1.2, edges can be directed, hence we must also define a multigraph where edges of this type are allowed to be present. Directed graphs that contain multiple directed edges between a ordered pair of vertices (u, v), are called directed multigraphs.

Definition 2.10.
A directed graph $G = (V, E)$ with multiple edges in the set of edges E that connects an ordered pair of vertices (u, v) or at least one directed loop v, v or both, is called a directed multigraph.

For a graph to be a directed multigraph it is not a requirement that the m edges connecting the same ordered pair of vertices, if the graph contains no loops, share the same initial and terminal vertex. Likewise a graph that contains a directed loop must be a directed multigraph if all edges contained in the graph are directed. An example of a directed multigraph is illustrated in 2.11.

Example 2.11. A graph $G = (V, E)$



Since the graph G contains a set of vertices $V = \{v_1, v_2, v_3, v_4, v_5\}$ and a set of edges $E = \{\{v_1, v_1\}, \{v_1, v_2\}, \{v_1, v_3\}, \{v_1, v_4\}, \{v_2, v_3\}, \{v_3, v_1\}, \{v_3, v_2\}, \{v_3, v_5\}, \{v_4, v_5\}, \{v_5, v_4\}\}$, which contains multiple directed edges, that connect the same pair of vertices, the graph is a directed multigraph. An example of this are the edges $\{\{v_4, v_5\}, \{v_5, v_4\}\}$, that connect the same pair of vertices (v_4, v_5). Note that these edges are not ordered in the same order as they do not share the same initial vertex and terminal vertex, which is not a requirement for this graph to be a directed multigraph. A loop is also contained in the graph which is represented by the edge: $\{v_1, v_1\}$.

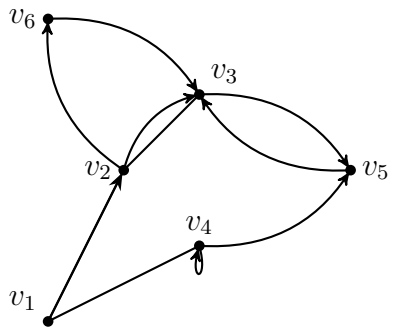
2.1.6 Mixed Graph

Previously in 2.1.1 and in 2.1.2 graphs have been defined where all edges contained in the graph are either directed or undirected. This poses the question concerning graphs containing at least one directed edge and one undirected edge. This type of graph is called a mixed graph, since the edges contained in the graph are mixed.

Definition 2.12.
A mixed graph $G = (V, E)$ has at least one edge that connects the ordered pair of vertices (u, v) and at least one edge that connects the unordered pair of vertices (w, x) where $u, v, w, x \in V$. This means that a mixed graph contains at least two edges $|E| \geq 2$.

This definition states that all graphs containing both an undirected edge and a directed edge are mixed graphs. Therefore both loops and multiple edges between a single pair of vertices (u, v) are allowed. Hence what separates a mixed graph from directed multigraphs and pseudographs is that it must contain at least one undirected edge and one directed edge.

Example 2.13. Mixed Graph



This graph is a mixed graph because at least one directed edge is present and at least one undirected edge is present. An example of a directed edge is $\{v_6, v_3\}$, and $\{v_2, v_3\}$ is an example of an undirected edge. The fact that multiple edges between the pair of vertices (v_3, v_5) and the loop $\{v_4, v_4\}$ does not prove nor disprove that this graph is a mixed graph, since they are allowed but not a requirement.

2.1.7 Overview of Properties of Different Types of Graphs

The table 2.1.7 shows the different properties, of different types of graphs. The graph type, can be found by examining whether the edges are undirected, directed or at least one of each is contained in the graph. The type of graph can also be established by examining whether multiple edges are present and or if loops are allowed.

Type	Edges	Multiple Edges Allowed?	Loops Allowed?
Simple graph	Undirected	No	No
Simple directed graph	Directed	No	No
Multigraph	Undirected	Yes	No
Pseudograph	Undirected	Yes	Yes
Directed multigraph	Directed	Yes	Yes
Mixed graph	Directed and undirected	Yes	Yes

2.2 Graph Terminology

Now that the different types of graphs have been defined and ways to represent these visually have been presented, we can expand our knowledge of graphs to be able to represent these in other ways than visually. This is especially useful in cases where it is difficult to represent a graph visually e.g. when a graph contains hundreds of vertices and hundreds of edges it is tedious to draw by hand.

2.2.1 Degrees

A way to describe a graph is to count the number of edges incident to a given vertex in the graph. An edge being incident to a vertex means that the edge has an endpoint at this vertex. This is useful when analyzing paths in graphs and the connectivity of a graph which we will return to in 2.3. The sum of the number of edges incident with a given vertex is denoted as the degree of this vertex.

Definition 2.14.
The degree of a vertex v in an undirected graph is the number of endpoints of edges that are located in v. The degree of v will be denoted as $deg(v)$.

By 2.14 a single edge connecting the vertex v to another vertex u, contributes once to $deg(v)$ and likewise once to $deg(u)$. As we know from chapter 2 it is possible for an edge to be a loop. In this case a given loop $e = \{v, v\}$ will contribute twice to $deg(v)$, which can be understood intuitively when taking into consideration, that e has two endpoints in v, and each of these endpoints contributes to the degree once. A vertex is called isolated, when it has a degree of 0, while a vertex is called pendant, when it has a degree of 1.
The sum of the degree of all vertices in a graph is directly related to the total sum of edges in the graph:

Theorem 2.15. The handshaking theorem
Let $G = (V, E)$ be an undirected graph where $|E| = m$, then

$$2m = \sum_{v \in V} deg(v).$$

Proof. The handshaking theorem
In theorem 2.15 it is stated that the total sum of degrees in a graph is twice the number of edges in the set of edges for this graph. This relation is proved by the fact that each edge has two endpoints, which contributes twice to the total amount of degrees in the graph. ■

When considering directed graphs, two different types of degrees are introduced, namely in-degrees and out-degrees.

Definition 2.16.
In a directed graph, the in-degree of a vertex v is denoted $deg^-(v)$ and is the number of edges having v as their terminal vertex. The out-degree of v is the number of edges having v as their initial vertex and is denoted by $deg^+(v)$.

Note that a loop at a vertex v in a directed graph will add one in-degree and one out-degree to the vertex v. In a directed graph each edge has one initial vertex and one terminal vertex.

Therefore the cardinality of the set of edges of a directed graph must be equal to the sum of all in-degrees which also must be equal to the sum of all out-degrees.

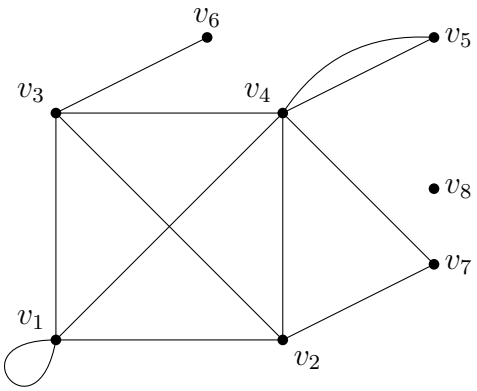
2.2.2 Neighbourhoods

When describing a graph it can prove useful to describe all vertices that are adjacent to a given vertex. If two vertices connected by an edge, they are adjacent. This is a way to describe the density or sparseness of a graph and can be useful when representing a graph in computer algorithms. A neighbourhood of a vertex is given by the following definition.

Definition 2.17.

The set of all vertices adjacent to a given vertex v, in the nonempty set V in the graph $G = (V, E)$, is called the neighbourhood of v and is denoted by $N(v)$. Given $A \subseteq V$, $N(A)$ denotes the set of all vertices in the graph G that are adjacent to at least one vertex in A. We can represent this by, $N(A) = \bigcup_{v \in A} N(v)$.

Example 2.18. A graph $G = (V, E)$



The set of vertices in the graph G is given by $V = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8\}$. The degrees are given by: $deg(v_1) = 5$, $deg(v_2) = deg(v_3) = 4$, $deg(v_4) = 6$, $deg(v_5) = deg(v_7) = 2$, $deg(v_6) = 1$ and $deg(v_8) = 0$. Now we can describe the set of vertices that are adjacent to the given vertices. The neighbourhood of each vertex is given by: $N(v_1) = \{v_1, v_2, v_3, v_4\}$, $N(v_2) = \{v_1, v_3, v_4, v_7\}$, $N(v_3) = \{v_1, v_2, v_4, v_6\}$, $N(v_4) = \{v_1, v_2, v_3, v_5, v_7\}$, $N(v_5) = \{v_4\}$, $N(v_6) = \{v_3\}$, $N(v_7) = \{v_2, v_4\}$ and $N(v_8) = \emptyset$. We can conclude from this description of the vertices that the vertex v_8 is isolated due to its neighbourhood being the empty set resulting in its degree being zero. Likewise we see that the vertex v_6 is pendant because it has a degree of exactly one and it therefore has exactly one other vertex in its neighbourhood.

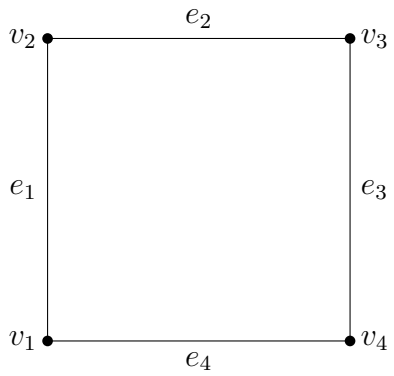
2.2.3 Adjacency Matrices

We can also represent a graph using an adjacency matrix. By using an adjacency matrix, we can disclose which vertices are neighbours and disclose the endpoints of all edges. The visual representation of a graph can be constructed based on the adjacency matrix, which is not possible based on degrees alone. In contrast to neighbourhoods an adjacency matrix specifies the number of edges that are present between any two pair of vertices. Therefore adjacency matrices are convenient when examining multigraphs and pseudographs. An adjacency matrix is defined by:

Definition 2.19.
An adjacency matrix A_G of a graph G, is the $n \times n$ (where $n = |V|$) zero-one matrix, with 1 as it's a_{ij} entry when v_i and v_j are adjacent, and 0 when they are not adjacent. An adjacency matrix of a graph is based on the ordering of its vertices.

Note that when multiple edges are present between two adjacent vertices, the adjacency matrix is no longer a zero-one matrix. and it's a_{ij} entry becomes the number of edges between two adjacent vertices. There are up to $n!$ different adjacency matrices for the same graph, since there are $n!$ different ways of ordering the vertices.

Example 2.20. Adjacency matrix of a given simple graph



The adjacency matrix of the simple graph, where we order the vertices v_1, v_2, v_3 and v_4, is given by:

$$\begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \end{array} \begin{array}{c} \begin{array}{cccc} v_1 & v_2 & v_3 & v_4 \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{array}$$

From the adjacency matrix we retrieve the following information:
The vertex v_1 is adjacent to the vertices v_2 and v_4.
The vertex v_2 is adjacent to the vertices v_1 and v_3.
The vertex v_3 is adjacent to the vertices v_2 and v_4.
The vertex v_4 is adjacent to the vertices v_1 and v_3.
Therefore we can describe a graph by an adjacency matrix.

2.3 Paths in Graphs

When using graphs to model problems, many solutions rely on being able to travel along the edges from vertex to vertex. The idea of being able to travel through vertices using edges connecting these vertices is called following a path.

2.3.1 Paths

A path begins at a given vertex in a graph and traverses edges from vertex to vertex along the graph. The path visits vertices that are connected to the edges it traverses due to these vertices being the endpoints of the edges it traverses.

A more precise definition of a path is given by the definition 2.21:

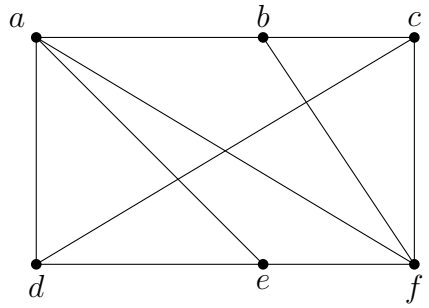
Definition 2.21.

Let n be a nonnegative integer and G an undirected graph. A path of length n from u to v in G is a sequence of n edges $e_1, \ldots, e_n$ of G for which there exists a sequence $x_0 = u, x_1, \ldots, x_n = v$ of vertices such that e_i has, for $i = 1, \ldots, n$, the endpoints x_{i-1} and x_i. When the graph is simple, we denote this path by its vertex sequence $x_0, x_1, \ldots, x_n$ (because listing these vertices uniquely determines the path). The path is a circuit if it begins and ends at the same vertex, that is, if $u = v$, and has length greater than zero. The path or circuit is said to pass through the vertices $x_1, x_2, \ldots, x_n - 1$ or traverse the edges $e_1, e_2, \ldots, e_n$. A path or circuit is simple if it does not contain the same edge more than once.

When using paths in graphs to solve problems, it is important to be aware whether the path starts at one vertex v and ends at another vertex u, or if it starts and ends at the same vertex, that is $v = u$. Such a path that connects one vertex to itself through a sequence of other vertices is called a circuit. It is useful to note that for any given edge in a path, not including the edge terminating in the vertex that the path terminates at, the terminal

vertex will be the initial vertex for the next edge traversed in the path.

Example 2.22. Paths in a simple graph



The graph G contains a variety of different paths. As an example a, e, f, c, b is a path of length 4, because the given 4 edges exist between the vertices in the path: $\{a, e\}$, $\{e, f\}$, $\{f, c\}$ and $\{c, b\}$. The path a, b, d, e does not exist because no edge exists in the graph that connects the vertices b and d, namely: $\{b, d\} \notin E$. The path a, b, f, e, a is a path of length 4 because the edges: $\{a, b\}$, $\{b, f\}$, $\{f, e\}$ and $\{e, a\}$ exists. Moreover this path is a simple circuit because it starts and ends at the vertex a. The path e, d, c, f, e, d is a path of length 5 because the edges: $\{e, d\}$, $\{d, c\}$, $\{c, f\}$, $\{f, e\}$ and $\{e, d\}$ exists. This given path is not simple because it contains the edge $\{e, d\}$ twice.

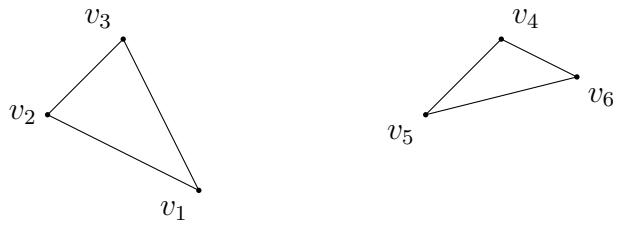
2.3.2 Connectivity

When following paths it is important to know if all vertices are connected to all other vertices through a path of some length n. When a path can be constructed between any two vertices of a given graph, then this graph is said to be connected.

Definition 2.23.
A connected undirected graph is a graph where a path exists between any two distinct vertices. If a graph is not connected, it is said to be disconnected.

When the set of vertices contains at least one vertex that cannot be included in any path that visits all other vertices, the graph is said to be disconnected. All disconnected graphs are able to be divided into separate connected graphs by creating a subgraph existing of vertices able to be connected by a path to any other vertex in the subgraph. When a connected subgraph is the largest possible connected subgraph for a set of vertices in the main graph and it is not a subgraph of any other connected subgraph of the main graph it is said to be a connected component.

Example 2.24. A disconnected graph G



G_1 G_2

The graph G is disconnected, because vertices not connected to all other vertices by paths exists, e.g. no paths connect v_1 to v_5. G can be divided into two connected components, namely G_1 and G_2. Therefore no vertices in G_1 can be connected to any vertex in G_2 by a path, and likewise no vertices in G_2 can be connected to any vertex in G_1 by a path.

2.3.3 Weighted Graphs

A weighted graph, is a graph where each edge has a designated value or number. Since paths traverses edges with assigned values, there are several more possibilities for the use of paths. A weighted graph is therefore used to solve many different problems, some of these problems originate when a path with the least total value must be found, which is called the shortest path.

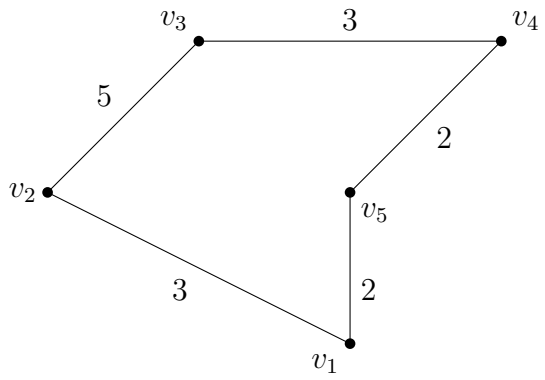
Definition 2.25.
A weighted graph is a graph where each edge is assigned a value. The total weight of a path in a weighted graph, is calculated by the sum of the weight of all the passed edges.

When finding the shortest path between 2 vertices in a weighted graph G, algorithms can be utilized to discover the shortest path between these vertices. The shortest path in a weighted graph depends on the direction of the edges in the graph, if the given graph is a directed graph.

Definition 2.26.
The shortest path from a vertex v to a vertex u, is the path connecting v to u with the least total weight in the weighted graph G.

By this definition, the shortest path from a vertex v to a vertex u is not necessarily the path with the least amount of edges which can be seen in example 2.27.

Example 2.27. The shortest path in a simple weighted graph G.

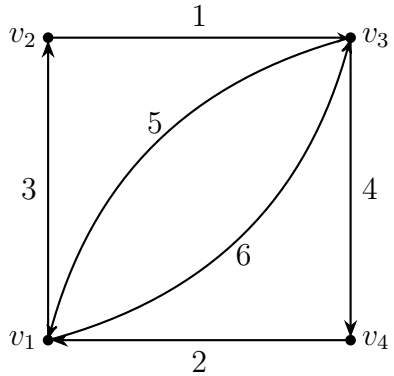


In the graph illustrated above it can be seen, that the shortest path from v_1 to v_3 is the path along the edges, $\{v_1, v_5\}$, $\{v_5, v_4\}$ and $\{v_4, v_3\}$. This means that the shortest path from v_1 to v_3 (or v_3 to v_1), is a path of length 7. We see that the shortest path is not the path containing the least number of edges, because the path containing the edges $\{v_1, v_2\}$ and $\{v_2, v_3\}$ is the path containing the least edges between v_1 and v_3 of length 8.

In the example above, the shortest path was discovered by examining all paths from v_1 to v_3 in the weighted graph. If the graph had thousands of vertices and edges it would be tedious to examine all paths. In this case a greedy algorithm by Edsger Dijkstra called Dijkstra's algorithm could be used. Dijkstra's algorithm, will be explained in 0.

Sometimes we wish to find the shortest path from a vertex v to another vertex u in a directed simple graph. Therefore, we are only allowed to go from an initial vertex to a terminal vertex, when finding the shortest path.

Example 2.28. The shortest path of a simple directed weighted graph G.



In this graph we see that, that the shortest path from v_1 to v_3 is the path along the directed edges from v_1 to v_2 and from v_2 to v_3. This means that the shortest path from v_1 to v_3, has a length of 4. Note that shortest path from v_3 to v_1 does not have the same length as from v_1 to v_3, as the path from v_3 to v_1 which has a length of 5.

3 | Methods of Proofs

When making statements in mathematics, these statements often generalize some property of different values. Therefore it is important to show that the given statement is true for any value of the specified domain that it is said to be valid for. The methods of proofs presented in this chapter are found in [1, p. 90 - 92] and [1, p. 331 - 338].

3.1 Proof by Induction

One way to prove the truth of a given proposition $P(n)$ for any positive integer n, is mathematical induction. In order to perform mathematical induction a few basic principles must be met: Firstly it must be possible to show the truth of the proposition using the first value in the domain for which the statement holds. Additionally it must be possible to find the next consecutive step after any given step that is known. The foundation of mathematical induction lies on these two principles respectively called the basis step and the inductive step. When $P(n)$ is true for the first integer in the domain, and it is also true for any given integer $k > n$, then it needs to be shown that $P(k+1)$ is also true in order to construct the inductive proof. This can be written as [1, p. 333]:

$$(P(1) \wedge \forall k(P(k) \rightarrow P(k+1))) \rightarrow \forall n P(n). \tag{3.1}$$

3.1.1 Reasons for Using Mathematical Induction

The reason why mathematical induction is a valid proof technique, is due to the axiom called the well-ordering property. The well-ordering property, which is an axiom for the set of positive integers specifies that every nonempty subset of the set of positive integers has a least element. The axiom does not provide any method for finding the least element nor does is define what element is the smallest therefore the axiom can be described as an existence axiom.

This axiom can be used to show that mathematical induction is a valid proof technique. To show that the axiom can be used to show the validity of mathematical induction we start by assuming that the statement $P(1)$ is true and that the conditional proposition $P(k) \rightarrow P(k+1)$ is true.

We want to show that $P(n)$ is true for all positive integers n. To prove this we assume that there exists a set S that contains a positive integer n for which the proposition $P(n)$ is false. If there exists a such positive integer n for which the statement is false, in the set S, this would mean that the set S is nonempty.

By the well-ordering axiom, this means that the set S has a least element, which is denoted m. Due to $P(1)$ being true, it must be the case that the element $m > 1$. Therefore there must exist a positive integer $m-1$, which is not in the set S, due to the well-ordering

property axiom. Thus, the statement $P(m-1)$ must be true and the conditional statement $P(m-1) \rightarrow P(m)$ is thereby also true. If the conditional statement is true, it must also be the case that the statement $P(m)$ is true. This statement contradicts that there exists an element m in the set of positive integers for which $P(n)$ is not true, leading to the conclusion that $P(n)$ must be true for all positive integers.

Before mathematical induction is used to prove conjectures, a number of points has to be made, because there are advantages and disadvantages. The advantage of using mathematical induction is that it can be applied to prove a conjecture if it is true. The disadvantage of using mathematical induction is that it cannot be used as a technique of finding new conjectures. Another disadvantage is that mathematical induction can be used to show that a conjecture is true even though it does not provide any useful insight as to why the conjecture is true. Therefore it should be considered whether it is preferable to use another technique of proof due to the insight which they can provide along the way.

3.1.2 Constructing a Proof by Mathematical Induction

The basis step is the first step of proof by induction. In the basis step the least element in the domain, from the statement's domain is inserted into the statement. When a proposition is created a basis step can be constructed as shown in 3.3.

Choosing the Correct Basis Step

The first step in choosing the correct value for the basis step is to choose the first number from the statement's domain. The first number can be negative, zero or positive. When the first number from the domain has been chosen, it is inserted in the proposition $P(n)$ instead of n. Then we calculate the statement in order to prove the basis step. To show an example of this, the proposition which can be found in 3.2 is used given by: $P(n) : \frac{n(n+1)}{2}$. The first element in domain of the statement is 1. Since it is the first positive integer it is inserted into the proposition which gives the correct basis step: $P(1) : \frac{1(1+1)}{2} = 1$.

Example 3.1. We will use induction to prove the summation formula. Here we have to prove that for all positive integers n we can use the following formula:

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2} \tag{3.2}$$

Proof. Firstly we create a proposition $P(n)$ that the sum of the first n positive integers, can be calculated with the formula from 3.2. To begin the proof by induction it is at first shown that the basis step $P(1)$ is true. The proposition $P(k)$ is assumed to be true and $P(k+1)$ has to be proven to be true on the basis of this assumption.

Basis Step: First we check if $P(1)$ is true:

$$1 = \frac{1(1+1)}{2}. \tag{3.3}$$

Since the sum of the first positive integer is 1, we see that the basis step $P(1)$ is true.

Inductive Step We now assume that the conditional statement $P(k)$ holds for a positive integer k. Therefore the following is assumed to be true:

$$1 + 2 + \cdots + k = \frac{k(k+1)}{2}. \tag{3.4}$$

From this assumption we have to show that $P(k+1)$ is true. Hence we insert $(k+1)$ in the formula 3.2:

$$1 + 2 + \cdots + k + (k+1) = \frac{(k+1)((k+1)+1)}{2}. \tag{3.5}$$

First we observe that the summation on the left hand side of 3.5 is $(k+1)$ greater than the summation on the left hand side of 3.4. The way we show 3.5 is true, is to add $(k+1)$ to both sides of the equation in 3.4:

$$1 + 2 + \cdots + k + (k+1) = \frac{k(k+1)}{2} + (k+1). \tag{3.6}$$

We multiply by $\frac{2}{2}$ on the second term on the right side of the equation in 3.6:

$$1 + 2 + \cdots + k + (k+1) = \frac{k(k+1) + 2(k+1)}{2}. \tag{3.7}$$

We factorize $(k+1)$ in the numerator on the right side in 3.7.

$$1 + 2 + \cdots + k + (k+1) = \frac{(k+1)(k+2)}{2}. \tag{3.8}$$

The last equation shows that $P(k+1)$ is true, under the assumption that the conditional statement $P(k)$ is true. Therefore we have finished the inductive step. Since we have completed the basis and inductive step, we have proven that the proposition $P(n)$ is true for all positive integers. ■

Example 3.2.
We will use induction to prove The Binomial Theorem. Here it is proved that for all positive integers n the following formula holds A.2:

$$(a+x)^n = a^n + na^{n-1}x + \frac{n(n-1)}{2!}a^{n-2}x^2 + \cdots + nax^{n-1} + x^n = \sum_{k=0}^{n} \binom{n}{k} a^{n-k}x^k, \quad (3.9)$$

where

$$\binom{n}{k} = \frac{n!}{(n-k)!k!}.$$

Proof. To begin the proof, $P(n)$ is defined.

$P(n)$: equation 3.9 holds for all $n \in \mathbb{N}$.

Then the basis step $n = 1$ is chosen, since it is the first number in the domain of natural numbers.

Basis Step: $P(1)$ is confirmed to be true:

Left-hand side of the equation:

$$(a+x)^1 = a + x \quad (3.10)$$

Right-hand side of the equation:

$$\sum_{k=0}^{n=1} \binom{n}{k} a^{n-k}x^k = \binom{1}{0} a^{1-0}x^0 + \binom{1}{1} a^{1-1}x^1 = \frac{1!}{(1-0)!0!} \cdot a + \frac{1!}{(1-1)!1!} \cdot x = a + x. \quad (3.11)$$

Since $a + x = a + x$, the basis step has been proven.

Inductive Step

In the inductive hypothesis we replace k by m due to k already being present in The Binomial Theorem. We assume $P(m)$ is true:

$$(a+x)^m = \sum_{k=0}^{m} \binom{m}{k} a^{m-k}x^k \quad (3.12)$$

We now wish to prove that $P(m+1)$ is true:

$$(a+x)^{m+1} = \sum_{k=0}^{m+1} \binom{m+1}{k} a^{(m+1)-k} x^k, \tag{3.13}$$

This can be rewritten as:

$$(a+x)(a+x)^m = \sum_{k=0}^{m+1} \binom{m+1}{k} a^{m-k+1} x^k. \tag{3.14}$$

Now we focus on the left hand side of the equation, which can be rewritten using The Binomial Theorem:

$$\sum_{k=0}^{m} \binom{m}{k} a^{m-k} x^k (a+x). \tag{3.15}$$

Then $(a+x)$ is factorized from the sum:

$$a \sum_{k=0}^{m} \binom{m}{k} a^{m-k} x^k + x \sum_{k=0}^{m} \binom{m}{k} a^{m-k} x^k. \tag{3.16}$$

Then a and x are multiplied into their respective factors:

$$\sum_{k=0}^{m} \binom{m}{k} a^{(m+1)-k} x^k + \sum_{k=0}^{m} \binom{m}{k} a^{m-k} x^{k+1}. \tag{3.17}$$

Afterwards the index of the right hand summation is shifted:

$$\sum_{k=0}^{m} \binom{m}{k} a^{(m+1)-k} x^k + \sum_{k=1}^{m+1} \binom{m}{k-1} a^{m-(k-1)} x^k. \tag{3.18}$$

In order for the summations to share the same index, the term $k = 0$ is taken out of the left hand summation, and the term $k = m+1$ is taken out of the right hand summation:

$$\begin{aligned} &\binom{m}{0} a^{(m+1)-0} x^0 + \binom{m}{m+1-1} a^{m-(m+1-1)} x^{(m+1-1)+1} \\ &\quad + \sum_{k=1}^{m} \binom{m}{k} a^{(m+1)-k} x^k + \sum_{k=1}^{m} \binom{m}{k-1} a^{m-(k-1)} x^k, \end{aligned}$$

which can be reduced to:

$$a^{m+1} + x^{m+1} + \sum_{k=1}^{m} \binom{m}{k} a^{m+1-k} x^k + \sum_{k=1}^{m} \binom{m}{k-1} a^{m-k+1} x^k. \tag{3.19}$$

Pascals Identity can be used to combine the summations. Pascals Identity is given by [2, p. 559]:

$$\binom{n}{r} + \binom{n}{r-1} = \binom{n+1}{r}, \tag{3.20}$$

when $0 < r \leq n$. We obtain:

$$a^{m+1} + x^{m+1} + \sum_{k=1}^{m} \binom{m+1}{k} a^{m-k+1} x^k. \tag{3.21}$$

To show that the 0th term still holds the following is confirmed:

$$a^{m+1} = \binom{m+1}{0} a^{m+1-0} x^0. \tag{3.22}$$

Likewise the $(m+1)$st term is confirmed:

$$x^{m+1} = \binom{m+1}{m+1} a^{m+1-m+1} x^{m+1}. \tag{3.23}$$

Then the terms $k = 0$ and $k = m+1$ are added to the summation and it now equal to the right-hand side of the equation 3.13:

$$\sum_{k=0}^{m+1} \binom{m+1}{k} a^{m-k+1} x^k = \sum_{k=0}^{m+1} \binom{m+1}{k} a^{m-k+1} x^k. \tag{3.24}$$

This proves the inductive step, hence proving The Binomial Theorem. ■

3.2 Proof by Contradiction

Another method for proving that a statement p is true, is proof by contradiction. Suppose there exists a statement $\neg p$ and it is possible to find a contradiction q in $\neg p$ such that

$\neg p \to q$ is true. The truth table of the conditional statement $\neg p \to q$, when q is false is illustrated below:

p	$\neg p$	q	$\neg p \to q$
T	F	F	F
F	T	F	T

Table 3.1: Truth table of the different truth values of the conditional statement $\neg p \to q$. (Note q can never be true, since it is a contradiction).

Because q is always false (since it is a contradiction) but $\neg p \to q$ is true, it must mean $\neg p$ is false, which means p is true. The next step is to find a contradiction q. Because q is a contradiction, it can be rewritten as $r \wedge \neg r$, since this is a contradiction when r is a proposition in the statement $\neg p$. If it is possible to show that $\neg p \to (r \wedge \neg r)$ is true for some proposition r, then it must mean, that the proposition p is true. The truth table below illustrates why p must be true, when $\neg p \to (r \wedge \neg r)$ is true:

p	$\neg p$	r	$\neg r$	$r \wedge \neg r$	$\neg p \to (r \wedge \neg r)$
T	F	T	F	F	T
T	F	F	T	F	T
F	T	T	F	F	F
F	T	F	T	F	F

Table 3.2: Truth table of the different truth values of the conditional statement $\neg p \to (r \wedge \neg r)$.

A proof by contradiction can be broken down into the following steps:

1. Define p.
2. Assume that $\neg p$ is true.
3. Find some contradiction q in $\neg p$.
4. Show that if a contradiction q exists in $\neg p$ then p must be true since $\neg p$ has to be false for $\neg p \to q$ to be true.

Example 3.3.
This proof by contradiction is based on [1, p. 90 - 91]

Proof. $\sqrt{2}$ is irrational
The proof begins by stating that p : "$\sqrt{2}$ is irrational".

Then it is assumed that $\neg p$ is true, which is $\neg p$: "It is not the case that $\sqrt{2}$ is irrational". This can be rewritten as $\neg p$: "$\sqrt{2}$ is rational". If the proposition $\neg p$ is

true, then there must exist integers a and b such that $\sqrt{2} = \frac{a}{b}$ where $a, b \in \mathbb{Z}, b \neq 0$ and a and b are in lowest terms.

Both sides of $\sqrt{2} = \frac{a}{b}$ are then raised to the power of 2, hence we obtain:

$$2 = \frac{a^2}{b^2}. \tag{3.25}$$

It is then possible to multiply with b^2 on both sides of the equation, to isolate a:

$$2b^2 = a^2. \tag{3.26}$$

Because $2b$ is always an even integer, we know that a must also be an even integer. It is therefore allowed to substitute a with $2c$ for some integer c:

$$2b^2 = 4c^2. \tag{3.27}$$

Both sides of the equation are now divided by 2:

$$b^2 = 2c^2. \tag{3.28}$$

Due to $2c$ always being a even integer we know that b must also be an even integer.

We started by assuming that $\neg p$: "$\sqrt{2}$ is rational" is true, but a contradiction has been found within $\neg p$ which can be written as q : "a and b must be in lowest terms for $\sqrt{2}$ to be a rational number". This is a contradiction, because a and b are both even integers, which means they are divisible with 2, and hence they are not in the lowest term. Therefore this must make it such that the proposition $\neg p$ is false since this is the only way the conditional statement $\neg p \rightarrow q$ is true. This implies that the proposition p : "$\sqrt{2}$ is irrational" is true, which completes the proof that $\sqrt{2}$ is an irrational number. ■

4 | Algorithms

Many problems can arise in mathematics, for example: given a set of integers, find the smallest integer in the set. To solve this kind of problem an algorithm can be used. An algorithm is a sequence of steps to produce an output. The definitions and Algorithms presented in this chapter can be found in [1, p. 201-241].

Definition 4.1.
An algorithm is a finite sequence of precise instructions for performing a computation or for solving a problem.

An algorithm can be represented by a pseudocode. A pseudocode that describing an algorithm, which finds the smallest element in a finite sequence can be seen in the example below:

Algorithm 1 Algorithm for finding the smallest element in a finite sequence

```
1: procedure min(a_1, a_2, ..., a_n: integers)
2: min := a_1
3:     for i = 2 to n do
4:         if min > a_i then
5:             min := a_i
6: return min{min is the smallest element}
```

Example 4.2. Finding the smallest element in a finite sequence:

$$7, 2, 1, 4, 6, 3$$

First the algorithm chooses 7 as the smallest element min. It then compare this element, to the second element in the sequence which is 2. min is then set to be 2, because $2 < 7$. The new minimum is then compared to the third element in the sequence, which is the element 1. min is then set to be 1, because $1 < 2$. It then compare the new min individually with the rest of the elements in the sequence. Since $min = 1$ is less than any other element in the sequence, it knows that 1 is the smallest element in the finite sequence.

When we are working with algorithms, there are several properties we need to know. These properties are:

1. *Input.* A specified set of values.
2. *Output.* What an algorithm returns after it's procedure.

3. *Definiteness.* The steps in an algorithm has to be precisely defined.
4. *Correctness.* An algorithm needs to produce the correct output, for each set of input values.
5. *Finiteness.* An algorithm should produce an output after a finite sequence of steps.
6. *Effectiveness.* Every step in the algorithm must be possible to perform, and in a finite amount of time.
7. *Generality.* The algorithm needs to be generally applicable to all problems of the desired type.

4.1 Types of Algorithms

This chapter covers *searching*, *sorting*, *greedy* and *brute force algorithms*.

4.1.1 Searching Algorithms

Searching algorithms are used in cases where we wish to find an element x in a list or conclude whether this element is present in the list. A *searching algorithm* is given by the following definition:

Definition 4.3.
A *searching algorithm* locates a given element x in a list consisting of distinct elements $a_1, a_2, \ldots, a_n$, or determines whether the element is not present in the list. The solution to the searching problem is i if $x = a_i$, when x is in the list. If x is not in the list the solution is 0.

The first *searching algorithm* we wish to present is the Linear Search Algorithm. This algorithm is also called the Sequential Search Algorithm. *The Linear Search Algorithm* is given by the following:

Algorithm 2 The Linear Search Algorithm

```
1: procedure linear search(x: integer, a_1, a_2, ..., a_n: distinct integers)
2: i := 1
3: while i ≤ n and x ≠ a_i do
4:     i := i + 1
5: if i ≤ n then location := i
6: else location := 0
7: return location{location is the subscript of the term that equals x, or is 0 if x is not
   found}
```

In the algorithm we observe that the Linear Search Algorithm requires an integer x and a list of distinct integers each respectively labeled with the numbers ranging from 1 to n, where n denotes the total amount of elements in the list. The algorithm then starts by setting the index i equal to 1. Then it starts comparing the element in the i'th position to x. If this element is not equal to x and the index is less or equal to n it then adds 1 to the index to compare the next element in the list to x. If x however is equal to the element in the i'th position it then sets the location variable equal to i. If no element in the list is equal to x the else statement is then used to set the location variable equal to 0.

Another type of *searching algorithm* is the Binary Search Algorithm. As opposed to the Linear Search Algorithm this algorithm relies on the list being ordered in increasing order. An example of this could be a list of integers in increasing order. This algorithm starts by comparing the element x to the element located in the middle of the list. Then the list is split into two smaller sublists and the same process is continued. The last element of the first sublist is then compared to x. If x is smaller than this element this sublist is split into two and if x is a larger integer the second sublist is kept. The pseudocode for the Binary Search Algorithm is shown below:

Algorithm 3 The Binary Search Algorithm

1: **procedure** *binary search*(x: integer, $a_1, a_2, \ldots, a_n$: increasing integers)
2: $i := 1$ {i denotes the minimum value in the search interval}
3: $j := n$ {j denotes the maximum value in the search interval}
4: **while** $i < j$ **do**
5: $m := \lfloor (i + j)/2 \rfloor$
6: **if** $x > a_m$ **then** $i := m + 1$
7: **else** $j := m$
8: **if** $x = a_i$ **then** *location* $:= i$
9: **else** *location* $:= 0$
10: **return** *location*{*location* is the subscript i of the term a_i equal to x, or 0 if x is not found}

In the pseudocode we see that The Binary Search Algorithm takes an integer x to search for and a list of integers ordered in increasing order. It then takes two values which consists of the first index of the search interval which is denoted i and the last index of the search interval denoted by j, which refers to the last element with index n in the search interval. While the left endpoint is less than the right endpoint (there are multiple elements left in the list) it takes a variable m and sets it equal to the floor function of the arithmetic average of i and j. The floor function takes an input x and gives the greatest integer less than or equal to x as output. This is the way The Binary Search Algorithm finds the middle element of the list. It then compares x to this middle term. If x is bigger than the middle term then it sets the left endpoint to the term one index after the middle term, by setting i equal to $m + 1$. If x is less than the middle term, then it sets the right endpoint

of the search interval equal to m. If x is equal to the i'th element then the location is set equal to i. If The Binary Search Algorithm goes through all elements without finding the element x, it then sets the location equal to 0.

An example of the usage of The Binary Search Algorithm is given by the following example:

Example 4.4. Searching for an element in a list using the Binary Search Algorithm

The algorithm wish to know if 169 is in the list of the first 16 square numbers:

1 4 9 16 25 36 49 64 81 100 121 144 169 196 225 256

First it split the list which has 16 elements into two smaller sublists containing 8 elements each:

1 4 9 16 25 36 49 64 81 100 121 144 169 196 225 256

Then it start by comparing 169 to the last element in the first list. Due to $169 > 64$, it now keep the last sublist and have therefore restricted the search for the element 169 to the sublist containing the 9th to 16th element of the original list. It then split the new list into two sublists containing 4 elements:

81 100 121 144 169 196 225 256

Now it compare 169 to the last element in the first sublist. Due to $169 > 144$, it keep the second sublist containing the 13th to 16th elements of the original list. We then split the list again:

169 196 225 256

Then 169 is compared to 196. Due to $169 < 196$, the first sublist containing the 13th and 14th element is kept and split into two:

169 196

The algorithm then compare 169 to 169 and find that 169 is the 13th element of the original list. The search is therefore complete, since it have found that the number 169 is the 13th element in the list and therefore in the 13th position.

4.1.2 Sorting Algorithms

Sorting algorithms are used to order a list of elements. The algorithms can solve problems where we have a list of elements and a way of ordering the list.

Definition 4.5.
A *sorting algorithm* is an algorithm where the purpose is to order a given list of elements in a given order.

In this section The Bubble Sort Algorithm which is capable of sorting lists will be introduced. The Bubble Sort Algorithm is used to sort a list of unordered elements. The algorithm orders a list of elements by comparing neighbouring elements and sorting them in increasing order. The algorithm works by interchanging a larger element with a smaller element, if and only if the smaller element follows the larger element. The algorithm always makes a full pass of the list which results in the largest number being at the end of the list. The procedure is iterated until the list is sorted in increasing order.

Algorithm 4 The Bubble Sort Algorithm

1: **procedure** *Bubble Sort*($a_1, \ldots, a_n$: real numbers with $n \geq 2$)
2: **for** $i := 1$ to $n - 1$ **do**
3: **for** $j := 1$ to $n - i$ **do**
4: **if** $a_j > a_{j+1}$ **then** interchange a_j and a_{j+1}
5: **return** $a_1, \ldots, a_n$ is in increasing order

In the pseudocode for The Bubble Sort algorithm we see that the algorithm starts off by examining the first element a_1 in a list of elements, then the algorithm compares the elements with its neighbouring element a_2. If the first element a_1 is the largest of the two elements, the two elements interchange position. The algorithm then takes the second element a_2 in the list, and compares it to its neighbour a_3, it then examines what element is the largest and interchanges the elements if the second element a_2 is the largest. This procedure is iterated over the whole list. After each iteration, only one element in the list is certain to be placed at the right spot, this element is the last element in the list. For the next iteration, the second to last element a_{n-1} is placed in the right position, because every iteration i places the $(n - i + 1)$'st element in the list in the correct position.

Example 4.6. The Bubble Sort Algorithm

The following list will be sorted using the bubble sort algorithm: $5, 7, 2, 4, 1$.

The algorithm starts by examining the first number in the list, which is 5. Then it compare 5 with the next number in the list, which is 7. Because $5 < 7$, we do not interchange the position of these elements. Then it compare 7 with the third element, which is 2. Because $7 > 2$ these elements interchange position, so the first 3 elements in the list are ordered as follows: $5, 2, 7$. Then it compare the fourth element in the list, which is 4 to the third element in the list which is 7. Because $7 > 4$ these elements interchange position. Then it compare the fifth element, which is 1 with the fourth element in the list which is 7. Because $7 > 1$ these elements interchange position.

After the first iteration with The Bubble Sort Algorithm the list has the following ordering: $5, 2, 4, 1, 7$. Because the number 7 is at the end of the list after the first iteration, it know that 7 is the greatest element in the list.

For the second iteration, it examine the first element in the new ordering of the list, which is 5. When 5 gets compared successively to the other elements in the list, it is possible to see that $5 > 2, 5 > 4, 5 > 1$ and therefore it ends in the fourth position, which results in a new ordering of the list: $2, 4, 1, 5, 7$.

For the third iteration the first element in the list is 2, and the second element is 4. Because $2 < 4$, the elements do not interchange position. We then compare the second element 4 to the third element of the list which is 1. Due to $4 > 1$, the elements interchange position. Therefore the list is ordered as follows: $2, 1, 4, 5, 7$.

A fourth iteration compares the first two elements in the list, which are 2 and 1, because $2 > 1$ they interchange position. The ordering is now complete because it have made $n - 1$ iterations of the list. The final list is ordered as follows: $1, 2, 4, 5, 7$.

The *Bubble Sort Algorithm*, is capable of sorting a list in increasing order as shown in example 4.6. It uses $n - 1$ iterations, because only one element in the list is certain to be placed in the right spot after every iteration.

4.1.3 Greedy Algorithms

Sometimes we run into optimization problems, and an example of this could be to find the shortest route between 2 cities. An approach to this type of problem, is to select the most optimal choice at each step, instead of considering all options at each step. This type of approach is utilized in *greedy algorithms*.

Definition 4.7.
A *greedy algorithm* finds a solution to a problem, by selecting the most optimal choice at each step.

When we use a *greedy algorithm* to solve a problem, we need to conclude whether it has found an optimal solution. We can determine this by proving that the specific algorithm always produces an optimal solution, or we can show that there exists a counterexample where the algorithm produces a non-optimal solution. An example of a *greedy algorithm* is found in [1, p. 210].

This algorithm takes a specified amount of money and finds the smallest amount of coins needed for an exchange.

Algorithm 5 Cashier's Algorithm

1: **procedure** *change*(c_1, c_2, …, c_r: values of denominations of coins, where $c_1 > c_2 > \cdots > c_r$; n: a positive integer)
2: **for** $i = 1$ to r **do**
3: $d_i := 0$ {d_i counts the coins of denomination c_i used}
4: **while** $n \geq c_i$ **do**
5: $d_i := d_i + 1$ {add a coin of denomination c_i}
6: $n := n - c_i$
7: **return** d_i {d_i is the number of coins in denomination c_i used in the change for $i = 1, 2, \ldots, r$}

Example 4.8. Using Cashier's Algorithm.
In the case that a cashier has to give back 3.68€ in change, 3.68 would be the input in the function along with the set $\{c_1, c_2, \ldots, c_8\}$ where $c_1 = 2$ euro, $c_2 = 1$ euro, $c_3 = 0.50$ euro, $c_4 = 0.20$ euro, $c_5 = 0.10$ euro, $c_6 = 0.05$ euro, $c_7 = 0.02$ euro and $c_8 = 0.01$ euro. Firstly we count how many 2 euro coins to use. To do that d_i is set to zero, as no 2 euro coins are counted yet. Then a loop will be entered where the condition for each iteration to be executed is that n (the remaining amount of change to be counted) needs to be greater than or equal to the current coin that is being counted (in the first loop in this example, this would be 2). In the loop 1 will be added to the number that indicates how many 2€ coins is to be given. Then 2 will be subtracted from n. When n is less than 2, which will be the second iteration, the while loop breaks, and we change the type of coin being focused on to be the largest coin that is smaller than the one currently used. In this case, it would be 1. This is then continued for every coin until n is smaller than the smallest coin. In this example this algorithm would return $\{1, 1, 1, 0, 1, 1, 1, 1\}$, where the value on the ith index refers to the number of coins on the ith index of c, needed to be returned as change. Therefore the most optimal combination of coins for returning 3.68€ in change is one 2€ coin, one 1€ coin, one 0.50€ coin, one 0.10€ coin, one 0.05€ coin, one 0.02€ coin and one 0.01€ coin.

4.1.4 Brute Force Algorithms

When using algorithms to find the optimal solution to a problem, the most straight-forward way to do so is usually by examining every single combination of choices, which is how *brute force algorithms* finds the most optimal solution to solve problems.

Definition 4.9.
A *brute force algorithm* is an algorithm that produces a solution by exhaustion, because it considers every possible solution one at a time.

The advantage of *brute force algorithms* is that they always produce the optimal solution, because they consider every possible solution. Furthermore they are usually easily understandable as their progression is linear. The trade-off of being precise and easily understandable is that a *brute force algorithm* is usually very time-consuming. When the sample size is large, this type of algorithm typically gets too slow for this trade-off to be acceptable. The *brute force algorithms* previously introduced are the algorithm for finding the smallest element in a finite sequence, The Linear Search Algorithm and The Bubble Sort Algorithm. Another example of a brute force algorithm is illustrated below:

Algorithm 6 Find Divisors

```
1: procedure Find Divisors(n: natural number)
2: Divisors := ∅
3: for i := 1 to n do
4:     if n modulus i = 0 then
5:         Divisors := Divisors ∪ {i}
6: return Divisors {Divisors is the set of all divisors of n}
```

This algorithm is a *brute force algorithm*, because it examines every whole number from 1 to n and checks if they are a divisor of n. It does not use any shortcuts to shorten the amount of time consumed by the algorithm. These shortcuts could be logical statements such as if the input is an odd number it does not need to examine any even numbers, which would lower the number of steps needed to solve a find divisors problem.

Example 4.10. Using a *brute force algorithm* to find divisors.

Consider that we wish to find the divisors of 9. Firstly we define the set of divisors to be the empty set, because no divisors are found yet. Then we examine the number 1 to see if 9 modulus $1 = 0$. As 9 is divisible by one, this statement will be true, and the number 1 is then added to the set of divisors. The same procedure is followed to examine the numbers $2, 3, 4, 5, 6, 7, 8$ and 9. Once that is done, all divisors of 9 will be in the set of divisors, such that $divisors = \{1, 3, 9\}$, which will be returned by the algorithm.

4.2 Shortest-Path Algorithms

A shortest-path problem is a problem where we wish to find the path with the least length from a given vertex to another given vertex. It could also be a problem such as the Travelling Salesperson problem, that asks for the circuit of the least length. We recall that the shortest path in a weighted graph is given by definition 2.26 and that the path of the shortest length is not the path with the least amount of edges but the path

with the least total sum of the weight of its edges. The algorithms used for solving this type of problem are called shortest-path algorithms. This type of algorithm is especially useful when examining connected weighted graphs. There exists several different shortest-path algorithms including Dijkstra's Algorithm and the A* Algorithm (pronounced A-Star Algorithm). In this project we will use Dijkstra's Algorithm to solve the shortest-path problems.

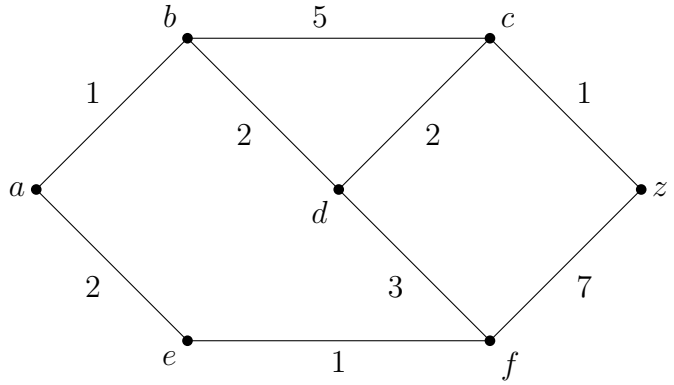
4.2.1 Dijkstra's Algorithm

Dijkstra's Algorithm solves a shortest-path problem, in connected weighted graphs, where all edges have been assigned a positive value.

Theorem 4.11.
When Dijkstra's Algorithm finds a path, between two vertices, it will always be a shortest path between these vertices.

Dijkstra's Algorithm, starts by examining the vertex that has been chosen as the starting vertex of a path in the graph. The algorithm's first iteration searches for all adjacent vertices, and chooses the vertex that has the path with the least length from the start vertex. Then the algorithm examines all vertices adjacent to the newly chosen vertex or the other previously chosen vertices (including the start vertex itself), it then chooses the next vertex with the shortest path from the start vertex to the end vertex. When the end vertex in the path is reached, we then know that the shortest path from the start vertex to the end vertex has been discovered.

Example 4.12. Using Dijkstra's Algorithm to find the shortest path



Given the simple weighted graph seen above, we wish to find the shortest path from the start vertex a to the end vertex z. Even though we could find the shortest path by simply inspecting the weighted graph, we wish to solve this problem using Dijkstra's Algorithm. The path that starts at the vertex a and visits exactly one other vertex, must contain a single edge that is incident to a. These paths that contain a single edge

are the paths a, b of length 1 and the path a, e of length 2. Therefore we conclude that the closest vertex to a is the vertex b.

We then proceed by finding the second closest vertex to a. We do this by examining all the paths that begin with the shortest path from the vertex a to a vertex in the set $\{a, b\}$, which is then followed by an edge that has one endpoint in the set $\{a, b\}$ and one endpoint not in the set. There are three paths to consider which are a, b, c of length 6, a, b, d of length 3 and a, e of length 2. The second closest vertex to a is e, and the shortest path between the two vertices a, e has length 2.

To find the third closest vertex to a we examine paths that begin with the shortest path from the vertex a to a vertex in the set $\{a, b, e\}$, which is followed by an edge that has one endpoint at a vertex in the set and one endpoint at a vertex not in the set. There are three paths to examine a, b, d of length 3, a, e, f of length 3 and a, b, c of length 6. Therefore we start by choosing the a, b, d of length 3 even though we have a tie. This will only result in the a, e, f being the next path chosen. Therefore we choose d as the third closest vertex to the vertex a.

We now examine the paths a, b, d, f with length 6, a, b, d, c with length 5, a, b, c with length 6 and a, e, f with length 3. We therefore choose f as the fourth closest vertex to a.

We now examine the paths a, e, f, d with length 6, a, e, f, z with length 10, a, b, d, f with length 6, a, b, d, c with length 5 and a, b, c with length 6. Therefore we choose c as the fifth closest vertex to a.

We now examine the paths a, b, d, c, z with length 6, a, e, f, d with length 6, a, e, f, z with length 10, a, b, d, f with length 6 and a, b, c with length 6. We therefore choose z as the sixth closest vertex to a. Because the path a, b, d, c, z contains the vertex z this path of length 6 is the shortest path between the vertex a and z.

The pseudocode of Dijkstra's Algorithm for finding the shortest in connected weighted simple graph is given by:

Algorithm 7 Dijkstra's Algorithm

1: **procedure** *Dijkstra*(G: weighted connected simple graph, all weights positive)
2: {G has vertices $a = v_0$, v_1, ..., $v_n = z$ and lengths $w(v_i, v_j)$
3: where $w(v_i, v_j) = \infty$ if $\{v_i, v_j\}$ is not an edge in G}
4: **for** $i := 1$ to n **do** $L(v_i) := \infty$
5: $L(a) := 0$
6: $S := \emptyset$
7: {The labels are now initialized so that the label of a is 0 and all others labels are ∞, and S is the empty set}
8: **while** $z \notin S$ **do**
9: $u :=$ a vertex not in S with $L(u)$ minimal
10: $S := S \cup \{u\}$
11: {this adds a vertex to S with minimum label}
12: **for** all vertices v not in S **do**
13: **if** $L(u) + w(u, v) < L(v)$ **then** $L(v) := L(u) + w(u, v)$
14: {this updates the labels of vertices not in S}
15: **return** $L(z)$ {$L(z)$ = length of a shortest path from a to z}

The algorithm requires a way to keep track of lengths from the starting vertex to other specific vertices. In the i'th iteration, the length from the starting vertex to a vertex v can be denoted by $L_i(v)$. The algorithm starts by labelling the vertex $a = v_0$ with 0 and all the other vertices in the graph with ∞. In the 0'th iteration we use the notation $L_0(a) = 0$ to label the shortest path from a to a and the notation $L_0(v_i) = \infty$ to label the length from a to any other vertex. These notations are used when the path only contains the vertex a. The reason for labeling all vertices other than a with ∞, is that we do not yet have any path to any other vertex in the graph. The set S is set to $S = \emptyset$ in the 0'th iteration. This is because, we have not yet added a to the set of vertices S.

While the sought vertex (the vertex we wish to find the distance to from the vertex a) has not been added to S, we continue iterating over the following steps: u is set to be a vertex not in S with the shortest distance to a, denoted by $L(u)$. We then add the element u to the set S. For all vertices v not in S we make a comparison. *If* the length from a to u added to the length of the edge from u to v is shorter than the currently known length from a to v, then we set the length from a to v equal to the length from a to u added to the length of the edge from u to v. This updates the labels of vertices not in S.

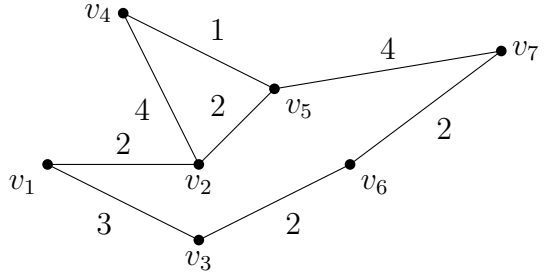
When the next iteration start, we know that the length $L(v)$ denotes the shortest distance from a to v. The vertex v is then set to u, because it has the shortest length to a of the vertices not in S. We then add this u to the set S, and enter the for loop again. We continue iterating until, we have added the sought vertex z to S, which is the set that contains the vertices where we know a shortest path from a to these vertices.

The properties of algorithms introduces in chapter 4, are applicable to Dijkstra's Algorithm.

1. *Input.* A specified weighted connected simple graph containing a finite number of vertices and edges, where the weight of all edges are positive.
2. *Output.* The length of a shortest path from a to z.
3. *Definiteness.* The steps in Dijkstra's Algorithm are defined in the pseudocode for Dijkstra's Algorithm.
4. *Correctness.* Dijkstra's Algorithms output is always correct, when all weights of the edges are positive. This will be proven in 4.2.2.
5. *Finiteness.* If the number of vertices and edges are finite, then Dijkstra's Algorithm uses a finite number of steps, to find the shortest path from a to z.
6. *Effectiveness.* If the number of vertices and edges are finite, then Dijkstra's Algorithm uses a finite amount of time, to find the shortest path from a to z.
7. *Generality.* Dijkstra's Algorithm finds the shortest path from a to z in any weighted connected simple graph containing a finite number of vertices and edges, where the weight of all edges are positive.

To illustrate the procedure given in the pseudocode for Dijkstra's Algorithm, it is used to find the shortest path between two vertices in example 4.13.

Example 4.13. Use of Dijkstra's algorithm in a undirected connected weighted graph Consider the undirected connected weighted graph $G = (V, E)$. This graph G is represented visually below:



To find the shortest path from v_1 to v_7 in this graph, the following steps are computed:

- $L(v)$ is set to ∞ for v_2, v_3, v_4, v_5, v_6 and v_7. $L(v_1)$ is set to 0.
- v_1 is the shortest path to v_1, therefore v_1 is added to the set S. Now lengths of the shortest paths to the vertices neighbouring a vertex in S are updated, such that $L(v_2) = 2$ and $L(v_3) = 3$.
- Since v_2 is the vertex not in S with the shortest length, it is added to S. Again lengths are updated for newly discovered vertices: $L(v_4) = 6, L(v_5) = 4$.

- Of all currently known shortest paths the path from v_1 to v_3 is the path of least length, and v_3 is therefore added to S. Now we set $L(v_6) = 5$.
- v_5 with $L(v_5) = 4$ is added to S. The length of v_7 is set to $L(v_7) = 8$. A shorter path from v_1 to v_4 has been found, namely v_1, v_2, v_5, therefore $L(v_4) = 5$.
- Now $L(v_4) = L(v_6) = 5$, therefore it does not matter which one is added to S. For the sake of the example we choose v_4 to be added to S. Now no new shortest paths are discovered or updated.
- Now v_6 is added to S. The length $L(v_7) = 7$ is updated
- v_7 is added to S.
- The sought vertex is now in S, and we can therefore trace back which vertices where visited in this path that lead to this being the shortest path, namely $\{v_1, v_3, v_6, v_7\}$.

To illustrate the procedure followed to produce this result, this table provides the current value of $L(v)$ for all $v \in V$ in each iteration:

Iterations	$L(v_1)$	$L(v_2)$	$L(v_3)$	$L(v_4)$	$L(v_5)$	$L(v_6)$	$L(v_7)$
0	*0*	∞	∞	∞	∞	∞	∞
1	**0**	*2*	*3*	∞	∞	∞	∞
2	0	**2**	3	*6*	*4*	∞	∞
3	0	2	**3**	6	4	*5*	∞
4	0	2	3	*5*	**4**	5	*8*
5	0	2	3	**5**	4	5	8
6	0	2	3	5	4	**5**	*7*
7	0	2	3	5	4	5	**7**

Table 4.1: Overview of shortest known paths for each iteration of Dijkstra's Algorithm. Numbers written in a bold font denotes that the given vertex is added to S. Numbers written in italic font indicates that the shortest known path is updated for the given vertex.

4.2.2 Proof that Dijkstra's Algorithm Finds A Shortest Path

We will prove theorem 4.11, which will be done by using induction. This proof is based on the proof in [1, p. 748-749].

Proof.
To prove that Dijkstra's Algorithm always finds a shortest path, we use an inductive hypothesis. The inductive hypothesis consists of the following assumptions about the

vertices, depending on whether $v \in S$ or $v \notin S$:

1. The label of every vertex $v \in S$, is the length of the shortest path from our initial vertex a to this vertex.
2. The label of every vertex $v \notin S$, is the length of the shortest path from our initial vertex a that contains only (besides the vertex itself) vertices in S.

Before we start the basis step, a proposition needs to be defined. The proposition $P(n)$ states: the inductive hypothesis holds, at the n'th iteration of Dijkstra's algorithm.

Basis Step: First we check if $P(0)$ is true.
In the basis step, we begin at iteration 0. Here we set the label of a to 0. This means the set of vertices $S = \emptyset$. This is because the path of a to itself does not contain any vertices. Afterwards we set the length from a to any other vertex to ∞. Which we know is true. This completes the basis step.

Inductive Step: Secondly we prove $P(k+1)$ is true whenever $P(k)$ is true.
We now assume that the proposition $P(k)$ holds for the k'th iteration. Then we check if the proposition $P(k+1)$ also holds for the $(k+1)$st iteration. At the $(k+1)$st iteration we add v to the set S. This means v was not in the set S at the k'th iteration, but it had the label that denoted the path of the least length. From the 2 assumptions in the inductive hypothesis, we know that the vertices in S at the $(k+1)$st iteration, are labeled with a shortest path from a to the vertices in S.

We also label v with the length of a shortest path from a to this vertex. If this was not the case this would result in, a path with a length less than $L_k(v)$ existing with a vertex not in S, at the end of the k'th iteration. We let u be a vertex not in S in that path. This results in a path from a to u only containing vertices in S, which is shorter than the path to v, and therefore u should have been chosen instead of v at the k'th iteration. This is a contradiction of the choice of v, which proves by contradiction that the first assumption is true at the $(k+1)$st iteration.

We let the vertex u be a vertex which is not in the set S, after $(k+1)$ iterations. We now have to find a shortest path from a to u. This path contains only elements in S but it either contains v or it does not. If a shortest path does not contain v, then the length of a shortest path will be $L_k(u)$ by our second assumption about the inductive hypothesis. If a shortest path contains the vertex v, then this path has the length: $L_k(v)$ added to the length of the edge from v to u which can be written as $w(v,u)$. Therefore our second assumption in the inductive hypothesis is true because: $L_{k+1}(u) = min\{L_k(u), L_k(v) + w(v,u)\}$. ■

Since we have proven that the 2 assumption in the inductive hypothesis are true. We have proven that Dijkstra's Algorithm produces a shortest path between any two chosen vertices in an connected weighted simple graph.

4.3 Complexity

When comparing algorithms this is usually done by comparing the complexity of these. In order to understand and define the complexity of an algorithm, it is necessary to understand the growth of functions, because the complexity of algorithms is based on certain comparisons of growth of functions compared to a catalogue of simple functions.

4.3.1 Big-O Notation

When we wish to describe the growth of a function the big-O notation is often used. This notation is given by the following definition and provides an upper bound for the growth of a given function.

Definition 4.14. Big-O Notation
Let f and g be functions from the set of integers or the set of real numbers to the set of real numbers. We say that $f(x)$ is $O(g(x))$ if there are constants C and k such that:

$$|f(x)| \leq C|g(x)|$$

whenever $x > k$. [This is read as "$f(x)$ is big-oh of $g(x)$."]

This definition tells us that the function $f(x)$ is big-O of the function $g(x)$ if the absolute value of the function $f(x)$ is equal to or less than the product of C and the absolute value of the function $g(x)$ as x grows without bound and is greater than the constant k. This means that $f(x)$ grows slower than or as fast as the product of $g(x)$ and some constant C as x grows without bound. In this relationship between the two functions, the constants C and k are called witnesses if their assigned value makes $f(x)$ big-O of $g(x)$. To show that the relationship $f(x)$ is $O(g(x))$ is true, we only need one pair of witnesses is required that makes it such that $|f(x)| \leq C|g(x)|$, whenever $x > k$.

Theorem 4.15.
Let $x > 1$, then

$$x^n > 1,$$

when $n \in \mathbb{N}$.

We now wish to prove this theorem by induction.

Proof.
We start the proof by defining our proposition $P(n) : x^n > 1$, when $x > 1 \wedge n \in \mathbb{N}$.

The basis step:
When proving the basis step, $n = 1$ is used:

$$x^1 > 1, \tag{4.1}$$

hence

$$x > 1. \tag{4.2}$$

Since we already defined $x > 1$, we know equation 4.1 is true, which proves the basis step.

The inductive step:
To prove the inductive step, we firstly assume that $P(k) : x^k > 1$ is true. To prove $P(k+1)$ we multiply by x on each side of the inequality, by which we obtain:

$$x^{k+1} > x \tag{4.3}$$

Since we know that $x > 1$, the following statement must be true:

$$x^{k+1} > 1, \tag{4.4}$$

whenever $x > 1$, this proves that $P(k + 1)$ is true. Since both the basis step and the inductive step has been proven, we can conclude that $P(n) : x^n > 1$ is true, whenever $x > 1 \wedge n \in \mathbb{N}$. ■

Example 4.16. Big-O Notation to describe the growth of a function

Show that $f(x) = x^3 + 2x^2 + x + 2$ is $O(x^3)$.

To show that $f(x)$ is $O(x^3)$ we want to find constants C and k such that $|x^3 + 2x^2 + x + 2| \leq C|x^3|$, when $x > k$.

We first note that we can estimate the size of $f(x)$ when $x > 1$. We can start by multiplying both sides of the inequality by x which gives us, $x^2 > x$. We can multiply both sides by x again which gives us $x^3 > x^2$. This gives us the following inequality: $x^3 > x^2 > x > 1$. We can use this inequality to replace x^2 and x by x^3 in the function $f(x)$ to construct the following inequality:

$0 \leq x^3 + 2x^2 + x + 2 \leq x^3 + 2x^3 + x^3 + 2x^3 = 6x^3$

Furthermore we chose that $x > 1$ and we observe that $C = 6$. Hence $C = 6$ and $k = 1$ are witnesses such that $f(x)$ is $O(x^3)$.

Note that once we have found one pair of witnesses, then an infinite pair of witnesses to the relationship can be found to the relationship $f(x)$ is $O(g(x))$. The following proof will show why this is true.

Proof.
Assume that a pair of witnesses C and k exists such that $|f(x)| \leq C|g(x)|$, when $x > k$. To assert that infinitely many pairs of witnesses exist when C and k makes it such that $f(x)$ is $O(g(x))$ we construct a new pair of witnesses C' and k'. These new witnesses are chosen to satisfy the following conditions: $C' > C$ and $k' > k$. To show that these witnesses makes it such that $f(x)$ is $O(g(x))$, we note that $|f(x)| \leq C|g(x)| \leq C'|g(x)|$,

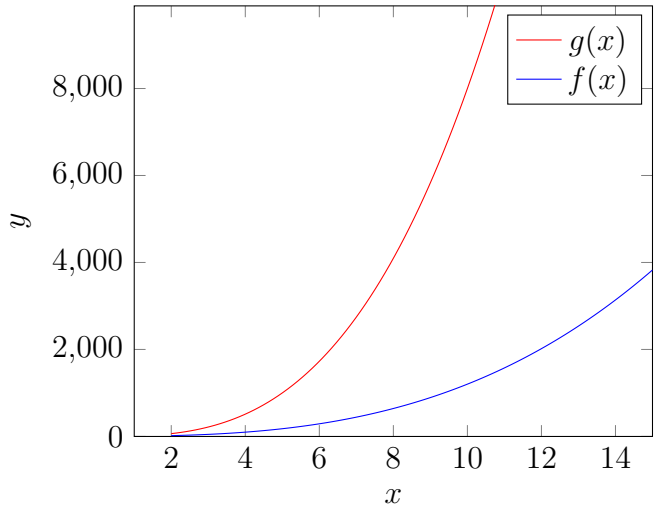
when $x > k' > k$. This is true since we have already stated that C and k are witnesses for the relationship $f(x)$ is $O(g(x))$ and that $C' > C$ and $k' > k$. This is due to the relationship $C|g(x)| \leq C'|g(x)|$ when $x > k' > k$, because the constant C' is greater than the constant C and that the absolute value of functions are identical, which makes the product of the right side equal to or greater than the left side of the relationship. Therefore it must be true that: $|f(x)| \leq C|g(x)| \leq C'|g(x)|$, when x grows without bound and the condition $x > k' > k$ is satisfied. ■

Therefore we can conclude that infinitely many witnesses exist such that $f(x)$ is $O(g(x))$, when one pair of witnesses has been found. We can apply this to the functions used in example 4.16 to find a new pair of witnesses in a new example.

Example 4.17. Constructing a new pair of witnesses such that $f(x)$ is $O(g(x))$

To construct a new pair of witnesses we choose two constants that satisfy the following conditions: $C' > C$ and $k' > k$. From example 4.16 we know that the witnesses $C = 6$ and $k = 1$ satisfy that $f(x) = x^3 + 2x^2 + x + 2$ is $O(x^3)$. We choose the witnesses $C = 8$ and $k = 3$, that satisfy the conditions. We then examine if the constants are witnesses such that $|f(x)| \leq C|g(x)|$, when $x > k$. We substitute the constants into the following inequality: $|x^3 + 2x^2 + x + 2| \leq 8x^3$, when $x > 2$. We can plot the two functions to see whether the inequality is true for $x > 2$.

Growth of the functions $f(x) = x^3 + 2x^2 + x + 2$ and $g(x) = 8x^3$, when $x > 2$



As illustrated in the plot the function $g(x)$ increases more rapidly than $f(x)$ as x grows without bound and satisfies the condition: $x > 2$. Therefore $C = 8$ and $k = 2$ is also a pair of witnesses such that $f(x)$ is $O(g(x))$.

Big-O estimates of polynomials

Instead of analyzing the growth of a polynomial each time we are presented with one, we would like a general rule that we can apply to estimate the growth of functions of this type. The following theorem asserts that the term with the highest degree in a given polynomial will always dominate the growth of the function such that a polynomial of degree n or less is $O(x^n)$.

Theorem 4.18. Big-O of polynomials
Let $f(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0$, where $a_0, a_1, \cdots, a_{n-1}, a_n$ are real numbers. Then $f(x)$ is $O(x^n)$.

The theorem can be proved using the triangle inequality from A.1 as follows if $x > 1$.

Proof.
We start by taking the absolute value on both sides of the equation of the polynomial with n as its highest degree:

$$|f(x)| = |a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0| \tag{4.5}$$

By the triangle inequality we know that taking the absolute value of the constants in front of each variable x will produce a higher or equal result than if it was the absolute value of the entire right side as seen in equation 4.5:

$$|a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0| \leq |a_n x^n| + |a_{n-1}x^{n-1}| + \cdots + |a_1 x| + |a_0| \tag{4.6}$$

We now use the rule that $|ab| = |a||b|$, which means that:

$$|a_n x^n| + |a_{n-1}x^{n-1}| + \cdots + |a_1 x| + |a_0| = |a_n||x^n| + |a_{n-1}||x^{n-1}| + \cdots + |a_1||x| + |a_0| \tag{4.7}$$

Because $x > 1$ we know that all terms including x are already positive which means we can remove the absolute value from all the factors containing x:

$$|a_n||x^n| + |a_{n-1}||x^{n-1}| + \cdots + |a_1||x| + |a_0| = |a_n|x^n + |a_{n-1}|x^{n-1} + \cdots + |a_1|x + |a_0| \tag{4.8}$$

We proceed by factorising x^n on the right side of the inequality:

$$|a_n|x^n + |a_{n-1}|x^{n-1} + \cdots + |a_1|x + |a_0| = x^n(|a_n| + \frac{|a_{n-1}|}{x} + \cdots + \frac{|a_1|}{x^{n-1}} + \frac{|a_0|}{x^n}) \tag{4.9}$$

We now wish to remove x from all the terms in the parentheses. To do this we wish to show that $\frac{|a_i|}{x^j} \leq |a_i|$, when $x^j > 1$. We start by examining this fraction when $|a_i| = 0$:

$$\frac{0}{x^j} = 0, \tag{4.10}$$

Therefore it must follow that:

$$\frac{|a_i|}{x^j} = |a_i|, \tag{4.11}$$

when $|a_i| = 0$.

We now examine the fraction when $|a_i| \neq 0$. We start by examining the inequality $x^j > 1$. We now multiply both sides by $|a_i|$:

$$|a_i|x^j > |a_i| \tag{4.12}$$

We now divide both sides by x^j:

$$|a_i| > \frac{|a_i|}{x^j}, \tag{4.13}$$

when $|a_i| \neq 0$.

Therefore it must follow that $\frac{|a_i|}{x^j} \leq |a_i|$, when $x^j > 1$. Thus we are able to remove all x terms in the denominator of the fractions in the parentheses. Hence it must hold that:

$$x^n(|a_n| + \frac{|a_{n-1}|}{x} + \cdots + \frac{|a_1|}{x^{n-1}} + \frac{|a_0|}{x^n}) \leq x^n(|a_n| + |a_{n-1}| + \cdots + |a_1| + |a_0|) \tag{4.14}$$

This results in the inequality:

$$f(x) \leq x^n(|a_n| + |a_{n-1}| + \cdots + |a_1| + |a_0|) \tag{4.15}$$

This shows that the witnesses $k = 1$ and $C = (|a_n| + |a_{n-1}| + \cdots + |a_1| + |a_0|)$ makes it such that $f(x)$ is $O(x^n)$, which completes the proof that the term of the highest degree in the polynomial dominates the growth of the function.

■

Growth of Combinations of Functions

When we wish to use the growth of functions to describe the complexity of algorithms it is important to note that some algorithms use several subprocedures that requires a different number of operations by a computer. It is therefore the sum of the steps required by these subprocedures that a computer requires to solve a problem with input of a given size. To give a big-O estimate for the number of steps required by the algorithm we need to introduce a way of achieving the big-O estimates of the sum of two functions. Furthermore we extend this to both finding the big-O estimates of the sum of two functions and finding the big-O estimate of the product of two functions. The big-O estimate of the sum of two functions is given by the following theorem:

Theorem 4.19. Big-O estimate of the sum of two functions
Suppose that $f_1(x)$ is $O(g_1(x))$ and that $f_2(x)$ is $O(g_2(x))$. Then $(f_1+f_2)(x)$ is $O(g(x))$, where $g(x) = (max(|g_1(x)|, |g_2(x)|)$ for all x.

This can be proved using the triangle inequality.

Proof. Proof of big-O estimate of the sum of two functions

We know from the definition of the big-O notation in 4.14 that there exists constants C_1, C_2, k_1 and k_2 such that:

$$|f_1(x)| \leq C_1|g_1(x)|, \tag{4.16}$$

when $x > k_1$, and

$$|f_2(x)| \leq C_2|g_2(x)|, \tag{4.17}$$

when $x > k_2$.

To estimate the sum of $f_1(x)$ and $f_2(x)$, we add them together:

$$|(f_1 + f_2)(x)| = |f_1(x) + f_2(x)| \tag{4.18}$$

We can now use the triangle inequality from A.1:

$$|f_1(x) + f_2(x)| \leq |f_1(x)| + |f_2(x)| \tag{4.19}$$

If we let x be greater than both k_1 and k_2 we can now use the respective inequalities for $|f_1(x)|$ from 4.16 and for $|f_2(x)|$ from 4.17:

$$|f_1(x)| + |f_2(x)| \leq C_1|g_1(x)| + C_2|g_2(x)| \tag{4.20}$$

Now we let $g(x) = (max(|g_1(x)|, |g_2(x)|))$ and substitute $g(x)$ for $g_1(x)$ and $g_2(x)$.

$$C_1|g_1(x)| + C_2|g_2(x)| \leq C_1|g(x)| + C_2|g(x)| = (C_1 + C_2)|g(x)| \tag{4.21}$$

We then combine the two constants and denote this new constant by C: $C = C_1 + C_2$.

$$(C_1 + C_2)|g(x)| = C|g(x)| \tag{4.22}$$

Furthermore it has been shown that:

$$|(f_1 + f_2)(x)| \leq C|g(x)|, \tag{4.23}$$

whenever $x > k$, where $k = max(k_1, k_2)$. Furthermore we can conclude that theorem 4.19 must be true, because we have found the witnesses $C = C_1 + C_2$ and $k = (max, (k_1, k_2))$ such that $(f_1 + f_2)(x)$ is $O(g(x)$. ■

Theorem 4.20. Big-O estimate of the product of two functions
Suppose that $f_1(x)$ is $O(g_1(x))$ and $f_2(x)$ is $O(g_2(x))$. Then $(f_1 f_2)(x)$ is $O(g_1(x)g_2(x))$.

Proof. $(f_1 f_2)(x)$ is $O(g_1(x)g_2(x))$

We start the proof by having:

$$|(f_1 f_2)(x)|.$$

We can split absolute value of the product of the two functions up in to the product of the absolute value of the two functions:

$$|(f_1 f_2)(x)| = |f_1(x)||f_2(x)|.$$

We know that $|f_1(x)| \leq C_1|g_1(x)|$ when $x > k_1$, and $|f_2(x)| \leq C_2|g_2(x)|$ when $x > k_2$ Therefore we get:

$$|(f_1 f_2)(x)| \leq C_1|g_1(x)|C_2|g_2(x)|,$$

when $x > max(k_1, k_2)$

This can be rewritten as:

$$|(f_1 f_2)(x)| \leq C_1 C_2|(g_1 g_2)(x)|$$

Since $C_1 C_2$ is just another constant C, we can write $C_1 C_2$ as C

$$|(f_1 f_2)(x)| \leq C|(g_1 g_2)(x)|$$

Because there exists constants $C = C_1 C_2$ and $k = max(k_1, k_2)$ such that $|(f_1 f_2)(x)| \leq C|(g_1 g_2)(x)|$ whenever $x > k$, we conclude that $(f_1 f_2)(x)$ is $O(g_1(x)g_2(x))$. ■

4.3.2 Big-Omega Notation

When analyzing growth of functions and algorithms, it can be useful to describe more than just the upper bound for the growth of the function. The notation Big-Omega denoted Big-Ω is used for the purpose of providing a lower bound for the growth of a given function.

Definition 4.21.
Let f and g be functions from the set of integers or the set of real numbers to the set of real numbers. We say that $f(x)$ is $\Omega(g(x))$ if constants $C > 0$ and k exists, such that

$$|f(x)| \geq C|g(x)|$$

Whenever $x > k$.

Note that by this definition $f(x)$ is $\Omega(g(x))$ if and only if $g(x)$ is $O(f(x))$.

Proof. Let $f(x)$ be $\Omega(g(x))$:

$$|f(x)| \geq C_1|g(x)|$$

for some C_1, whenever $x > k_1$ for some constant k_1. Then the constant $C_2 = \frac{1}{C_1}$ exists such that

$$C_2|f(x)| \geq |g(x)|$$

whenever $x > k_1$ for some k_1. For the sake of continuity, the inequality can be written as

$$|g(x)| \leq C_2|f(x)|.$$

■

Example 4.22. Example of use of big-Ω
Given two functions

$$f(x) = 5x^2 - 2x + 4,$$
$$g(x) = x^2$$

Show that $f(x)$ is $\Omega(g(x))$.

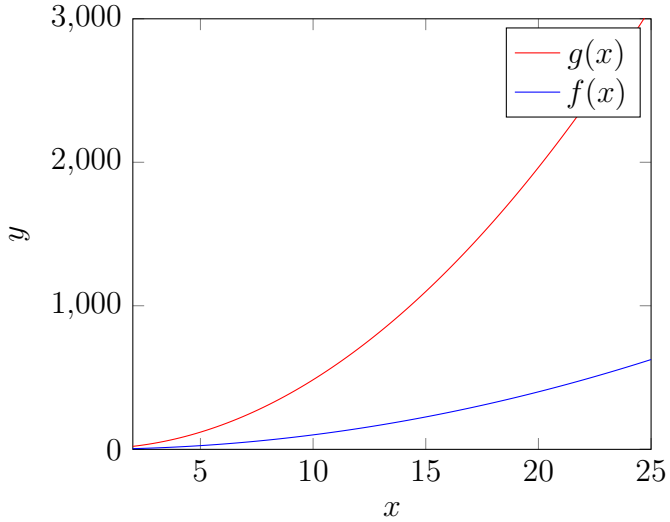
If if the witnesses $k = 2$ and $C = 1$ are chosen as the witnesses we get the following inequality:

$$|5x^2 - 2x + 4| \geq |x^2|$$

when $x > 2$.

We can examine if this is true by plotting the functions:

Growth of the functions $f(x) = 5x^2 - 2x + 4$ and $g(x) = x^2$, whenever $x > 2$



Furthermore we can conclude that the witnesses $k = 2$ and $C = 1$ makes it such that $f(x)$ is $\Omega(g(x))$ with.

4.3.3 Big-Theta Notation

As we saw in 4.3.1, Big-O provides an upper bound in terms of the growth function $f(x)$, but it does not provide a lower bound for the growth of the function $f(x)$. Meanwhile in 4.3.2, we learned that big-Ω gives a lower bound in terms of a function $f(x)$.
When we want to give the lower and upper bound of the growth of a function $f(x)$, to a reference function $g(x)$, we can use big-Θ.

When we have a function $f(x)$, and a reference function $g(x)$, such that $f(x)$ is both $O(g(x))$ and $\Omega(g(x))$, we say $f(x)$ is $\Theta(g(x))$. The notation Big-Θ is defined as:

Definition 4.23.
Let f and g be functions from the set of integers or the set of real numbers to the set of real numbers. We say that $f(x)$ is $\Theta(g(x))$ if $f(x)$ is $O(g(x))$ and $f(x)$ is $\Omega(g(x))$. When $f(x)$ is $\Theta(g(x))$, we say that f is big-Theta of $g(x)$, that $f(x)$ is of order $g(x)$, and that $f(x)$ and $g(x)$ are of the same order.

This means that $f(x)$ is $\Theta(g(x))$, if and only if there exists 2 constants C_1 and C_2 and a positive number k such that:

$$C_1|g(x)| \leq |f(x)| \leq C_2|g(x)|.$$

We use $C_1|g(x)|$ as the lower bound function $\Omega(g(x))$ and $C_2|g(x)|$ as our upper bound

function $O(g(x))$. An example of this, can be seen below:

Example 4.24. Show that $x^2 + 5x + 7$ is $\Theta(x^2)$

First we let $k = 1$, such that $x > 1$:

$$x^2 + 5x + 7 < x^2 + 5x^2 + 7x^2 = 13x^2.$$

The reason behind picking $x^2 + 5x^2 + 7x^2$, is because we know that it is greater than $x^2 + 5x + 7$ whenever $x > 1$. The first constant $C_2 = 13$ is the upper bound of the growth of the function.

Secondly when we let $x > 1$ we also see:

$$x^2 < x^2 + 5x + 7.$$

This means $C_1 = 1$, again x^2 is chosen, because it is less than x^2+5x+7, whenever $x > 1$

We now use $C_2 = 13$, $C_1 = 1$ and $g(x) = x^2$ in:

$$C_1|g(x)| \leq |f(x)| \leq C_2|g(x)|.$$

This results in:

$$x^2 \leq x^2 + 5x + 7 \leq 13x^2,$$

whenever $x > 1$. This shows that $x^2 + 5x + 7$ is $\Theta(x^2)$.

4.3.4 Complexity in Algorithms

The topic of complexity is relevant when discussing algorithms, because it can be used to assess the amount of time consumed by an algorithm to produce an output. Examining complexity is useful when comparing time consumed by algorithms. This is due to just measuring the time it takes to run, would not be comparable, because this time may vary a lot when run on different machines. It is important to note, that only time-complexity will be taken into account in this project, because it is usually more relevant than space-complexity, which refers to the amount of memory needed to compute an algorithm.

To incorporate complexity into algorithms, we say that an algorithm's complexity is calculated by the number of operations the algorithm uses. Usually the number of operations is related to size of the input. An algorithm does not necessarily use the same number of operations every time the algorithm is run, if the inputs are different. An example of

an algorithm where this applies, is the Linear Search Algorithm, because it performs a different amount of operations depending on what index the chosen integer has in the list. Therefore it is necessary to have some way of determining how the complexity is measured. This can be done using worst-case complexity or average-case complexity. We will proceed using worst-case complexity. When determining the complexity based on the worst case for a specific algorithm, the complexity is measured using the largest possible number of operations used by the algorithm to produce an output for a given input-size.
The following example is based on The Linear Search Algorithm introduced in section 4.1.1, and n refers to the size of the list given as an input.

Example 4.25. Complexity of Linear Search Algorithm.
In the case of the Linear Search Algorithm we note that the algorithm exits the loop when it finds x in the list. Therefore in the worst-case it does not exit the loop before $i \leq n$ is false.
For each iteration of the loop, the algorithm makes two comparisons, namely $i \leq n$ and $x \neq a_i$. If it does not find x in the list, this comparison will be made n times. Then it will begin the $(n+1)$st loop, where $i \leq n$ is false, resulting in the algorithm exiting the loop, and $x \neq a_i$ not being computed. At this stage the algorithm has performed $2n+1$ operations. It will do one last comparison after the loop has concluded, namely $i \leq n$. Therefore the algorithm uses $2n+2$ operations to produce an output.

Using theorem 4.15, we know that $2n+2$ is $\Theta(n)$.

Therefore The Linear Search Algorithm is said to be of complexity $\Theta(n)$.

As seen in example 4.25 the complexity of algorithms is tied closely to the growth of functions.

In the case of complexity of algorithms, we do not distinguish between "small" differences such as whether an algorithm is $\Theta(n^2)$ or if it is $\Theta(n^7)$. All complexities of the form $\Theta(n^b)$ where $b > 1$ are considered to be polynomial complexity. The complexities we do however distinguish between are constant, logarithmic, linear, linearithmic, polynomial, exponential and factorial complexity. The complexities of these can be seen in the table below:

Complexity	Terminology
$\Theta(1)$	Constant complexity
$\Theta(\log(n))$	Logarithmic complexity
$\Theta(n)$	Linear complexity
$\Theta(n \cdot \log(n))$	Linearithmic complexity
$\Theta(n^b)$	Polynomial complexity
$\Theta(b^n)$	Exponential complexity
$\Theta(n!)$	Factorial complexity

Table 4.2: Terminology for different complexities.

Example 4.25 is a fairly simple example of calculating the complexity of an algorithm, because the algorithm used is of linear complexity. Now we will calculate the complexity of a more complex algorithm, namely Dijkstra's Algorithm introduced in section 2.26.

Theorem 4.26.
The time complexity of Dijkstra's Algorithm's is $\Theta(n^2)$.

Proof.
In the calculation of the complexity of Dijkstra's Algorithm n refers to the number of vertices in the graph given as an input to the algorithm.

In the first loop the algorithm compares i to n, and this happens $n + 1$ times, because it is done once for every vertex, and once, where $i > n$ causing the loop to break.

In the next loop, running on the basis of the condition $z \notin S$ being true, one comparison is made everytime $z \notin S$ and once where $z \in S$. Therefore in the worst-case scenario, z will be added to S as the very last vertex, causing this comparison to happen n times. For every iteration of this loop, which in the worst case will be executed n times, one comparison is made for every vertex not in S, namely $L(u) + W(u, v) < L(v)$, when no vertices are in S. Therefore the construction with both a *while* loop and a *for* loop has a complexity of n^2.

In the worst-case scenario the algorithm uses $n^2 + n + 1$ operations. This function is $\Theta(n^2)$, resulting in Dijkstra's Algorithm having a complexity of $\Theta(n^2)$ which causes the algorithm to be of polynomial complexity. ■

5 | Interest Rate Theory

This chapter will focus on how to use mathematical models in economics to calculate simple interest, periodic compounding interest, continuous compounding of interest rates, and discount factors. These are relevant as they help explain the factors that are present in our optimization problem. Economics is a very broad topic but this chapter will focus on the time value of money, as this is a fundamentally important topic in finance that is directly linked to inflation and interest rates. The reason why inflation ties in with interest rates is due to inflation making money worth less in the future than if you were to go spend them today. Hence you would expect to be compensated for the decrease in the value of your money as you decide to postpone your consumption. Furthermore we will look at future values of amounts invested and borrowed. Likewise we will look at present values of amounts to be paid or received at a certain time in the future. In this chapter the interest rate $r > 0, r \in \mathbb{R}$, the terms of investing $t \in \mathbb{R}^+$ and frequency of compounding $m \in \mathbb{N}$ will be used. These terms will be explained throughout this chapter. The theory in this chapter has been collected from [3].

5.1 Simple Interest

The regular form of interest, is simple interest. Suppose that an initial amount of liquidity is deposited into a bank account, the initial deposit is then held in the bank account for a given period. During this period interest will only be earned on the initial deposit. A more precise definition of simple interest is given by definition 5.1:

Definition 5.1.
Interest in simple interest is only based on the principal, which remains unchanged during the entire period of investment.

The future value of an investment attracting simple interest is given by the theorem 5.2.

Theorem 5.2.
Let the future value of an investment after one year be denoted by:

$$V(1) = P(1 + r),$$

where P is the principal, and $r \cdot P$ denotes the interest earned on the principal during the year. Then the future value after t years is denoted by:

$$V(t) = (1 + tr)P.$$

Proof.

To prove theorem 5.2 the proposition $V(t) : (1 + tr)P$ is defined.

Basis Step: $V(1)$ is chosen as the basis step. To prove the basis step, the interest rate multiplied by the principal is added to the principal:

$$V(1) : rP + P. \tag{5.1}$$

Then P is factorised:

$$V(1) : (1 + r)P. \tag{5.2}$$

Inductive Step: To prove the inductive step, we first assume that

$$V(k) : (1 + kr)P, \tag{5.3}$$

is true after k years. To find the future value for the $(k + 1)$st years, the interest rP is added to the future value after k year:

$$V(k + 1) : (1 + kr)P + rP. \tag{5.4}$$

The term $(1 + kr)P$ is expanded and rP is factorised:

$$V(k + 1) : P + krP + rP = P + rP(k + 1). \tag{5.5}$$

Then the principal is factorised:

$$V(k + 1) : (1 + (k + 1)r)P, \tag{5.6}$$

This concludes the proof. ■

The function $V(t)$, denotes the amount of money, at a given time t. The part of the equation in parentheses, $1 + tr$, is denoted as the growth factor.
If we invest the principal P, in an already started period, we denote the time of investment by $s \in \mathbb{N} \wedge s < t$. The formula to calculate the value of the invested amount is given by:

$$V(t) = (1 + (t - s)r)P.$$

In the equation, s is subtracted from t, because no interest is earned before the time of investment.
When an investment has reached the end of a given period and we want to find the principal, the following formula is used:

$$V(0) = V(t)(1 + tr)^{-1}.$$

The amount of money at time $t = 0$, denoted $V(0)$, is called the present or discounted value of $V(t)$, and $(1+tr)^{-1}$, is called the discount factor, which we will look into in section 5.4.

5.2 Periodic Compounding of Interest

Suppose that an initial deposit P is deposited into an account. We are given an interest rate r on the initial deposit, however unlike in simple interest, the interest earned in periodic compounding is compounded periodically. A compounding period is typically annually, quarterly, monthly, weekly or daily. This means interest is earned not only on the initial deposit, but also on the interest that has compiled from previous periods.

Definition 5.3.
Periodic compounding interest is based on the principal and interest that has compiled from previous periods. The interest is compounded at a frequency of m where the interest rate is divided by m.

If m payments are payed annually, then the time between any two payments in years is calculated by $\frac{1}{m}$. This also means that the first payment will be payed after $\frac{1}{m}$ years. In the period the principal is held in the account, each payment will increase the current value of the deposit by a factor of $1+\frac{r}{m}$. After t years with m periods in a year, with an unchanged interest rate r the principal P will have increased to a value of:

$$V(t) = (1 + \frac{r}{m})^{tm} P. \tag{5.7}$$

Example 5.4.
Suppose we deposit a principal P into a bank account which pays an interest $\frac{r}{12}$ monthly. To calculate the principal after 1 year we can use the equation:

$$V(1) = (1 + \frac{r}{12})^{1 \cdot 12} P = (1 + \frac{r}{12})^{12} P.$$

This means, that the principal will have increased by $\frac{r}{12}$ after the first month. When the second payment is due, the principal will not only earn interest from the initial deposit, but also from the interest payed in the previous month. The principal after 2 years is calculated as:

$$(1 + \frac{r}{12})^{2 \cdot 12} P = (1 + \frac{r}{12})^{24} P.$$

If we keep our deposit in the bank account, the principal after t years is calculated by the following:

$$(1 + \frac{r}{12})^{t \cdot m} P.$$

Because the number of months after t years, $t \cdot m$, is an exponent, the amount of interest earned increases exponentially with time.

Note that the future value of of the deposit denoted $V(t)$ is dependent on the parameters r, m, t and the initial deposit P and will increase if one of the parameters increases and the

others remain unchanged. It is apparent from 5.7 that the future value of a deposit $V(t)$ increases as the parameters t, r and P increases. The future value increases as t increases, because the period is now longer which results in more interests being earned and due to the interest earned increasing exponentially over time. The future value increases as the interest increases due to a greater interest rate making the initial deposit grow by a larger growth factor. Lastly the principal P changes the future value as the initial deposit has a direct effect on how much the deposit increases after each interest payment is paid. Furthermore we wish to prove that m makes the future value increase when this parameter increases.

Theorem 5.5.
Let

$$V(t) = (1 + \frac{r}{m})^{tm} P$$

be the future value of an investment attracting periodically compounded interest, then the future value of the investment increases as the compounding frequency increases:

$$(1 + \frac{r}{m})^{tm} P < (1 + \frac{r}{k})^{tk} P,$$

whenever $k > m > 0$.

Proof. The future value of a deposit increases as m increases

To verify that the future value $V(t)$ increases as the compounding frequency m increases, we only need to look at the growth factor from equation 5.7. This is due to the future value of the investment increasing as the growth factor increases, because the interest earned increases as the growth factor increases. To see this we start by dividing P on both sides of the inequality:

$$\begin{aligned} P(1 + \frac{r}{m})^{tm} &< P(1 + \frac{r}{k})^{tk} \\ (1 + \frac{r}{m})^{tm} &< (1 + \frac{r}{k})^{tk} \end{aligned}$$

This growth factor is given by: $(1 + \frac{r}{m})^{tm}$. To show that $V(t)$ increases as m increases, we choose a constant k such that $k > m$. Now we only need to verify that a growth factor with k substituted for m is greater than m:

$$(1 + \frac{r}{m})^{tm} < (1 + \frac{r}{k})^{tk}. \tag{5.8}$$

We can now reduce this by taking the power of t on both sides of the equation. We do this by raising both sides of the inequality to the power of $\frac{1}{t}$. This is due to the growth factor being a non-zero positive number, because $t \geq 0$, $r > 0$ and $k > m > 0$:

$$(1+\frac{r}{m})^m < (1+\frac{r}{k})^k. \tag{5.9}$$

We can now expand the left side of the inequality in 5.9 using The Binomial Theorem which is given by A.2, and n is substituted with m.

$$(1+\frac{r}{m})^m = \sum_{k=0}^{m} \binom{m}{k} 1^{m-k}(\frac{r}{m})^k, \tag{5.10}$$

where

$$\binom{m}{k} = \frac{m!}{(m-k)!k!}.$$

We start by examining the first term, where we let $k=0$ in equation 5.10. We do this by dividing $m!$ by $(m-0)!$ which is 1. It is then multiplied by $1^{m-0} \cdot (\frac{r}{m})^0 = 1$ resulting in the following equation:

$$\binom{m}{0} 1^{m-0}(\frac{r}{m})^0 = \frac{m!}{(m-0)!0!} \cdot 1^{m-0} \cdot (\frac{r}{m})^0 = 1. \tag{5.11}$$

Then we calculate the term where $k=1$. Firstly we divide $m!$ by $(m-1)!$, which is m. Then we have $m \cdot \frac{r}{m} = r$:

$$\binom{m}{1} 1^{m-1}(\frac{r}{m})^1 = \frac{m!}{(m-1)!1!} \cdot 1^{m-1} \cdot (\frac{r}{m})^1 = m \cdot \frac{r}{m} = r. \tag{5.12}$$

In the term where $k=2$, we divide $m!$ by $(m-2)!$ which is $m \cdot (m-1)$:

$$\binom{m}{2} 1^{m-2}(\frac{r}{m})^2 = \frac{m!}{(m-2)!2!} \cdot 1^{m-2} \cdot (\frac{r}{m})^2 = \frac{m!}{(m-2)!2!} \cdot \frac{r^2}{m^2} = \frac{m \cdot (m-1)}{2!} \cdot \frac{r^2}{m^2}. \tag{5.13}$$

Then we factorize m from $(m-1)$ so that we are able to remove m^2 from the fraction $\frac{r^2}{m^2}$:

$$\frac{m \cdot (m-1)}{2!} \cdot \frac{r^2}{m^2} = \frac{m^2(1-\frac{1}{m})}{2!} \cdot \frac{r^2}{m^2} = \frac{1-\frac{1}{m}}{2!} \cdot r^2. \tag{5.14}$$

To calculate the m'th term of the sum, m is inserted as k. Then we expand this term:

$$\binom{m}{m} 1^{m-m}(\frac{r}{m})^m = \frac{m!}{(m-m)!m!} \cdot 1^{m-m} \cdot (\frac{r}{m})^m = \frac{m!}{0!m!} \cdot 1 \cdot \frac{r^m}{m^m} = \frac{m!}{m!} \cdot \frac{r^m}{m^m}. \tag{5.15}$$

We now wish to factorize m^m from the upper part of the fraction $\frac{m!}{m!}$ to isolate r^m in the equation. We therefore wish to find an x such that m^m multiplied by x equals $m!$, which can be written as:

$$m^m \cdot x = m!. \tag{5.16}$$

We then isolate x:

$$x = \frac{m!}{m^m}. \tag{5.17}$$

We now know that we can factorize m^m as follows:

$$\frac{m^m \cdot \frac{m!}{m^m}}{m!} = \frac{m!}{m!}. \tag{5.18}$$

Due to $m!$ being the product of m factors and m^m also being the product of m factors we can rewrite $\frac{m!}{m^m}$ as follows:

$$\frac{1}{m} \cdot \frac{2}{m} \cdot \frac{3}{m} \cdot \ldots \cdot \frac{m-2}{m} \cdot \frac{m-1}{m} \cdot \frac{m}{m} = \frac{1}{m} \cdot \frac{2}{m} \cdot \frac{3}{m} \cdot \ldots \cdot \frac{m-2}{m} \cdot \frac{m-1}{m}. \tag{5.19}$$

We again rewrite this as follows:

$$\frac{1}{m} \cdot \frac{2}{m} \cdot \frac{3}{m} \cdot \ldots \cdot \frac{m-2}{m} \cdot \frac{m-1}{m} = \frac{1}{m} \cdot \frac{2}{m} \cdot \frac{3}{m} \cdot \ldots \cdot (1 - \frac{2}{m}) \cdot (1 - \frac{1}{m}). \tag{5.20}$$

We are now able to factorize m^m from equation 5.15:

$$\frac{m!}{m!} \cdot \frac{r^m}{m^m} = \frac{m^m \cdot (\frac{1}{m} \cdot \frac{2}{m} \cdot \frac{3}{m} \cdot \ldots \cdot (1 - \frac{2}{m}) \cdot (1 - \frac{1}{m}))}{m!} \cdot \frac{r^m}{m^m}. \tag{5.21}$$

We then observe that the factor m^m in the numerator of the left-hand fraction removes m^m in the denominator of the right-hand fraction:

$$\frac{\frac{1}{m} \cdot \frac{2}{m} \cdot \frac{3}{m} \cdot \ldots \cdot (1 - \frac{2}{m}) \cdot (1 - \frac{1}{m})}{m!} \cdot r^m. \tag{5.22}$$

Note that we can rewrite terms with m in the denominator by adding $\frac{m}{m}$ and subtracting it again:

$$\frac{a}{m} = \frac{m}{m} - \frac{m}{m} + \frac{a}{m}, \tag{5.23}$$

Where $a \in \mathbb{N}$. We now combine $\frac{a}{m}$ and $-\frac{m}{m}$ which causes a to change sign from positive to negative sign:

$$1 - \frac{m}{m} + \frac{a}{m} = 1 - \frac{m-a}{m}. \tag{5.24}$$

Therefore we can change equation 5.22 to:

$$\frac{(1-\frac{m-1}{m}) \cdot (1-\frac{m-2}{m}) \cdot (1-\frac{m-3}{m}) \cdot \ldots \cdot (1-\frac{2}{m}) \cdot (1-\frac{1}{m})}{m!} \cdot r^m. \tag{5.25}$$

We now combine the 0'th 1'st 2'nd and m'th k terms from the equations 5.11, 5.12, 5.14 and 5.25 to finish the expansion of the binomium, $(1+\frac{r}{m})^m$, which gives us:

$$(1+\frac{r}{m})^m = \sum_{k=0}^{m} \binom{m}{k} \cdot 1^{m-k} \cdot (\frac{r}{m})^k. \tag{5.26}$$

$$(1+\frac{r}{m})^m = 1+r+\frac{1-\frac{1}{m}}{2!} \cdot r^2 + \cdots + \frac{(1-\frac{m-1}{m}) \cdot (1-\frac{m-2}{m}) \cdot (1-\frac{m-3}{m}) \cdot \ldots \cdot (1-\frac{2}{m}) \cdot (1-\frac{1}{m})}{m!} \cdot r^m. \tag{5.27}$$

We wish to prove that the inequality, $(1+\frac{r}{m})^m < (1+\frac{r}{k})^k$, holds when $k > m > 0$. We start off by substituting the fractions on the upper terms of each fraction that contains m by k where $k > m$. This means that the denominators in the fractions in the numerator will remain as m. Thus we obtain the inequality:

$$\begin{aligned} &(1+\frac{r}{m})^m = \\ &1 + r + \frac{1-\frac{1}{m}}{2!} \cdot r^2 + \cdots + \frac{(1-\frac{m-1}{m}) \cdot (1-\frac{m-2}{m}) \cdot (1-\frac{m-3}{m}) \cdot \ldots \cdot (1-\frac{2}{m}) \cdot (1-\frac{1}{m})}{m!} \cdot r^m \\ &\leq 1 + r + \frac{1-\frac{1}{k}}{2!} \cdot r^2 + \cdots + \frac{(1-\frac{m-1}{k}) \cdot (1-\frac{m-2}{k}) \cdot (1-\frac{m-3}{k}) \cdot \ldots \cdot (1-\frac{2}{k}) \cdot (1-\frac{1}{k})}{m!} \cdot r^m. \end{aligned}$$

This inequality holds because each factor in the numerator on the left-hand side of the inequality is not greater than each corresponding term on the right-hand side for example, $(1-\frac{m-1}{m})$ is no greater than $(1-\frac{m-1}{k})$, when $k > m > 0$. This can be proven from the condition that k is greater than m:

$$k > m.$$

We now divide by k on both sides of the inequality:

$$1 > \frac{m}{k}.$$

We then divide by m on both sides of the inequality and obtain two fractions:

$$\frac{1}{m} > \frac{1}{k}.$$

We continue by adding 1 on both sides.

$$1 + \frac{1}{m} > 1 + \frac{1}{k}.$$

We now obtain the desired terms on each side of the inequality by subtracting the terms $\frac{1}{k}$ and $\frac{1}{m}$ on both sides:

$$1 - \frac{1}{k} > 1 - \frac{1}{m}.$$

We have now established that the terms containing k are greater than those containing m. We now substitute k for m in The Binomial Theorem, resulting in $k - m$ additional terms, by which we obtain the expansion of the binomium, $(1 + \frac{r}{k})^k$:

$$\begin{aligned} &1 + r + \frac{1 - \frac{1}{k}}{2!} \cdot r^2 + \cdots + \frac{(1 - \frac{m-1}{k}) \cdot (1 - \frac{m-2}{k}) \cdot (1 - \frac{m-3}{k}) \cdot \ldots \cdot (1 - \frac{2}{k}) \cdot (1 - \frac{1}{k})}{m!} \cdot r^m \\ &< 1 + r + \frac{1 - \frac{1}{k}}{2!} \cdot r^2 + \cdots + \frac{(1 - \frac{k-1}{k}) \cdot (1 - \frac{k-2}{k}) \cdot (1 - \frac{k-3}{k}) \cdot \ldots \cdot (1 - \frac{2}{k}) \cdot (1 - \frac{1}{k})}{k!} \cdot r^k \\ &= (1 + \frac{r}{k})^k. \end{aligned}$$

This second inequality holds because we established that $k > m$, which causes the right-hand side to contain more positive terms to be added to the sum as compared to the sum on the right side. The sum on the right side contains $k - m$ more terms than the corresponding term in the binomial expansion of $(1 + \frac{r}{m})^m$. This completes the proof that increasing the parameter m will cause the future value to increase when the other parameters remain unchanged. ■

5.3 Continuous Compounding of Interest

Suppose we deposit a principal P into a bank account, then the principal P is subject to interest from the bank. After the first payment from the bank, the amount of money will have grown larger. The interest acquired will now also be subject to interest, which means that $V(t)$ grows exponentially as t increases.

Definition 5.6.
Continuous compounded interest is periodically compounded interest, where the compounding frequency is infinitely large.

In chapter 5.2, the formula for finding the future value of a principal, with interest rate r after t years was:

$$V(t) = \left(1 + \frac{r}{m}\right)^{tm} P.$$

This can be rewritten as:

$$V(t) = \left[\left(1 + \frac{r}{m}\right)^{\frac{m}{r}}\right]^{tr} P. \tag{5.28}$$

Theorem 5.7.
The future value of continuous compounded interest at time t of an initial value P compounded at a frequency m times a year is given by:

$$V(t) = e^{tr} P,$$

where the growth factor $\left(1 + \frac{1}{m}\right)^m$ approaches e as m approaches ∞:

$$e = \lim_{m \to \infty} \left(1 + \frac{1}{m}\right)^m.$$

Theorem 5.7 will be proven in the following proof to show that e can be used in the formula for the future value of continuous compounded interest.

Proof.

It will be proven that:

$$e = \lim_{m \to \infty} \left(1 + \frac{1}{m}\right)^m.$$

To begin the proof, we let:

$$y = \left(1 + \frac{1}{m}\right)^m. \tag{5.29}$$

Then we take the natural logarithm on both sides of the equation, which is allowed because both sides of the equation are positive. We do this because we want to achieve a fraction of two functions where the functions approach either 0 or ∞ so that we can use L'Hospital's

rule [4]. This is allowed because both the numerator and the denominator are differentiable, and the denominator is not 0:

$$\lim_{x\to\infty} \frac{f(x)}{g(x)} = \lim_{x\to\infty} \frac{f'(x)}{g'(x)}. \tag{5.30}$$

We also pick the natural logarithm because it is the inverse function of e^x.

$$ln(y) = ln\Big(\Big(1+\frac{1}{m}\Big)^m\Big). \tag{5.31}$$

We then use the rule for the natural logarithm: $ln(a^r) = r \cdot ln(a)$, to move the exponent m in 5.31 outside the natural logarithm:

$$ln(y) = m \cdot ln\Big(1+\frac{1}{m}\Big). \tag{5.32}$$

Since we are proving that the growth factor approaches e as m approaches infinity, we now let m approach infinity in 5.32:

$$\lim_{m\to\infty} ln(y) = \lim_{m\to\infty} m \cdot ln\Big(1+\frac{1}{m}\Big). \tag{5.33}$$

Since the limit of $ln(y)$ is 0 as m approaches ∞, we will rewrite 5.32, this is because if we can make the right-hand side approach $\frac{0}{0}$ as m approaches ∞, we can use L'Hospital's rule. Therefore we rewrite 5.32 as:

$$ln(y) = \frac{ln(1+\frac{1}{m})}{\frac{1}{m}}. \tag{5.34}$$

We take the limit on both sides as m approaches ∞, and we obtain:

$$\lim_{m\to\infty} ln(y) = \lim_{m\to\infty} \frac{ln(1+\frac{1}{m})}{\frac{1}{m}}. \tag{5.35}$$

Due to $\frac{1}{m}$ approaching 0 when m approches ∞, we have:

$$\lim_{m\to\infty} ln(y) = \frac{ln(1+0)}{0} = \frac{0}{0}. \tag{5.36}$$

We obtain that $ln(y)$ approaches $\frac{0}{0}$ as m approaches ∞, which allows us to use L'Hospital's rule from equation 5.30:

$$\lim_{x\to\infty} \frac{f(x)}{g(x)} = \lim_{x\to\infty} \frac{f'(x)}{g'(x)}. \tag{5.37}$$

Using L'Hospital's rule and the chain rule on 5.35 we obtain the following:

$$\lim_{m\to\infty} ln(y) = \lim_{m\to\infty} \frac{\frac{1}{1+\frac{1}{m}} \cdot \frac{-1}{m^2}}{\frac{-1}{m^2}}. \tag{5.38}$$

We note that $\frac{-1}{m^2}$ is in both the numerator and denominator. Consequently we obtain:

$$\lim_{m\to\infty} ln(y) = \lim_{m\to\infty} \frac{1}{1+\frac{1}{m}}. \tag{5.39}$$

We evaluate the limit and obtain:

$$\lim_{m\to\infty} ln(y) = \frac{1}{1+0}. \tag{5.40}$$

$$\lim_{m\to\infty} ln(y) = 1. \tag{5.41}$$

We then substitute $y = (1+\frac{1}{m})^m$ on the left side of the equation, and put the limit inside the natural logarithm, which is allowed, due to the natural logarithm being a continuous function near e:

$$ln\Big(\lim_{m\to\infty}\Big(1+\frac{1}{m}\Big)^m\Big) = 1. \tag{5.42}$$

We can remove the natural logarithm on the left side of the equation, using e^x which is the inverse function of $ln(x)$:

$$\lim_{m\to\infty}\Big(1+\frac{1}{m}\Big)^m = e^1. \tag{5.43}$$

$$\lim_{m\to\infty}\Big(1+\frac{1}{m}\Big)^m = e. \tag{5.44}$$

We have now proven that the growth factor $\Big(1+\frac{1}{m}\Big)^m$ approaches e as m approaches ∞.

If we replace the growth factor in 5.28 with e, which is allowed when $r > 0$ we get the following:

$$V(t) = \Big[\lim_{m\to\infty}\Big(1+\frac{r}{m}\Big)^{\frac{m}{r}}\Big]^{tr} P = e^{tr}P.$$

Which is the same equation seen in theorem 5.7. ■

5.4 Discount Factor

Sometimes we know the future value of an invested amount $V(t)$, but we do not know the initial value $V(0)$. To calculate the initial value of an investment, we use the discount factor.
The formula in the following definition can be used to find the initial value of an investment:

Definition 5.8.
Formula for finding the present value of a future payment, when continues compounding interest applies:

$$V(0) = V(t) \cdot e^{-tr}$$

In this instance, e^{-tr}, is the discount factor.

For simple interest, periodic compounding interest and continuous compounding interest, there exists different discount factors:

- $(1 + tr)^{-1}$, is the discount factor for simple interest.
- $(1 + \frac{r}{m})^{-tm}$, is the discount factor for periodic compounding interest.
- e^{-tr}, is the discount factor for continuous compounding interest.

Example 5.9.
Find the present value of 1000000€ to be received after 20 years assuming continuous compounding interest at 6%.

Since the interest rate is continues compounding interest, and we have to find the present value, we use the formula:

$$V(0) = V(t) \cdot e^{-tr}$$

Where $V(20) = 1000000$, $t = 20$ and $r = 0.06$

$$V(0) = 1000000 \cdot e^{-20 \cdot 0.06}$$

$$V(0) = 301194$$

We find that the present value of 1000000€ to be received after 20 years assuming continuous compounding interest at 6% is 301194€.

6 | Basis Problem

In the following chapter the weight of the respective edges in the presented directed weighted graphs are not displayed in order for the graphs readable. The parameters, conditions and general problem outline presented in this chapter are found in [5]. The meaning of rate of return is found in [6]:

6.1 Definition of The Basis Problem

The problem states that we control a storage unit for natural gas in a fixed amount of time. The storage unit has a maximum and minimum capacity of gas at a given moment in time that cannot be exceeded. We have a forward projection on the price of natural gas in the period that we control the unit. There exists a limit on the number of gas units that are able to be exported from or imported to the storage unit at any given moment in time. A discount factor is applied such that inflation is taken into account, by lowering the value of money through the period. The reason we are using the discount factor for continuous compounding is that inflation is compounded continuously. The owner of the storage unit, has agreed to buy the remaining gas units in the storage unit at the end of the period. This sale at time T is not restricted by the limit on exporting gas units from the storage unit at any time t. The costs related to leasing the storage unit, is not taken into account. The main goal is to maximize the profit obtainable by buying and selling natural gas in the given period of controlling the storage unit.

The problem definition states a number of parameters that are necessary in order to solve the basis problem. These parameters are defined as:

- q_0 is the number of gas units contained in the storage unit, at the beginning of the lease.
- q_{min} and q_{max} is the lower and upper limit of gas units in the storage unit.
- u_{max}, $i_{max} \in \mathbb{N}$ is number of units that can be exported from or imported to the storage unit between any two periods.
- p_t is the forward-price in euro denoted by € on a unit of gas where $t \in \{1, \ldots, T\}$.
- r is a constant interest rate, such that e^{-r} is a stable yearly discount factor.

Maximise the present value of the profit, with respect to Δq_t:

$$\max_{\Delta q_t} \Big\{ -\sum_{t=1}^{T} e^{-r\frac{t}{T}} \Delta q_t p_t + e^{-r} q_T p_T \Big\} \tag{6.1}$$

This mathematical definition of the maximization problem states that the profit is to be maximized with respect to Δq_t, which denotes the change in the number of units of gas between two periods. The profit is defined as the sum of the product of the discount factor when continuous compounding applies, the change in gas units and the price at time t from $t = 1$ as this is the start of the period to T which denotes the end of the period. The product of the discount factor, the units of gas and the price at the last term is then added to the sum and the entire sum is multiplied by (-1), to ensure that buying gas causes a negative cash flow, and vice versa.

This mathematical definition has the following conditions:

- $q_{min} \leq q_t \leq q_{max}$
- $\Delta q_t \in \{-u_{max}, -u_{max} + 1, \ldots, -1, 0, 1, \ldots, i_{max} - 1, i_{max}\}$

for $t \in \{1, \ldots, T\}$, when $q_t = q_0 + \sum_{s=1}^{t} \Delta q_s$, is the amount stored at time t.

The first condition states, that the number of units of gas contained in the storage unit, is always within the closed interval between the minimum and maximum capacity of the storage unit. The second condition sates, the the difference in the number of units of gas between 2 periods, is always within the closed interval between the maximum number of units that can be exported from or imported to the storage unit.

6.2 Simple Version of The Basis Problem

Since the number of periods and the number of gas units are discrete, the basis problem can be represented as a directed weighted graph. If we let the weight of the edges represent the price of buying and selling gas units, the basis problem becomes a longest path problem. We want to find the longest path, because we want to maximize our profits represented by positive edges.

To illustrate the idea behind solving the basis problem we will construct a simple version of the basis problem. This is done by reducing the number of edges and vertices in the graph. The parameters introduced in 6.1 excluding r are assigned values in order to construct the simple problem. Furthermore, the discount factor is not taken into account in the simple version of the basis problem:

- $q_0 = 1$.
- $q_{min} = \{0, 0, 0, 0\}$.
- $q_{max} = \{2, 2, 2, 2\}$.

- $i_{max} = \{1, 1, 1, 1\}$.
- $u_{max} = \{1, 1, 1, 1\}$.
- $p_t = \{20, 18, 25, 22\}$.
- $T = 4$.

This assignment of values to the parameters describes a problem, where we control the storage unit for 4 terms. The maximum capacity of gas units in the storage unit is 2 units of gas, and the minimum is 0 units of gas. At most 1 unit of gas can either be exported from or imported to the storage unit at each term. At the start of the period the storage unit contains 1 unit of gas.

The equation that maximizes the profit changes in the simple version of the basis problem. The mathematical definition for maximizing the profit in the simple version of the basis problem can be constructed as follows:

$$\max_{\Delta q_t} \Big\{ -\sum_{t=1}^{T} \Delta q_t p_t + q_T p_T \Big\}. \tag{6.2}$$

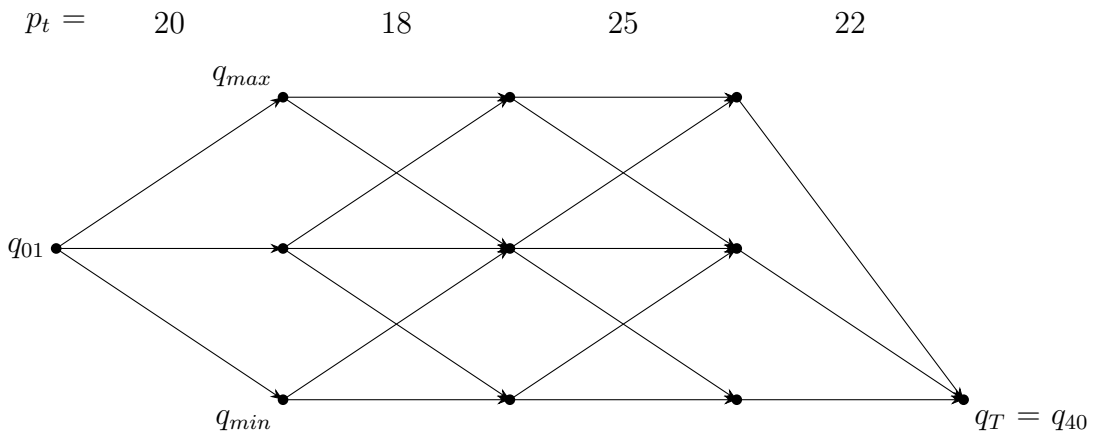
This mathematical definition of the maximization problem states that the profit is to be maximized with respect to Δq_t, which denotes the change in the number of units of gas between two periods. The profit is defined as the sum of the product of the change in gas units and the price at time t from $t = 1$ as this is the start of the period to T which denotes the end of the period. The number of units of gas and the price at the last term is then added to the sum and the entire sum is multiplied by (-1), to ensure that buying gas causes a negative cash flow, and vice versa.

The mathematical definition has the following conditions:

- $0 \leq q_t \leq 2$.
- $\Delta q_t \in \{-1, 0, 1\}$

for $t \in \{1, \ldots, T\}$, when $q_t = q_0 + \sum_{s=1}^{t} \Delta q_s$, is the amount stored at time t.

To represent the possible strategies for selling or buying gas, we construct a directed weighted graph with the placement of the vertices referring to the term and the number of gas units that are currently present in the storage unit. For example at time $t = 0$, when the storage contains a single unit of gas the vertex is denoted by: q_{01}. The weight of each edge represents the cash flow between 2 time periods and therefor 2 adjacent vertices.



Note the longer the path is, the bigger the profit is, because a positive flow of cash is represented by a positive edge and vice versa. To combat this we have to multiply all weights by a factor of (-1) because Dijkstra's finds the shortest path. The graph contains edges with negative weights, and as stated in 4.2 no negative edges are allowed in a graph for Dijkstra's Algorithm when we want to find the shortest path in a directed weighted graph. Consequently we find the edge with the least weight in the graph, which in this case is: $\{q_{32}, q_{40}\}$ which has a weight of -44. To ensure all edges are positive $(-44) \cdot (-1) + 1 = 45$ is added to the weight of each edge.

The computed steps done to find the shortest path is summarized in the following table:

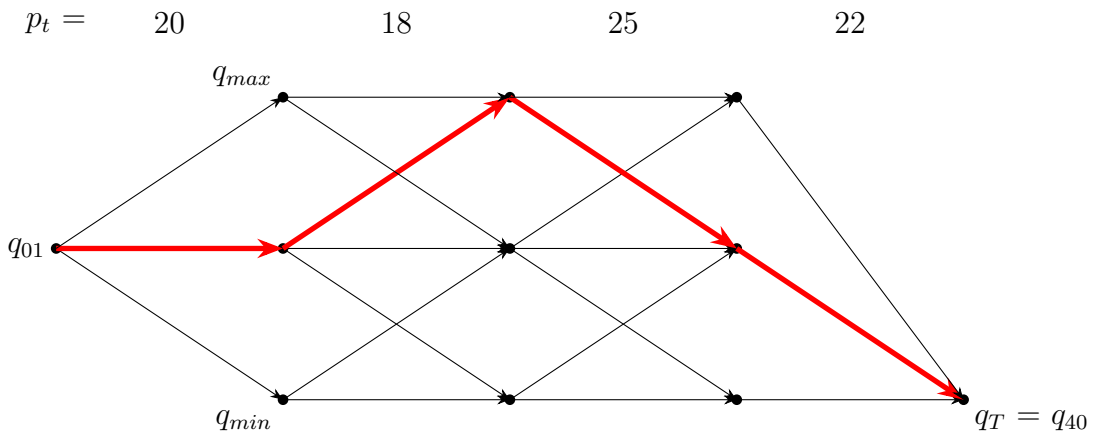
I	$L(q_{01})$	$L(q_{10})$	$L(q_{11})$	$L(q_{12})$	$L(q_{20})$	$L(q_{21})$	$L(q_{22})$	$L(q_{30})$	$L(q_{31})$	$L(q_{32})$	$L(q_{40})$
0	*0*	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	**0**	*25*	*45*	*65*	∞	∞	∞	∞	∞	∞	∞
2	0	**25**	45	65	*70*	*88*	∞	∞	∞	∞	∞
3	0	25	**45**	65	70	88	*108*	∞	∞	∞	∞
4	0	25	45	**65**	70	88	108	∞	∞	∞	∞
5	0	25	45	65	**70**	88	108	*115*	*140*	∞	∞
6	0	25	45	65	70	**88**	108	*108*	*133*	*158*	∞
7	0	25	45	65	70	88	**108**	108	*128*	*153*	∞
8	0	25	45	65	70	88	108	**108**	128	153	*153*
9	0	25	45	65	70	88	108	108	**128**	153	*151*
10	0	25	45	65	70	88	108	108	128	153	**151**

Table 6.1: Overview of shortest known paths for each iteration I of Dijkstra's Algorithm. Numbers written in a bold font denotes that the given vertex is added to S. Numbers written in italic font indicates that the shortest known path is updated for the given vertex.

To solve this problem using the procedure from Dijkstra's Algorithm the following steps are computed:

- Before the first iteration begins, all labels besides the one for q_{01} are set to ∞ and the label for q_{01} is set to 0.
- In the first iteration the vertex q_{01} is added to S, hence updating labels to $L(q_{10}) = 25, L(q_{11}) = 45, L(q_{12}) = 65$.
- In the second iteration the vertex q_{10} is added to S, and labels are updated to $L(q_{20}) = 70, L(q_{21}) = 88$.
- In the third iteration the vertex q_{11} is added to S. Then the label for q_{22} is updated to $L(q_{22}) = 108$.
- The vertex q_{12} is added to S in the fourth iteration. In this iteration no shorter paths to any vertices are found. Therefore no labels are updated.
- In the fifth iteration q_{20} is added to S, updating labels to $L(q_{30}) = 115, L(q_{31}) = 140$.
- In the sixth iteration q_{21} is added to S, then labels are updated: $L(q_{30}) = 108, L(q_{31}) = 133, L(q_{32}) = 158$.
- q_{22} is added to S in the seventh iteration, and labels are updated to $L(q_{31}) = 128, L(q_{32}) = 153$.
- In the eighth iteration q_{30} is added to S, hence updating the label $L(q_{40}) = 153$.
- In the ninth iteration q_{31} is added to S. Then the label for q_{40} is updated: $L(q_{40}) = 151$.
- When the algorithm reaches the tenth iteration, the goal vertex q_{40} has the label of the least length of all vertices not in S, therefore it is added to S, and the shortest path has been found. Tracing back the steps taken, we find that the shortest path visits $q_{01}, q_{11}, q_{22}, q_{31}$ and q_{40}.

The directed weighted graph seen below contains edges colored with red. These edges represent the longest path trough the directed weighted graph.



The details of the path containing the edges coloured in red is seen in the table below:

Table 6.2: Solution for the simple problem

t	*Units at t*	*Change at t*	*Profit/cost at t*	*Total profit/cost at t*
0	1	0	0	0
1	1	0	0	0
2	2	+1	−18	−18
3	1	−1	25	7
4 = T	0	−1	22	29

The profit or cost related to the most optimal path is displayed in table 6.2. Likewise the total profit/cost at a time t is displayed in the fifth column of the table. In the table we observe that the most optimal path is to not buy a unit of gas at $t = 1$, buy 1 unit of gas at $t = 2$, sell 1 unit of gas at $t = 3$ and sell 1 unit of gas at $t = 4$. This gives the most optimal profit of 29€. This most optimal path is given by: $q_{01}, q_{11}, q_{22}, q_{31}, q_{40}$ with a total weight of 151.

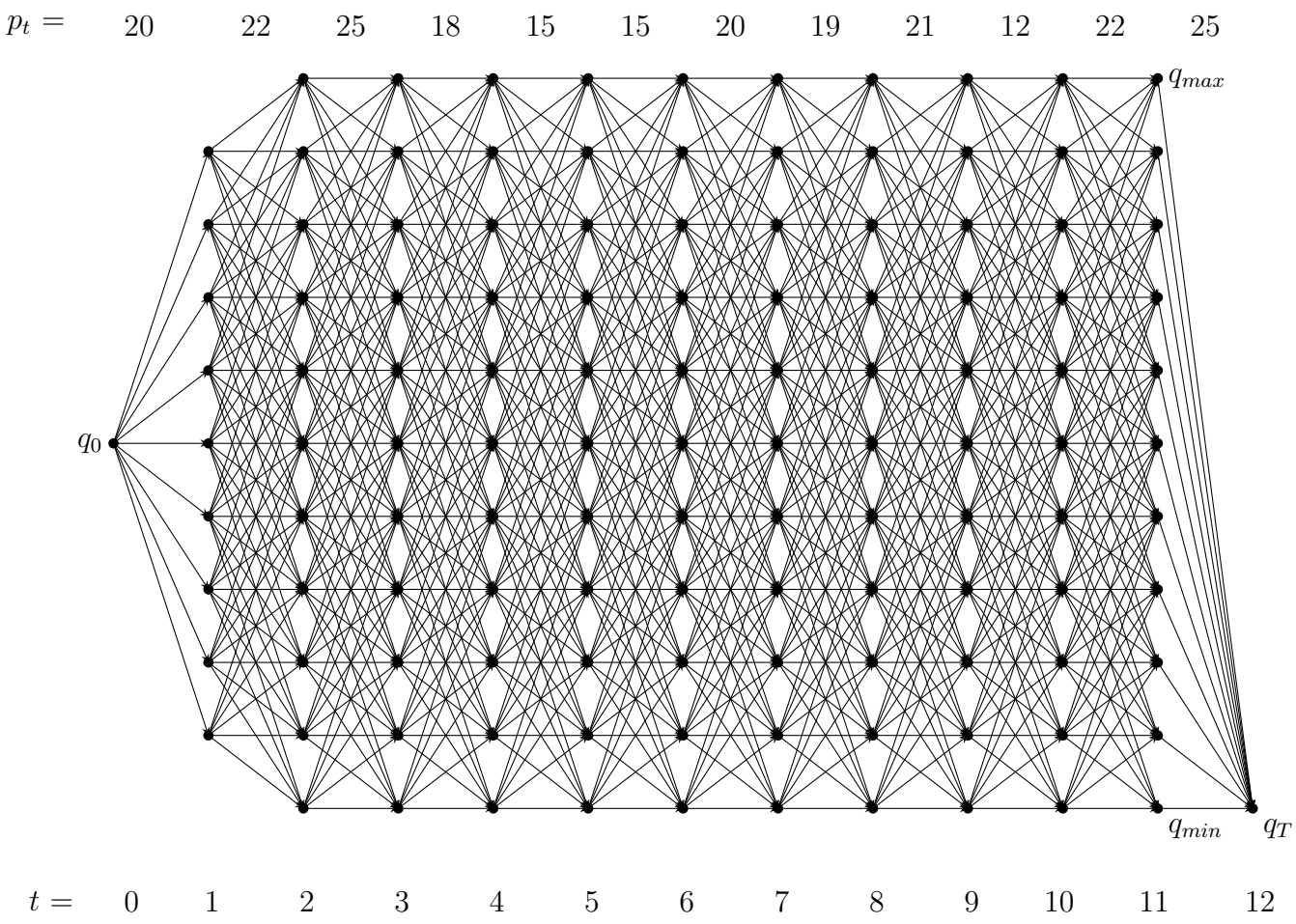
6.3 Solving The Basis Problem

In the problem definition of the basis problem, we are given the following data:

- The minimum capacity of the storage unit is: $q_{min} = 0$.
- The maximum capacity of the storage unit is: $q_{max} = 10$.
- The maximum number of gas units that can be imported into the storage unit between 2 periods is: $i_{max} = 4$.

- The maximum number of gas units that can be exported from the storage unit between 2 periods is: $u_{max} = 4$.
- The number of gas units in the storage unit at the start of the first period is: $q_0 = 5$.
- The interest rate is: $r = 0.04$.
- The total number of months we control the storage unit is: $T = 12$.
- The price for 1 unit of gas in each period is: $(p_t)_{t=1}^{T} = \{20, 22, 25, 18, 15, 15, 20, 19, 21, 12, 22, 25\}$.

The directed weighted graph, of the basis problem is illustrated below:



In the graph, the initial vertex labelled q_0 represents the number of gas units in the storage unit at time $t = 0$. As seen in the graph, the storage unit contains 5 units of gas at $t = 0$. The maximum number of units of gas, that can be exported from or imported to the storage unit between 2 periods is 4 units of gas. Our initial vertex therefore has a degree of 9. This is because there are 9 possibilities of importing and exporting in the next period which are $\Delta q_1 \in \{-4, -3, -2, -1, 0, 1, 2, 3, 4\}$. The upper limit q_{max} of the storage unit at any time, is illustrated as all the vertices horizontal to the label q_{max}. The

lower limit of the storage unit at any time, is illustrated as all the vertices horizontal to label q_{min}. The price of 1 unit of gas p_t at a given time t is placed above the graph, to indicate the price of 1 gas unit between 2 adjacent points in time. The weight of an edge between two vertices is given by the price p_t multiplied by the change in gas units from the initial vertex to the terminal vertex of an edge and the discount factor. Note that when gas is imported, the weight of an edge becomes negative. Which means that an edge has a positive weight, when gas units are exported. The terminal vertex labelled as q_T is the number of gas units in the storage unit in the period $t = 12$. Note that all vertices at period $t = 11$ are connected to the terminal vertex q_T. This is because the owner of the storage unit will purchase all remaining units of gas in the storage unit, at the end of the last period.

6.4 Using Python to Solve The Basis Problem

When trying to find the longest path through the graph representing possible strategies for buying and selling gas, it becomes tedious to manually use Dijkstra's Algorithm for larger graphs. Therefore the pseudocode for Dijkstra's Algorithm can be modified to be interpretable by a programming language. In this paper, the programming language Python 3.11 is used to run the algorithm. The script for generating the graph introduced in this section is written with inspiration from [7, p. 34-36].

It is necessary to have some way of representing a graph data-wise in Python before trying to run Dijkstra's Algorithm. Similar to running Dijkstra's Algorithm, it would be very time-consuming to create this data structure manually. Therefore another algorithm is written in python in order to create this data structure.

```
1 def make_edges(terms, q_max, q_min, u_max, i_max, q_0, prices):
2     edges = []
3     root = [f"q_0_{q_0}"]
4     vertices = root
5     vertices_units = [q_0]
6     added_constant = max(prices) * max(q_max) + 1
```

Summary 6.1: Creating the graph part 1

The function *make_ edges* starts by creating an empty list, *edges*. This is done to state this variable is a list, such that elements can be appended to this list later on in the algorithm. Then a variable *root* is assigned the value of q_0 along with the number of units of gas at this point in time as a way to logically represent vertices. The reason to use a list, is so that it can be assigned to the variable *vertices*, so that other vertices can be appended to the set of vertices. *vertices_ units* is a list currently only containing the number of gas units at the 0'th term of the period. As more vertices are added to the list *vertices*, the corresponding value of gas units present will be added to the list *vertices_ units*. It is crucial that the number of gas units will also have the same index in *vertices_ units* as the corresponding

vertex has in *vertices*, how this is done, can be seen later on in the algorithm, namely in line 24 and line 25. Then a *for* loop is initiated:

```
7     for t in range(terms):
8         closed_vertices = vertices
9         closed_vertices_units = vertices_units
10        vertices = []
11        vertices_units = []
12        for i in range(len(closed_vertices)):
13            units_to_buy_max = min((q_max[t] - closed_vertices_units[i]),
      u_max[t])
14            units_to_sell_max = min((closed_vertices_units[i] - q_min[t])
    , i_max[t])
```

Summary 6.2: Creating the graph part 2

The loop runs over each term one at a time, creating edges connecting vertices in the current term to newly created vertices in the next term. All vertices from the previous term/iteration will be appended to the list *closed_ vertices*, because the newly created vertices will be appended to the list *vertices*, that will be saved in the variable *closed_ vertices* in the beginning of the next iteration. Therefore the list of vertices can be redefined as an empty list, so that all the vertices created in this iteration, can be stored in this list.
In the indented loop on line 12, i runs over the indices of the elements in the list of previously created vertices. The *for* loop starts every iteration by calculating the upper and lower bounds for change in units of gas, for the vertex in the previously created vertices that has the index of the current value of i.
Then the loop continues:

```
15            if not t == terms - 1:
16                for units_to_buy in range(1, units_to_buy_max + 1):
17                    if closed_vertices_units[i] + units_to_buy >= q_min[t
    ]:
18                        initial_vertex = closed_vertices[i]
19                        terminal_vertex = f"q_{t+1}_{units_to_buy+
    closed_vertices_units[i]}"
20                        edge_weight = e**(-0.04 * (t + 1) / terms) *
    units_to_buy * (-prices[t]) * (-1) + int(added_constant)
21                        edge = (initial_vertex, terminal_vertex,
    edge_weight)
22                        edges.append(edge)
23                        if terminal_vertex not in vertices:
24                            vertices.append(terminal_vertex)
25                            vertices_units.append(closed_vertices_units[i
    ] + units_to_buy)
```

Summary 6.3: Creating the graph part 3

The *if* statement on line 15, splits the terms into two parts. One part where the next term will not be the last term, and one, where the next term is the ending of the controlling period. This is done, because all units of gas will be sold in the last period, as stated in the

problem definition. Then a loop is entered at line 16 that creates an edge to a vertex in the next term for each possible number of gas units that can enter the storage unit during this term (the calculation of this was done in line 13) if and only if this will not result in less gas being in the storage unit than q_{min} allows for this term. When an edge is created, it has three properties, namely an initial vertex, a terminal vertex and a weight. The weight is calculated on line 20, using the discount factor along with the price in the term and the number of units bought. Then *edge_ weight* is multiplied by (-1) to convert the problem to a longest path problem. Also the constant for forcing all edges to be greater than 0 is added. The edge is then added to the list of edges introduced in line 2. Lastly if and only if the terminal vertex of a newly created edge is not already in the list of vertices, meaning that it has already been created, then the terminal vertex is appended to the list of vertices. This is done to make sure no duplicate vertices are created. Since the number of gas units at the newly created vertex is added to the list of gas units for vertices, it will have the same index as the vertex has in the list of vertices.

```
26                 for units_to_sell in range(1, units_to_sell_max + 1):
27                     if closed_vertices_units[i] - units_to_sell <= q_max[
   t]:
28                         initial_vertex = closed_vertices[i]
29                         terminal_vertex = f"q_{t+1}_{
   closed_vertices_units[i]-units_to_sell}"
30                         edge_weight = e**(-0.04 * (t + 1) / terms) *
   units_to_sell * prices[t] * (-1) + int(added_constant)
31                         edge = (initial_vertex, terminal_vertex,
   edge_weight)
32                         edges.append(edge)
33                         if terminal_vertex not in vertices:
34                             vertices.append(terminal_vertex)
35                             vertices_units.append(closed_vertices_units[i
   ] - units_to_sell)
```

Summary 6.4: Creating the graph part 4

The loop beginning on line 26 does exactly the same for the case of units of gas being sold as the loop beginning on line 16 does for units of gas being bought. Generally the signs are changed in the calculations, and otherwise they are mostly similar.

```
36                 initial_vertex = closed_vertices[i]
37                 terminal_vertex = f"q_{t+1}_{closed_vertices_units[i]}"
38                 edge_weight = int(added_constant)
39                 edge = (initial_vertex, terminal_vertex, edge_weight)
40                 edges.append(edge)
41                 if terminal_vertex not in vertices:
42                     vertices.append(terminal_vertex)
43                     vertices_units.append(closed_vertices_units[i])
```

Summary 6.5: Creating the graph part 5

As neither the loop beginning on line 16 nor the loop beginning on line 26 covers the case where the number of gas units remain constant from one term to the next, this has to be

accounted for from line 36 through 40. The edge has the same properties as in the cases where gas is sold or bought, but in this case, the weight of the edge is equal to the added constant. The reason for this is that the equation for calculating weight is:

$$weight = e^{-r\frac{t}{T}} \cdot \Delta q_t \cdot P_t \cdot (-1) + constant. \tag{6.3}$$

When no units of gas are bought or sold, there is no change in gas units meaning $\Delta q_t = 0$. This is inserted in the equation 6.3:

$$weight = e^{-r\frac{t}{T}} \cdot 0 \cdot P_t \cdot (-1) + constant = constant. \tag{6.4}$$

Before the script continues with the cases, where the next term will be the last term, the terminal vertex is added to the list of vertices, if and only if it is not already in that list.

```
44              else:
45                  initial_vertex = closed_vertices[i]
46                  terminal_vertex = f"q_{t+1}_0"
47                  edge_weight = e**(-0.04 * (t + 1) / terms) *
    closed_vertices_units[i] * prices[t] * (-1) + int(added_constant)
48                  edge = (initial_vertex, terminal_vertex, edge_weight)
49                  edges.append(edge)
50      return edges, root
```

Summary 6.6: Creating the graph part 6

When the next term will be the last one, vertices in the current term must be the initial vertex for only one edge, and this edge must terminate in the vertex that represents the last term, and the storage containing 0 units of gas. Therefore a single edge connecting these vertices is created for each vertex in the second to last term.
Then the loop entered on line 7 is exited and the function returns a list containing all edges in the graph, and a root that was defined in line 3.

Since a way of representing the graph in python has been created, it is possible to introduce Dijkstra's Algorithm in python.

```
1 def dijkstra(edges, root, end):
2     start_vertex = [0, root[0], "init"]
3     open_vertices = []
4     open_vertices.append(start_vertex)
5     closed_vertices = []
6     active_vertex = start_vertex
```

Summary 6.7: Dijkstra's Algorithm part 1

The algorithm starts by identifying the starting vertex in line 2, where the way of storing data for vertices is also introduced. The data for a vertex is stored in a list where the first element, is a label for the currently known length to this vertex from the starting vertex. The second element is the name of the vertex itself. The last element, is a neighbouring

vertex that is the last vertex in the shortest known path from the starting vertex to the current vertex other than itself. Then it is stated that *open_ vertices* is a list. The list will contain all vertices for which a path with a length less than ∞ is known, and the starting vertex is then added to this list. In line 5 an empty list is defined. This list will act as the set S in the pseudocode for Dijkstra's Algorithm. As the starting vertex currently has the lowest label of the vertices not in S, it is chosen to be the active vertex, such that the label of other vertices can be updated if they are connected to the starting vertex by a single edge.

```
7     while end not in active_vertex:
8         i = 0
9         for edge in edges:
10            if edge[0] == active_vertex[1]:
11                open_vertex = [edge[2], edge[1], active_vertex[1]]
12                open_vertex[0] += active_vertex[0]
```

Summary 6.8: Dijkstra's Algorithm part 2

Then a *while* loop is entered on line 7, the condition for the loop repeating is that the goal vertex is not the active vertex, which indicates that it has been added to S. Later on in the algorithm i is used as a counting variable. On line 9 a *for* loop is entered that runs over every edge in the list containing all edges in the graph. This loop starts with an *if* statement that runs on the condition that the initial vertex of the current edge is equal to the name of the active vertex. If this is the case, then the terminal vertex is stored as the open vertex, such that its label can be updated. Lastly the first element of the open vertex is defined as the length from the starting vertex to the active vertex added to the length of the edge connecting these vertices.

```
13                if open_vertex not in closed_vertices:
14                    already_open = False
15                    known_dist = float('inf')
16                    i = 0
17                    for vertex in open_vertices:
18                        if open_vertex[1] == vertex[1]:
19                            already_open = True
20                            known_dist = vertex[0]
21                            index = i
22                        i += 1
23                    if not already_open or known_dist > open_vertex[0]:
24                        open_vertices.append(open_vertex)
25                        if already_open:
26                            open_vertices.pop(index)
```

Summary 6.9: Dijkstra's Algorithm part 3

The *if* statement on line 13 checks that the vertex stored as the open vertex has not already been added to S, because in that case we have already found the shortest path to that vertex, and therefore we do not need to examine it further. If the open vertex is not in S we assume that it has not already been opened before, and that the known label is ∞.

The *for* loop on line 17 runs over all the vertices for which a label less than ∞ is known. These vertices gets compared to the open vertex, to see if we already know a distance to the open vertex shorter than ∞. If that is the case, the known label is set to the previous label of the vertex. Then on line 23 through line 26 the label is updated for the vertex if the path trough the active vertex is shorter than the previously known path.

```
27          open_vertices.pop(0)
28          closed_vertices.append(active_vertex)
29          open_vertices = sorted(open_vertices)
30          active_vertex = open_vertices[0]
```

Summary 6.10: Dijkstra's Algorithm part 3

The loop entered on line 27 ends by removing the first element of the open vertices, which is the active vertex, and this vertex is then added to the list with closed vertices. Then the list with open vertices is sorted. At first the labels of the vertices are compared, if two vertices have the same label, then they are sorted alphabetically based on their names. Then the first vertex in the list of open vertices is chosen as the active vertex, and the loop repeats.

```
31      length = active_vertex[0]
32      path = []
33      i = 0
34      while start_vertex[1] not in path:
35          path.append(active_vertex[1])
36          parent = active_vertex[2]
37          for vertex in closed_vertices:
38              if parent == vertex[1]:
39                  active_vertex = vertex
40          i += 1
41      path.reverse()
42      return path, length
```

Summary 6.11: Dijkstra's Algorithm part 4

After the goal vertex is reached, the next step is to calculate the path that was taken. Hence the *while* loop initiated on line 7 is exited. A variable *length* is stated to be the label of the goal vertex. The variable *path* is stated to be an empty list, which will later contain the vertices visited by the shortest path to the goal vertex. On line 34 a *while* loop is entered on the condition, that the start vertex has not been added to the path. Then the active vertex is added to the path. Afterwards the algorithm examines which vertex let to discovering the shortest path to the active vertex, and then this vertex is defined to be the new active vertex, leading to the loop repeating. When the starting vertex has been added to the path, the loop exits, and the ordering of the list is reversed, such that the vertices are ordered in chronological order. Lastly, the algorithm returns the path and the length of the path.

Once both the algorithm for generating the graph and Dijkstra's Algorithm are defined, an additional function *main* is defined. The purpose of this function is to run the other

functions in correctly.

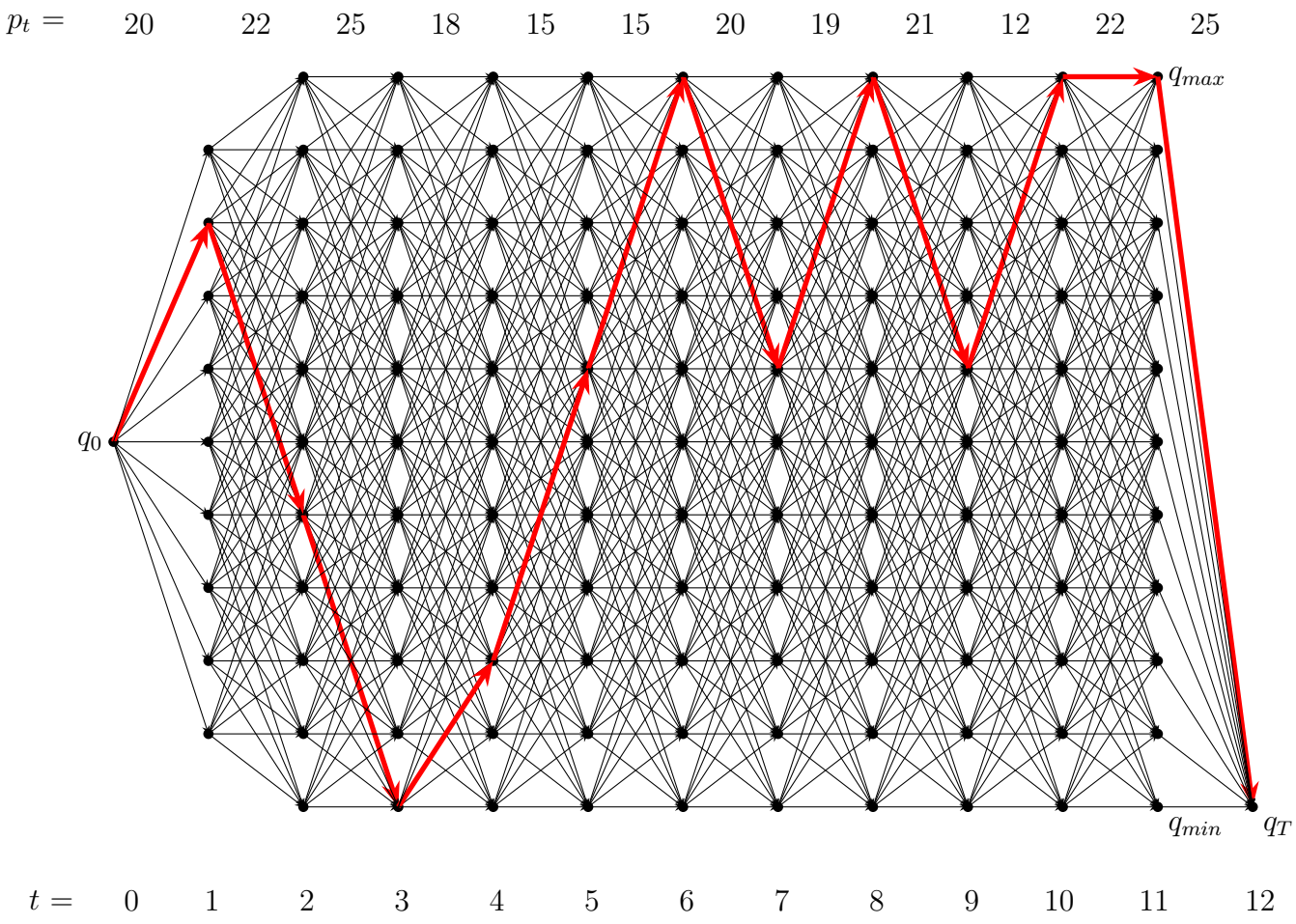
```
1  def main():
2      terms = 12
3      q_max = 10, 10, 10, 10, 8, 8, 8, 8, 10, 10, 10, 10
4      q_min = 0, 0, 0, 4, 4, 6, 6, 4, 4, 0, 0, 0
5      u_max = 4, 4, 2, 2, 1, 1, 1, 1, 2, 2, 4, 4
6      i_max = 4, 4, 2, 2, 1, 1, 1, 1, 2, 2, 4, 4
7      q_0 = 5
8      prices = [20, 22, 25, 18, 15, 15, 20, 19, 21, 12, 22, 25]
9      edges, root = make_edges(terms, q_max, q_min, u_max, i_max, q_0)
10     path, length = dijkstra(edges, root, "q_12_0")
11     length = (length - terms * (max(q_max) * max(prices) + 1)) * -1
12     print(f"The shortest path is {path} with a length of {length}")
```

Summary 6.12: Main function

The main function starts with statements in line 2 through 8 that define the parameters in the basis problem. Then the graph is generated in line 9, and Dijkstra's Algorithm is used on the graph in line 10. Then on line 11 length of the path is converted back to the actual problem graph, because the weights were adapted to be compatible with Dijkstra's Algorithm in the function for generating the graph.

6.5 Solution to The Basis Problem

The solution to the basis problem, is discovered by using the algorithm developed in 6.4. This algorithm finds the longest path through the directed weighted graph in 6.3 which is seen in the following:



While the algorithm finds the longest path through the graph, it also finds the profits by choosing this path. To calculate the profits earned, the algorithm uses the formula seen below:

$$-\sum_{t=1}^{T} e^{-r\frac{t}{T}} \Delta q_t p_t + e^{-r} q_T p_T \tag{6.5}$$

By using 6.5, the algorithm calculates the maximum profit:

$-((e^{-\frac{0.04\cdot 1}{12}} \cdot 3 \cdot 20) + (e^{-\frac{0.04\cdot 2}{12}} \cdot (-4) \cdot 22) + (e^{-\frac{0.04\cdot 3}{12}} \cdot (-4) \cdot 25) + (e^{-\frac{0.04\cdot 4}{12}} \cdot 2 \cdot 18) + (e^{-\frac{0.04\cdot 5}{12}} \cdot 4 \cdot 15) + (e^{-\frac{0.04\cdot 6}{12}} \cdot 4 \cdot 15) + (e^{-\frac{0.04\cdot 7}{12}} \cdot (-4) \cdot 20) + (e^{-\frac{0.04\cdot 8}{12}} \cdot 4 \cdot 19) + (e^{-\frac{0.04\cdot 9}{12}} \cdot (-4) \cdot 21) + (e^{-\frac{0.04\cdot 10}{12}} \cdot 4 \cdot 12) + (e^{-\frac{0.04\cdot 11}{12}} \cdot 0 \cdot 22) + (e^{-0.04} \cdot 10 \cdot 25)) = 252.7197791$

The algorithm finds that the largest possible profit which is 252.72€.
The details of the path containing the edges coloured in red is seen in the table below:

Table 6.3: Solution for the basis problem

t	*Units at t*	*Change at t*	*Profit/cost at t*	*Total profit/cost at t*
0	5	0	0.00	0.00
1	8	+3	-59.80	-59.80
2	4	-4	87.42	27.61
3	0	-4	99.00	126.62
4	2	+2	-35.52	91.09
5	6	+4	-59.00	32.09
6	10	+4	-58.81	-26.72
7	6	-4	78.15	51.43
8	10	+4	-74.00	-22.57
9	6	-4	81.52	58.95
10	10	+4	-46.43	12.52
11	10	0	0.00	12.52
12 = T	0	-10	240.19	252.72

From the optimal solution to maximize the profit displayed in 6.3, it can be seen that the maximum cost in the period is at time step 1, where there is a cost of 59.80€. Therefore, to achieve the highest possible profit during the period, it would require an investment of 59.80€ in the first period. On the other hand the net profit at the end would total 252.72€.

To calculate the average profit for each unit of gas in this path, we take the average price of 1 unit of gas bought and subtract it from the average sales price of 1 unit of gas.

Average buy price:

$$\frac{59.80 + 35.52 + 59 + 58.81 + 74 + 46.43}{26} = 12.829 \tag{6.6}$$

The average price, when buying 1 unit of gas in the optimal path is 12.829€

Average sell price:

$$\frac{87.42 + 99 + 78.15 + 81.52 + 240.19}{26} = 22.549 \tag{6.7}$$

The average selling price of a unit of gas in the optimal path is 22.549€

The average profit pr. unit of gas is calculated by:

$$22.549 - 12.829 = 9.72 \tag{6.8}$$

The average profit pr. unit bought and sold in the optimal path is 9.72€. Using the average profit pr. unit of gas it is possible to find the rate of return.

Definition 6.1.
A rate of return (RoR) is the net gain or loss of an investment over a specified time period, expressed as a percentage of the investment's initial cost.

The rate of return can be found by the following formula:

$$\text{Rate of return} = \frac{\text{Current value} - \text{Initial value}}{\text{Initial value}} \cdot 100\% \tag{6.9}$$

Using the formula from 6.1 the rate of return is calculated as follows:

$$\frac{9.72}{12.829} \cdot 100\% = 75.77\% \tag{6.10}$$

The rate of return in the optimal path is 75.77%.

When analysing these calculation, we see that the average profit pr. sold gas unit is 9.72€. This profit amounts to 75.77% increase, in relation to the amount of money spend on the units.

Note in the start of the period, the storage unit contains 5 units of gas. Since there are no costs associated with the contract of leasing the storage unit these 5 units are cost-free. This gives a arbitrary rate of return on invested capital. If the 5 free units of gas are subtracted from the total number of gas units sold, then the rate of return will change drastically. The 5 units of gas are now borrowed and used for trading throughout the period, but are returned in last period T. This means that the number of units of gas sold in this period T needs to be subtracted by 5 units of gas.

Average buy price:

$$\frac{59.80 + 35.52 + 59 + 58.81 + 74 + 46.43}{21} = 15.883 \tag{6.11}$$

The average price, when buying 1 unit of gas in the optimal path is 15.883€.

Average sell price:

$$\frac{87.42 + 99 + 78.15 + 81.52 + 120.1}{21} = 22.199 \tag{6.12}$$

The average selling price of a unit of gas in the optimal path is 22.199€.

The average profit pr. unit of gas is calculated by:

$$22.199 - 15.883 = 6.316 \tag{6.13}$$

The average profit pr. unit bought and sold in the optimal path is 6.316€.

$$\frac{6.316}{15.883} \cdot 100\% = 39.77\% \tag{6.14}$$

The rate of return in the optimal path is 39.77%.

When analysing these calculation, we see that the average profit pr. sold unit of gas is 6.316€. This profit amounts to 39.77%, which means that for every 100€ invested into natural gas, we expect 39.77€ in rate of return.

If we compare the rate of returns from equation 6.10 with equation 6.14:

$$\frac{39.77\% - 75.77\%}{75.77\%} \cdot 100\% = -47.51\%$$

It is seen how the rate of return decreases with −47.51%, when 5 units of gas are not given for free in the basis problem.

7 | Expanded Problem

In the following chapter the weight of edges in the presented directed weighted graphs are not displayed to make the graphs readable. The outline of the problem posted in this chapter, has been collected from [5].

7.1 Definition of The Expanded Problem

The expanded problem is based on the definition of the basis problem found in section 6.1. The expanded problem takes, that the maximum amount of gas units that can be imported or exported can change throughout the period into account. The minimum capacity q_{min} of the storage unit is greater during the winter, because of the typically greater demand for gas. This means that the upper and lower limit on the units of gas that can be contained in the storage unit are dependent on the time t. These limits are denoted by $q_{min,t}$ which is the lower limit at time t and $q_{max,t}$ which is the upper limit at time t.

In the basis problem, we are able to sell all remaining gas units, before the end of the period. In the expanded problem the contract includes the condition that a certain amount of gas has to be contained in the storage unit at the end of the period, which is denoted by q_{goal}. If the agreed amount of gas contained in the storage unit, at the end of the period is not met, the owner of the storage unit, will pay a reduced price for the remaining gas units. This reduction in price can be calculated using the penalty factor, denoted by α, the value of which is in the interval $0 < \alpha < 1$.

In the expanded problem the following data is given:

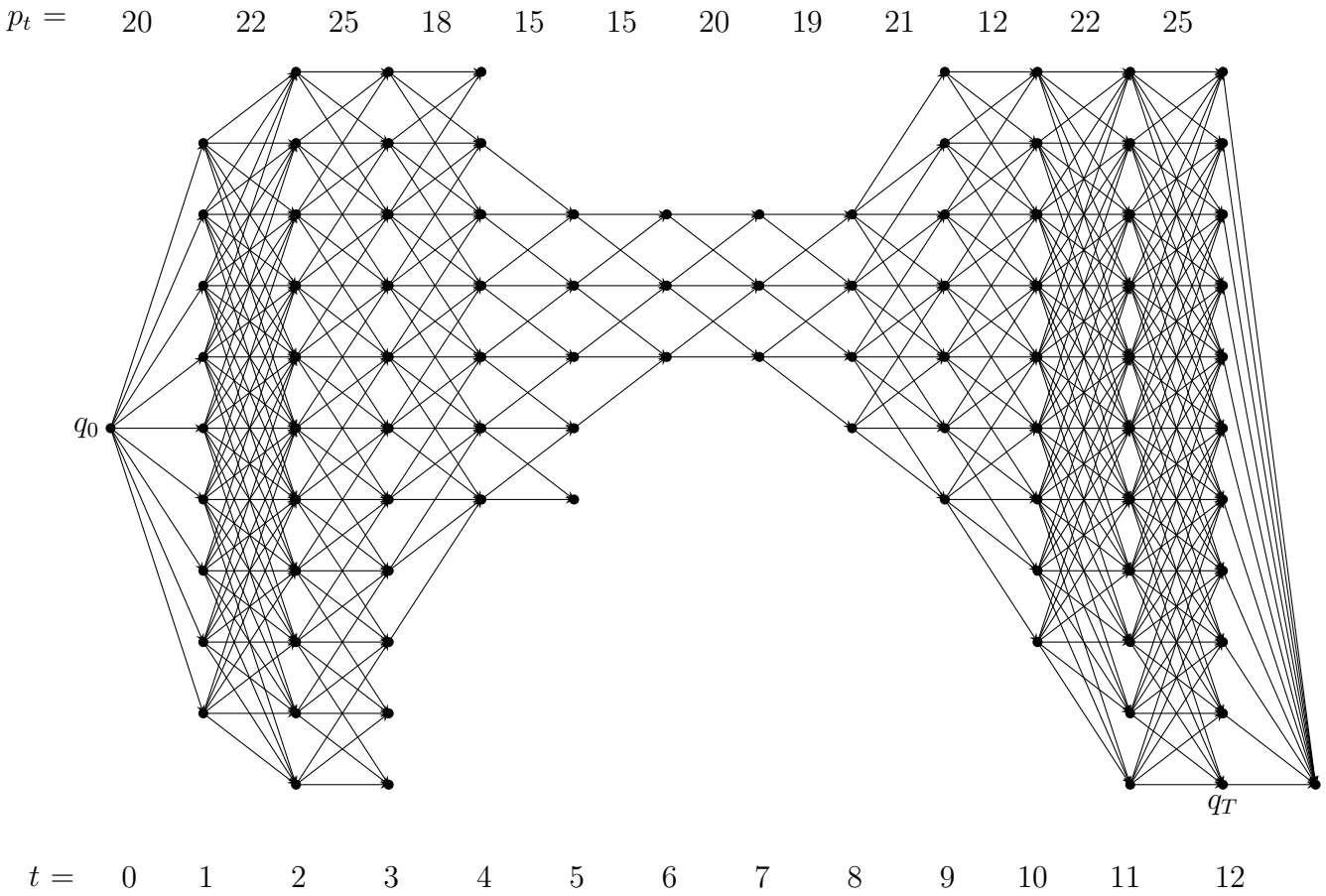
- The number of units of gas which needs to be stored in the storage unit at time T is: $q_{goal} = 5$.
- The penalty factor, $\alpha = 0.7$, is applied if $q_T \neq q_{goal}$, by subtracting α from 1 and then multiplying it by the product of the discount factor, units at time q_T and price at time p_T.
- The minimum capacity of the storage unit is: $(q_{min})_{t=1}^{T} = (0, 0, 0, 4, 4, 6, 6, 4, 4, 0, 0, 0)$. Note this means that period begins during the summer, due to q_{min} being the greatest during $t = 6$ and $t = 7$.
- The maximum capacity of the storage unit is: $(q_{max})_{t=1}^{T} = (10, 10, 10, 10, 8, 8, 8, 8, 10, 10, 10, 10)$.
- The maximum number of gas units, that can be imported into the storage unit between 2 periods is: $(i_{max})_{t=1}^{T} = (4, 4, 2, 2, 1, 1, 1, 1, 2, 2, 4, 4)$.
- The maximum number of gas units, that can be exported from the storage unit between 2 periods is: $(u_{max})_{t=1}^{T} = (4, 4, 2, 2, 1, 1, 1, 1, 2, 2, 4, 4)$.

The purpose of the expanded problem is to maximise the present value of the profit, with respect to Δq_t as given by the following expression:

$$\max_{\Delta q_t} \Big\{ -\sum_{t=1}^{T} e^{-r\frac{t}{T}} \Delta q_t p_t + e^{-r} q_T p_T (1 - \alpha \cdot \mathbb{1}[q_T \neq q_{goal}]) \Big\} \tag{7.1}$$

Note that the indicator function $\mathbb{1}[q_T \neq q_{goal}]$ has a value of 1 when $q_T \neq q_{goal}$ is true, and a value of 0 whenever $q_T \neq q_{goal}$ is false.

The directed weighted graph, of the expanded problem is illustrated below:



The graph illustrates different strategies for trading gas throughout the period. Note that some vertices have an out-degree of 0. These vertices are included in the graph but never used (excluding $q_{13,0}$) in any path, since it is possible to reach them, but it is not possible to reach any other vertices from these.

7.2 Using Python to Solve The Expanded Problem

Once again this graph would be difficult to navigate manually, as in the worst case $n!$ possible paths to q_{13} exists. Therefore python 3.11 is once again used to find the shortest path. Although one script has already been presented to solve the basis problem, changes to the script are necessary for the script to find the shortest path in the graph now that boundaries can vary at each time t and a penalty factor applies when the goal at the end of the period is not met. In this section we will only present the differences from the python script that solved the basis problem to the new python script that solves the expanded problem, and hence all lines of code not explained in this section are explained in section 6.4. When explaining differences in the two scripts, both the line from the basis problem and the line from the expanded problem is presented. The line from the basis problem is always presented before the line from the expanded problem.

```
1 def make_edges(terms, q_max, q_min, u_max, i_max, q_0, prices):
```

Summary 7.1: Creating the graph line 1

```
1 def make_edges(terms, q_max, q_min, u_max, i_max, q_0, prices, goal="Null
    ", penalty=0):
```

Summary 7.2: Creating the graph for the expanded problem line 1

For the function to create a graph, that takes q_{goal} into consideration, it takes two additional arguments compared to the function that creates the graph for the basis problem, namely *goal* and *penalty*. The argument *goal* has the value of the string *"null"* if it is not defined, and the variable *"penalty"* has the value of 0, when no argument for *penalty* is given.

```
20                         edge_weight = e**(-0.04 * (t + 1) / terms) *
    units_to_buy * (-prices[t]) * (-1) + int(added_constant)
```

Summary 7.3: Creating the graph line 20

```
20                         edge_weight = e**(-0.04 * (t + 1) / (terms - 1))
    * units_to_buy * (-prices[t]) * (-1) + int(added_constant)
```

Summary 7.4: Creating the graph for the expanded problem line 20

```
30                         edge_weight = e**(-0.04 * (t + 1) / terms) *
    units_to_sell * prices[t] * (-1) + int(added_constant)
```

Summary 7.5: Creating the graph line 30

```
30                         edge_weight = e**(-0.04 * (t + 1) / (terms - 1))
    * units_to_sell * prices[t] * (-1) + int(added_constant)
```

Summary 7.6: Creating the graph for the expanded problem line 30

In both line 20 and line 30 changes are made. In both terms 1 is subtracted from the number of terms, to get the actual number of terms. This is done, because an additional

term is given when calling the function. This makes it possible to not have 0 units of gas in the storage unit at the T'th term.

```
47      edge_weight = e**(-0.04 * (t + 1) / terms) * closed_vertices_units[i]
        * prices[t] * (-1) + int(added_constant)
```

Summary 7.7: Creating the graph line 47

```
47                  if goal != "Null" and goal != closed_vertices_units[i]:
48                      edge_weight = e**(-0.04) * closed_vertices_units[i] *
        prices[t] * (1 - penalty) * (-1) + int(added_constant)
49                  else:
50                      edge_weight = e**(-0.04) * closed_vertices_units[i] *
        prices[t] * (-1) + int(added_constant)
```

Summary 7.8: Creating the graph for the expanded problem line 47 - 50

In line 47 the weights of the edges incident to the goal vertex are calculated in a different way. This is done because if we do not visit q_{goal} a penalty is applied to the price of the remaining units of gas in the storage unit. Therefore line 47 is replaced by an *if* statement that multiplies the price of the remaining units of gas by 1 subtracted by the penalty factor if a q_{goal} is defined and it is not met. If q_{goal} is hit or it is not defined, the owner will pay the full price for the remaining units of gas.

The same version of Dijkstra's Algorithm as the one introduced in section 6.4 is used to solve the expanded problem.

The only differences in the main function compared to the one used in the basis problem is that an additional term with the properties of the term T.

```
1  def main():
2      terms = 12
3      q_max = 10, 10, 10, 10, 8, 8, 8, 8, 10, 10, 10, 10
4      q_min = 0, 0, 0, 4, 4, 6, 6, 4, 4, 0, 0, 0
5      u_max = 4, 4, 2, 2, 1, 1, 1, 1, 2, 2, 4, 4
6      i_max = 4, 4, 2, 2, 1, 1, 1, 1, 2, 2, 4, 4
7      q_0 = 5
8      prices = [20, 22, 25, 18, 15, 15, 20, 19, 21, 12, 22, 25]
9      edges, root = make_edges(terms, q_max, q_min, u_max, i_max, q_0)
10     path, length = dijkstra(edges, root, "q_12_0")
11     length = (length - terms * (max(q_max) * max(prices) + 1)) * -1
12     print(f"The shortest path is {path} with a length of {length}")
```

Summary 7.9: Main function

```
1  def main():
2      terms = 13
3      q_max = 10, 10, 10, 10, 8, 8, 8, 8, 10, 10, 10, 10, 10
4      q_min = 0, 0, 0, 4, 4, 6, 6, 4, 4, 0, 0, 0, 0
5      u_max = 4, 4, 2, 2, 1, 1, 1, 1, 2, 2, 4, 4, 4
6      i_max = 4, 4, 2, 2, 1, 1, 1, 1, 2, 2, 4, 4, 4
7      q_0 = 5
```

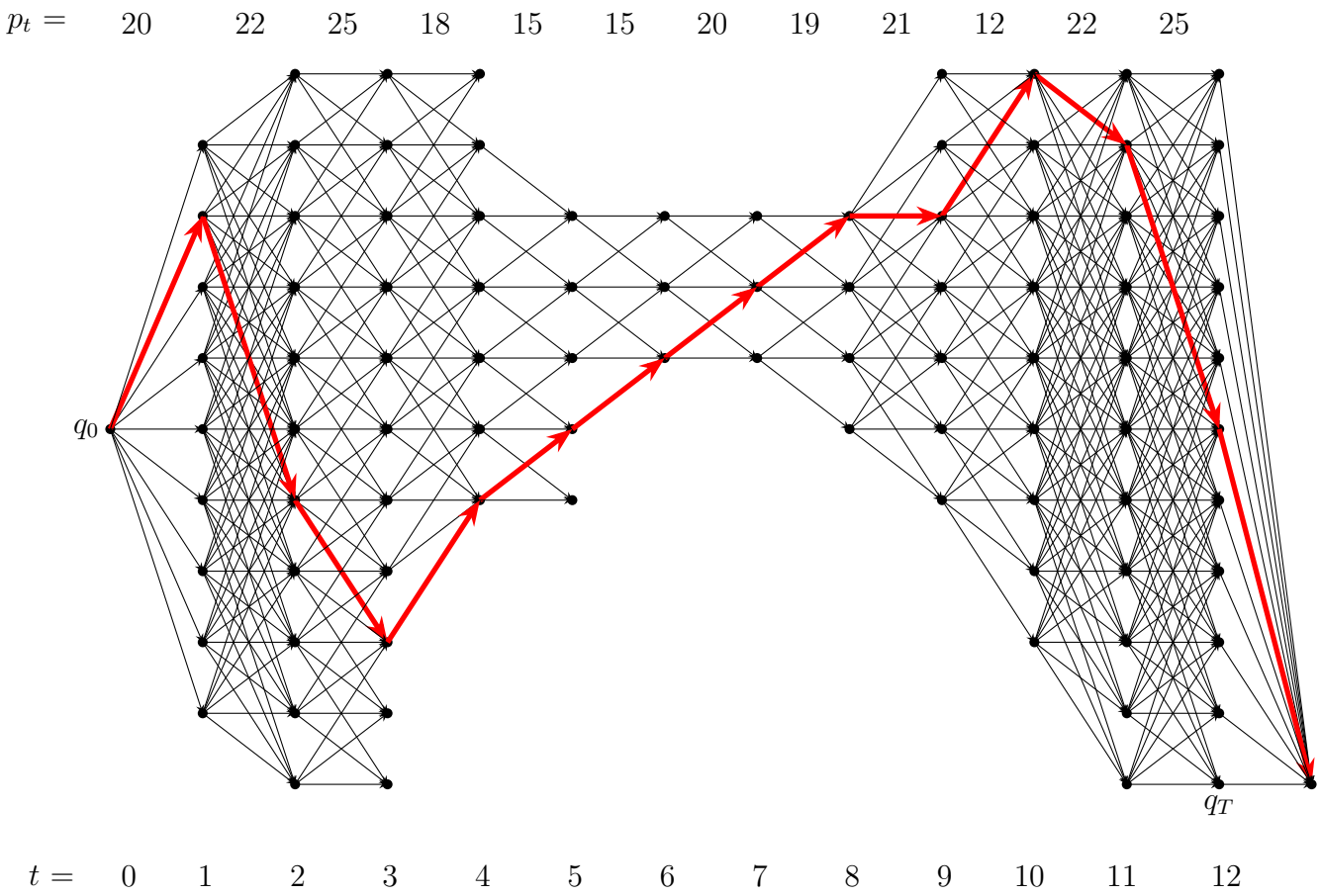
```
    prices = [20, 22, 25, 18, 15, 15, 20, 19, 21, 12, 22, 25, 25]
    edges, root = make_edges(terms, q_max, q_min, u_max, i_max, q_0,
prices, 5, 0.7)
    path, length = dijkstra(edges, root, "q_13_0")
    length = (length - terms * (max(q_max) * max(prices) + 1)) * -1
    print(f"The shortest path is {path} with a length of {length}")
```

Summary 7.10: Main function

The changes in the main function are made, such that it is possible to meet q_{goal} using Dijkstra's Algorithm.

7.3 Solution to The Expanded Problem

The solution to the expanded problem, is discovered by using the algorithm presented in section 7.2. This algorithm finds the longest path through the directed weighted graph in section 7.1 which is seen in the following:



The algorithm finds the longest path through the graph, but it also finds the profit by choosing this path. To calculate the profit earned, the algorithm uses the formula seen below

with the appropriate weights of the edges traversed by the path:

$$Profit = -\sum_{t=1}^{T} e^{-r\frac{t}{T}} \Delta q_t p_t + e^{-r} q_T p_T (1 - \alpha \cdot \mathbb{1}[q_T \neq q_{goal}]) \tag{7.2}$$

By using the formula from 7.2, the algorithm calculates the maximum profit:

$-((e^{\frac{-0.04\cdot 1}{12}} \cdot 3 \cdot 20) + (e^{\frac{-0.04\cdot 2}{12}} \cdot -4 \cdot 22) + (e^{\frac{-0.04\cdot 3}{12}} \cdot -2 \cdot 25) + (e^{\frac{-0.04\cdot 4}{12}} \cdot 2 \cdot 18) + (e^{\frac{-0.04\cdot 5}{12}} \cdot 1 \cdot 15) + (e^{\frac{-0.04\cdot 6}{12}} \cdot 1 \cdot 15) + (e^{\frac{-0.04\cdot 7}{12}} \cdot 1 \cdot 20) + (e^{\frac{-0.04\cdot 8}{12}} \cdot 1 \cdot 19) + (e^{\frac{-0.04\cdot 9}{12}} \cdot 0 \cdot 21) + (e^{\frac{-0.04\cdot 10}{12}} \cdot 2 \cdot 12) + (e^{\frac{-0.04\cdot 11}{12}} \cdot -1 \cdot 22) + (e^{0.04} \cdot -4 \cdot 25)) + (e^{0.04} \cdot -5 \cdot 25)(1 - 0.7 \cdot 0)) = 188.27$

The algorithm finds that the largest possible profit which is 188.27€.
Note in the last term of the left side of the equation, the penalty factor $\alpha = 0.7$ is multiplied with 0, due to $q_T = q_{goal} = 5$. The details of the path containing the edges coloured in red is seen in the table below:

Table 7.1: Solution for the expanded problem

t	***Units at t***	***Change at t***	***Profit/cost at t***	***Total profit/cost at t***
0	5	0	0.00	0.00
1	8	+3	-59.80	-59.80
2	4	-4	87.42	27.61
3	2	-2	49.50	77.12
4	4	+2	-35.52	41.59
5	5	+1	-14.75	26.84
6	6	+1	-14.70	12.14
7	7	+1	-19.54	-7.40
8	8	+1	-18.50	-25.90
9	8	0	0	-25.90
10	10	+2	-23.21	-49.11
11	9	-1	21.20	-27.90
12 = T	5	-4	96.08	68.17
After 12	0	-5	120.10	188.27

Note that in table 7.1 the number of units of gas is 5 at time T and q_{goal} is therefore met. From the optimal solution to maximize the profit displayed in table 7.1, it can be seen that the maximum cost in the period is at time step 1, where there is a cost of at least 59.80€. Therefore, to achieve the highest possible profit during the period, it would require an investment of 59.80€ in the first period. On the other hand the net profit at the end would total 188.27€.

To calculate the average profit for each unit of gas in this path, we take the average price

of 1 unit of gas bought and subtract it from the average sales price of 1 unit of gas.

Average buy price:

$$\frac{59.80 + 35.52 + 14.75 + 14.7 + 19.54 + 18.50 + 23.21}{16} = 11.626 \tag{7.3}$$

The average price, when buying 1 unit of gas in the optimal path is 11.626€

Average sell price:

$$\frac{87.42 + 49.5 + 21.2 + 96.08 + 120.1}{16} = 23.394 \tag{7.4}$$

The average selling price of a unit of gas in the optimal path is 23.394€

The average profit pr. unit of gas is calculated by:

$$23.394 - 11.626 = 11.768 \tag{7.5}$$

The average profit pr. unit bought and sold in the optimal path is 11.768€. Using the average profit pr. unit of gas it is possible to find the rate of return.

Using the formula from definition 6.1 the rate of return is calculated as follows:

$$\frac{11.768}{11.626} \cdot 100\% = 101.195\% \tag{7.6}$$

The rate of return in the optimal path is 101.195%.

When analysing these calculation, we see that the average profit pr. sold gas unit is 11.768€. This profit amounts to 101.195% increase, in relation to the amount of money spend on the units.

Note that in the start of the period, the storage unit contains 5 units of gas. Since there are no costs associated with the contract of leasing the storage unit these 5 units are cost-free. This gives an arbitrary rate of return on invested capital. If the 5 free units of gas are subtracted from the total number of gas units sold, then the rate of return will change drastically. The 5 units of gas are now borrowed and used for trading throughout the period, but are returned in last period T. This means that the number of units of gas sold in this period T needs to be subtracted by 5 units of gas.

Average buy price pr. unit of gas:

$$\frac{59.80 + 35.52 + 59 + 58.81 + 74 + 46.43}{11} = 16.911 \tag{7.7}$$

The average price, when buying 1 unit of gas in the optimal path is 16.911 €.

Average sell price pr. unit of gas:

$$\frac{87.42 + 49.5 + 21.2 + 96.08}{11} = 23.109 \tag{7.8}$$

The average selling price of a unit of gas in the optimal path is 23.109 €.

The average profit pr. unit of gas is calculated by:

$$23.109 - 15.883 = 6.198 \tag{7.9}$$

The average profit pr. unit bought and sold in the optimal path is 6.198 €.

$$\frac{6.198}{16.911} \cdot 100\% = 36.65\% \tag{7.10}$$

The rate of return in the optimal path is 36.65%.

When analysing these calculations, we see that the average profit pr. sold unit of gas is 6.198 €. This profit amounts to 36.65%, which means that for every 100 € invested into natural gas, we expect 36.65 € in rate of return.

If we compare the rate of returns from equation 7.6 with equation 7.10:

$$\frac{36.65\% - 101.195\%}{101.195\%} \cdot 100\% = -63,78\%$$

It is seen how the rate of return decreases with −63.78%, when 5 units of gas are not given for free in the expanded problem.

Is it ever worth it to not hit the set goal in relation to the penalty factor?

In the solution to the expanded problem, it is seen that the optimal strategy meets q_{goal}. To research if this is always the case, the algorithms are run with different values for α which is 0.7 in the expanded problem. The first value of α where the optimal strategy

does not meet q_{goal} is displayed below and the first value after that where the strategy does meet q_{goal}.

α	Does the optimal strategy meet q_{goal}?
0.0195102	No
0.0195103	Yes

Table 7.2: Penalty factor and if the optimal strategy meets q_{goal}

As seen in table 7.2, the penalty factor becomes insignificantly small whenever it is less than $\alpha = 0.0195102$. This table is based on the output of the function in appendix A.5

8 | Discussion

When solving the problems posed in this project, some choices had to be made regarding the choice of approach to solve the problem, which in our case was graph theory and in graph theory the choice of algorithm to solve the longest-path problem. Additionally some parameters were given that could also have changed the outcome of the project if they were different. The choice of algorithm and price parameter will be discussed in the following sections. The definition of calls and puts used in this chapter are found in [**?**].

8.1 Choice of Algorithm

To solve the problems posed in this project, Dijkstra's Algorithm has been implemented. Even though we chose this algorithm, different algorithms are capable of solving the problems. The reasons for choosing Dijkstra's Algorithm will be explained in this section.

When considering different algorithms to implement, these are assessed based on different parameters such as speed, correctness and simplicity. When examining the speed of which an algorithm finds the shortest path, a way to compare algorithms was introduced in section 4.3. This way of comparing the speeds of algorithms by comparing the worst case time complexity of algorithms will be used.

In subsection 4.1.4 brute force algorithms were presented as a type of algorithm that finds a solution by exhaustion as defined in 4.9. A property of a brute force algorithm is that it always finds an optimal solution, therefore when assessing the correctness of such an algorithm, we know that the solution is correct. However, when looking at the time complexity of a brute force algorithm for finding the shortest path, it would consider all possible paths from the start vertex to the goal vertex, and have a complexity of $\Theta(n!)$ [1, p. 237-238]. When assessing the simplicity of using a brute force algorithm, it does not have extra complications if the graph contains negative weights or if it has to find the longest path. Therefore the graph would not have to be adapted to fit such an algorithm. As stated in subsection 4.3.4 Dijkstra's Algorithm has a complexity of $\Theta(n^2)$, and it always finds an optimal path as proven in subsection 4.2.2. However unlike a brute force algorithm, the graph needs to be adapted such that Dijkstra's Algorithm can find the shortest path. When comparing the two algorithms, it is clear that they both find an optimal solution. The brute force algorithm is easier to implement, than Dijkstra's Algorithm, but Dijkstra's Algorithm has a much better worst case time complexity. Since it is fairly simple to adapt the graph to fit Dijkstra's Algorithm, and the complexity of a brute force algorithm is much worse in cases with large input sizes, the speed of solving the problem is a fair trade off for the extra work needed to adapt the graph.

8.2 Natural Gas Prices

In this project, a price for trading units of gas in different time intervals were given. In practice, this is done by using call or put options. Call and put options are contracts, where the buyer is entitled, but not obligated, to either buy(call) or sell(put) securities at a specific price at a specific date. The agreed upon time where the buyer is allowed to exercise their right of buying(call) or selling(put) securities is known as the exercise time, while the agreed upon price is called the strike price or exercise price. If the price of the security has increased further than the strike price, then a buyer of a call option, is able to buy the security from the seller of the obligation at the strike price and sell it at the market price. On the other hand if the price of the security is lower, then the call option is worthless, because there would be no point in buying the security at a higher price than the market price. Vice versa, if the price of the security has decreased further than the strike price, then a buyer of a put option, is able to buy the security at the market price, and sell the security to the seller of the obligation at the strike price.

In the basis problem, we engage in contracts with the counterparts on the market to negotiate the prices we are able to buy and sell units of gas for at a given time in the period. These contracts most likely consists of call and put options. In the problem we are only allowed to buy and sell natural gas, to some predetermined seller of the options. Therefore we do not care about the market price for natural gas, because we are not able to profit off of buying from or selling to the market. Usually when buying call or put options, the buyer must pay a premium to the seller of the options. Costs related to this premium and the contract are not taken into account in the basis problem or the expanded problem. This makes the solution posed in the project further from how trading of gas would function in practice.

If the basis problem does not use options, then the prices we are able to buy or sell natural gas are merely a projection of the future market prices of natural gas. If this is the case, then our calculations of profits are a mere projection and not precise.

9 | Perspectivation

In this project, graph theory and algorithms have been used to find the optimal strategy for buying and selling gas in a given period. Such algorithms can be utilized when solving a shortest path problem, which includes a different variety of problems. Another problem of this type could be, when a car has to travel to a set of cities at least once, while covering the least distance. In this problem cities are represented with vertices, and roads are represented with weighted edges, which allows for the drawing of a weighted graph. In this weighted graph, the problem is solved as a shortest path problem, where Dijkstra's algorithm discovers the path that visits all cities while travelling the least distance.

In the solution to the problem posed in this project, all parameters have been given, therefore we did not need to assume anything about the model created to find the optimal solution. Therefore the model is said to be a *white box* model. In reality unknown parameters such as exactly how inflation impacts the value of money exists, and hence the model would be a *grey box* model. In a white box problem, all internal parameters are known, and a very precise model can therefore be made, such that it is certain that the model perfectly describes the system. In a grey box problem, some internal parameters are known, but not all. Therefore some parts of the model can be accurately constructed, whereas other parts of the model needs to be constructed without knowledge of internal workings of the system. Therefore it is not known if the model exactly fits the system in a grey box model.

Several parameters could have made this problem more realistic. These parameters could, for example, be fixed costs, a variable interest rate over the period of time in regards to inflation and generally more unknown factors that would make it difficult to determine the optimal strategy before the period is already over.

The future prices of natural gas are defined in [5, p. 5], but the problem outline does not explain how these prices are obtained. In reality this is done by using options as explained in section 8.2. As explained in this section, a premium must be paid when options are traded. Since the premiums when trading options are not taken into account, the preciseness of the problem outline when obtaining the prices of natural gas is insufficient when comparing it to a real life scenario.

10 | Conclusion

Graph theory can be used to represent different strategies for buying and selling natural gas during a given period of time. The different types of strategies for buying and selling natural gas can be represented with graphs. These graphs represent different strategies by using vertices and edges. Vertices can be used to represent the number of units of gas at any given moment in time, while edges can be used to represent, the price of buying, selling or holding natural gas. To represent a problem where time is involved as in this case where we are able to buy, sell or hold between two time periods, directed graphs are used such that the linearity of time is complied with. Therefore edges are directed subsequently from one time to the next chronological time step. The weight of the paths contained in the graph, is the profit or cost generated by following these strategies for buying and selling natural gas.

When the graphs for modelling different trading strategies get larger, it becomes more tedious to find the optimal path. Therefore algorithms are introduced in chapter 4 in order to find the optimal strategy, which in this case will be the longest path. Both Dijkstra's Algorithm and a brute force algorithm for finding the longest path, are capable of finding the longest path from one vertex to another through a graph, although changes to the graph may be necessary, such that Dijkstra's Algorithm works properly. In this project Dijkstra's Algorithm was chosen to find the optimal strategy, because this algorithm requires less computations compared to a brute force algorithm. Yet again going through the steps of such an algorithm manually becomes tedious when analyzing bigger graphs. Therefore Python is introduced in order to solve the problems quicker than following the steps of Dijkstra's Algorithm manually, while still being correct.

In this project we have implemented an algorithm in python by using graph theory to determine how much gas should be bought or sold at a given time to maximize the profit or minimize the losses during a given period. The algorithm was utilized when solving the basis problem, while the modified version of the algorithm was utilized when solving the expanded problem.

11 | Bibliography

[1] Rosen K. In: Discrete Mathematics and Its Application. McGraw-Hill Education; 2019. .

[2] Essex C, Adams R. In: Calculus Volume 1. Pearson Canada inc.; 2022. .

[3] Capinski M, Zastawniak T. In: Mathmatics for Finance. Springer; 2003. p. 21–34.

[4] L'Hôpital's rule review [site]. Khan Academy; 2010 [visited 05-12-2022]. Available from: https://www.khanacademy.org/math/differential-calculus/dc-context-app/dc-lhopital-composite-exp/a/lhopitals-rule-review.

[5] Optimering af gaslager [PDF]. Aalborg University; 2021 [visited 24-11-2022]. Available from: https://www.moodle.aau.dk/pluginfile.php/2695460/mod_resource/content/3/matok%20P1%20projekt.pdf.

[6] Rate of Return (RoR) Meaning, Formula, and Examples [site]. Investopedia; 2010 [visited 15-12-2022]. Available from: https://www.investopedia.com/terms/r/rateofreturn.asp.

[7] Optimering af gaslager [PDF]. Aalborg University; 2021 [visited 15-12-2022]. Available from: https://projekter.aau.dk/projekter/files/459937110/P1_B_345.pdf.

A | Appendix

A.1 The Triangle Inequality

The following is from [2, p. 8].

> **Theorem A.1. The triangle inequality**
>
> The triangle inequality asserts that the absolute value of a sum of or difference between numbers is less than or equal to the sum of their absolute values. This inequality is given by:
>
> $|a \pm b| \leq |a| + |b|$.

Proof. Proof of the the triangle inequality

To prove theorem A.1, we start of by defining the absolute value. The absolute value of a given integer x, is given by: $|x| = max(x, -x)$. Therefore it must follow that $-x = |x|$ and $+x = |x|$, when $x = 0$ and $x < |x|$, when $x < 0$. Furthermore $x = |x|$, when $x > 0$. Thus it must follow that: $\pm x \leq |x|$.

We only have 4 cases to consider without loss of generality. These four cases are:

- $x = 0$
- $x > 0$, $y > 0$
- $x < 0$, $y < 0$
- $x > 0$, $y < 0$

Case 1: $x = 0$ As we have asserted above $x = |x|$, when $x = 0$, therefore we have:

$$|x| = 0 \tag{A.1}$$

Therefore we have:

$$|x| + |y| = 0 + |y| = |y| \tag{A.2}$$

and because $|x + y| = |0 + y| = |y|$, it must follow that in this case:

$$|x| + |y| = |x + y| \tag{A.3}$$

note that the same is true if $y = 0$ and when both x and y are zero.

Case 2: $x > 0$ and $y > 0$

In this case it must follow that the sum of x and y is also positive since they are both positive nonzero integers:

$$x + y > 0 \tag{A.4}$$

Thus we now from the definition of absolute values that x is equal to the absolute value of x and likewise for y:

$$|x| = x$$
$$|y| = y$$

Therefore the sum of the two absolute values of the integers must be equal to the absolute value of the sum of the two integers:

$$|x| + |y| = x + y$$
$$|x + y| = x + y$$

Thus in this case when x and y are both nonzero positive integers it must follow that:

$$|x| + |y| = |x + y| \tag{A.5}$$

Case 3: $x < 0$ and $y < 0$

Because x and y are both nonzero negative integers it must follow that:

$$|x| = -x$$
$$|y| = -y$$

The minus in front of the x and y asserts that the negative integer changes sign from positive to negative. We now take the sum of the two absolute values and the absolute value of the sum of the two integers:

$$|x| + |y| = (-x) + (-y)$$
$$|x + y| = -(x + y) = (-x) + (-y)$$

Therefore we can conclude that: $|x| + |y| = |x + y|$, when $x < 0$ and $y < 0$.

Case 4: $x > 0$ and $y < 0$

When x is a positive nonzero integer it follows that:

$|x| = x$

When y is a negative nonzero integer it must follow that:

$|y| = -y$

This brings us to the equation:

$$|x| + |y| = x + (-y) \tag{A.6}$$

This bring us to 3 sub cases to consider:

- $x + y = 0$
- $x + y > 0$
- $x + y < 0$

Case 4a: $x + y = 0$

$$|x + y| = |0| = 0. \tag{A.7}$$

Since we know that $y < 0$ and it follows that $-y > 0$, it must follow that:

$$x + (-y) > |x + y| = |0| = 0. \tag{A.8}$$

Thus in this case it must follow that:

$|x| + |y| > |x + y|$, when $x > 0$ and $y < 0$ and $x + y = 0$.

Case 4b: $x + y > 0$

$|x + y| = x + y$.

Again it must follow that $-y > 0 > y$ since y is a negative nonzero integer. Thus we must have:

$$|x| + |y| = x + (-y) > x + y. \tag{A.9}$$

Thus it must follow in this case that:

$|x| + |y| > |x + y|$, when $x > 0$, $y > 0$ and $x + y > 0$.

Case 4c: $x + y < 0$

Because we know that the sum of x and y is negative we can move the minus out of a parenthesis when taking the absolute value of the two integers:

$|x + y| = -(x + y) = (-x) + (-y)$

We know that $x > 0 > -x$, therefore it must follow that:

$x + (-y) > (-x) + (-y)$.

Since the sum of the absolute value of the two integers is given by, $|x| + |y| = x + (-y)$, it must follow that $|x| + |y| > |x + y|$.

Therefore we have established that the triangle inequality holds for all cases of x and y values, which completes our proof of theorem A.1.

■

A.2 The Binomial Theorem

In the following definition we used [2, p. 555] in section 9.8.

Theorem A.2. The binomial theorem

The binomial theorem also denoted by the binomial expansion describes the expansion of powers of a binomial. The binomial theorem is given by:

$$(a+x)^n = a^n + na^{n-1}x + \frac{n(n-1)}{2!}a^{n-2}x^2 + \cdots + nax^{n-1} + x^n = \sum_{k=0}^{n} \binom{n}{k} a^{n-k}x^k$$

where

$$\binom{n}{k} = \frac{n!}{(n-k)!k!},$$

where $a, x \in \mathbb{R}$ and $n \in \mathbb{N}$.

The Binomial Theorem is proven in 3.2.

A.3 Source Code Used to Solve The Basis Problem

```
def make_edges(terms, q_max, q_min, u_max, i_max, q_0, prices, goal="Null
    ", penalty=0):
    edges = []
    root = [f"q_0_{q_0}"]
    vertices = root
    vertices_units = [q_0]
    added_constant = max(prices) * max(q_max) + 1
```

```
    for t in range(terms):
        closed_vertices = vertices
        closed_vertices_units = vertices_units
        vertices = []
        vertices_units = []
        for i in range(len(closed_vertices)):
            units_to_buy_max = min((q_max[t] - closed_vertices_units[i]), u_max[t])
            units_to_sell_max = min((closed_vertices_units[i] - q_min[t]), i_max[t])
            if not t == terms - 1:
                for units_to_buy in range(1, units_to_buy_max + 1):
                    if closed_vertices_units[i] + units_to_buy >= q_min[t]:
                        initial_vertex = closed_vertices[i]
                        terminal_vertex = f"q_{t+1}_{units_to_buy+closed_vertices_units[i]}"
                        edge_weight = e**(-0.04 * (t + 1) / terms) * units_to_buy * (-prices[t]) * (-1) + int(added_constant)
                        edge = (initial_vertex, terminal_vertex, edge_weight)
                        edges.append(edge)
                        if terminal_vertex not in vertices:
                            vertices.append(terminal_vertex)
                            vertices_units.append(closed_vertices_units[i] + units_to_buy)
                for units_to_sell in range(1, units_to_sell_max + 1):
                    if closed_vertices_units[i] - units_to_sell <= q_max[t]:
                        initial_vertex = closed_vertices[i]
                        terminal_vertex = f"q_{t+1}_{closed_vertices_units[i]-units_to_sell}"
                        edge_weight = e**(-0.04 * (t + 1) / terms) * units_to_sell * prices[t] * (-1) + int(added_constant)
                        edge = (initial_vertex, terminal_vertex, edge_weight)
                        edges.append(edge)
                        if terminal_vertex not in vertices:
                            vertices.append(terminal_vertex)
                            vertices_units.append(closed_vertices_units[i] - units_to_sell)
                initial_vertex = closed_vertices[i]
                terminal_vertex = f"q_{t+1}_{closed_vertices_units[i]}"
                edge_weight = int(added_constant)
                edge = (initial_vertex, terminal_vertex, edge_weight)
                edges.append(edge)
                if terminal_vertex not in vertices and q_min[t] <= closed_vertices_units[i] <= q_max[t]:
                    vertices.append(terminal_vertex)
                    vertices_units.append(closed_vertices_units[i])
            else:
```

```
                initial_vertex = closed_vertices[i]
                terminal_vertex = f"q_{t+1}_0"
                if goal != "Null" and goal != closed_vertices_units[i]:
                    edge_weight = e**(-0.04) * closed_vertices_units[i] *
    prices[t] * (1 - penalty) * (-1) + int(added_constant)
                else:
                    edge_weight = e**(-0.04) * closed_vertices_units[i] *
    prices[t] * (-1) + int(added_constant)
                edge = (initial_vertex, terminal_vertex, edge_weight)
                edges.append(edge)
    return edges, root

def dijkstra(edges, root, end):
    start_vertex = [0, root[0], "init"]
    open_vertices = []
    open_vertices.append(start_vertex)
    closed_vertices = []
    active_vertex = start_vertex
    while end not in active_vertex:
        i = 0
        for edge in edges:
            if edge[0] == active_vertex[1]:
                open_vertex = [edge[2], edge[1], active_vertex[1]]
                open_vertex[0] += active_vertex[0]
                if open_vertex not in closed_vertices:
                    already_open = False
                    known_dist = float('inf')
                    i = 0
                    for vertex in open_vertices:
                        if open_vertex[1] == vertex[1]:
                            already_open = True
                            known_dist = vertex[0]
                            index = i
                        i += 1
                    if not already_open or known_dist > open_vertex[0]:
                        open_vertices.append(open_vertex)
                        if already_open:
                            open_vertices.pop(index)
        open_vertices.pop(0)
        closed_vertices.append(active_vertex)
        open_vertices = sorted(open_vertices)
        active_vertex = open_vertices[0]
    length = active_vertex[0]
    path = []
    i = 0
    while start_vertex[1] not in path:
        path.append(active_vertex[1])
        parent = active_vertex[2]
        for vertex in closed_vertices:
            if parent == vertex[1]:
                active_vertex = vertex
```

```
94          i += 1
95      path.reverse()
96      return path, length
97
98  def main():
99      terms = 13
100     q_max = 10, 10, 10, 10, 8, 8, 8, 8, 10, 10, 10, 10, 10
101     q_min = 0, 0, 0, 4, 4, 6, 6, 4, 4, 0, 0, 0, 0
102     u_max = 4, 4, 2, 2, 1, 1, 1, 1, 2, 2, 4, 4, 4
103     i_max = 4, 4, 2, 2, 1, 1, 1, 1, 2, 2, 4, 4, 4
104     q_0 = 5
105     prices = [20, 22, 25, 18, 15, 15, 20, 19, 21, 12, 22, 25, 25]
106     edges, root = make_edges(terms, q_max, q_min, u_max, i_max, q_0,
        prices, 5, 0.7)
107     path, length = dijkstra(edges, root, "q_13_0")
108     length = (length - terms * (max(q_max) * max(prices) + 1)) * -1
109     print(f"Den korteste vej er {path} med en l ngde  p   {length}")
110 main()
```

Summary A.1: Source code

A.4 Source Code Used to solve The Expanded Problem

```
1  import numpy as np
2
3  def make_edges(terms, q_max, q_min, u_max, i_max, q_0, prices, goal="Null
       ", penalty=0):
4      edges = []
5      root = [f"q_0_{q_0}"]
6      vertices = root
7      vertices_units = [q_0]
8      added_constant = max(prices) * max(q_max) + 1
9      for t in range(terms):
10         closed_vertices = vertices
11         closed_vertices_units = vertices_units
12         vertices = []
13         vertices_units = []
14         for i in range(len(closed_vertices)):
15             units_to_buy_max = min((q_max[t] - closed_vertices_units[i]),
       i_max[t])
16             units_to_sell_max = min((closed_vertices_units[i] - q_min[t])
       , u_max[t])
17             if not t == terms - 1:
18                 for units_to_buy in range(1, units_to_buy_max + 1):
19                     if closed_vertices_units[i] + units_to_buy >= q_min[t
       ]:
20                         initial_vertex = closed_vertices[i]
21                         terminal_vertex = f"q_{t+1}_{units_to_buy+
       closed_vertices_units[i]}"
```

```
22                          edge_weight = e**(-0.04 * (t + 1) / (terms - 1))
    * units_to_buy * (-prices[t]) * (-1) + int(added_constant)
23                          edge = (initial_vertex, terminal_vertex,
    edge_weight)
24                          edges.append(edge)
25                          if terminal_vertex not in vertices:
26                              vertices.append(terminal_vertex)
27                              vertices_units.append(closed_vertices_units[i
    ] + units_to_buy)
28                  for units_to_sell in range(1, units_to_sell_max + 1):
29                      if closed_vertices_units[i] - units_to_sell <= q_max[
    t]:
30                          initial_vertex = closed_vertices[i]
31                          terminal_vertex = f"q_{t+1}_{
    closed_vertices_units[i]-units_to_sell}"
32                          edge_weight = e**(-0.04 * (t + 1) / (terms - 1))
    * units_to_sell * prices[t] * (-1) + int(added_constant)
33                          edge = (initial_vertex, terminal_vertex,
    edge_weight)
34                          edges.append(edge)
35                          if terminal_vertex not in vertices:
36                              vertices.append(terminal_vertex)
37                              vertices_units.append(closed_vertices_units[i
    ] - units_to_sell)
38                  initial_vertex = closed_vertices[i]
39                  terminal_vertex = f"q_{t+1}_{closed_vertices_units[i]}"
40                  edge_weight = int(added_constant)
41                  edge = (initial_vertex, terminal_vertex, edge_weight)
42                  edges.append(edge)
43                  if terminal_vertex not in vertices and q_min[t] <=
    closed_vertices_units[i] <= q_max[t]:
44                      vertices.append(terminal_vertex)
45                      vertices_units.append(closed_vertices_units[i])
46              else:
47                  initial_vertex = closed_vertices[i]
48                  terminal_vertex = f"q_{t+1}_0"
49                  if goal != "Null" and goal != closed_vertices_units[i]:
50                      edge_weight = e**(-0.04) * closed_vertices_units[i] *
     prices[t] * (1 - penalty) * (-1) + int(added_constant)
51                  else:
52                      edge_weight = e**(-0.04) * closed_vertices_units[i] *
     prices[t] * (-1) + int(added_constant)
53                  edge = (initial_vertex, terminal_vertex, edge_weight)
54                  edges.append(edge)
55      return edges, root
56
57  def dijkstra(edges, root, end):
58      start_vertex = [0, root[0], "init"]
59      open_vertices = []
60      open_vertices.append(start_vertex)
61      closed_vertices = []
```

```
    active_vertex = start_vertex
    while end not in active_vertex:
        i = 0
        for edge in edges:
            if edge[0] == active_vertex[1]:
                open_vertex = [edge[2], edge[1], active_vertex[1]]
                open_vertex[0] += active_vertex[0]
                if open_vertex not in closed_vertices:
                    already_open = False
                    known_dist = float('inf')
                    i = 0
                    for vertex in open_vertices:
                        if open_vertex[1] == vertex[1]:
                            already_open = True
                            known_dist = vertex[0]
                            index = i
                        i += 1
                    if not already_open or known_dist > open_vertex[0]:
                        open_vertices.append(open_vertex)
                        if already_open:
                            open_vertices.pop(index)
        open_vertices.pop(0)
        closed_vertices.append(active_vertex)
        open_vertices = sorted(open_vertices)
        active_vertex = open_vertices[0]
    length = active_vertex[0]
    path = []
    i = 0
    while start_vertex[1] not in path:
        path.append(active_vertex[1])
        parent = active_vertex[2]
        for vertex in closed_vertices:
            if parent == vertex[1]:
                active_vertex = vertex
        i += 1
    path.reverse()
    return path, length

def main():
    terms = 13
    q_max = 10, 10, 10, 10, 8, 8, 8, 8, 10, 10, 10, 10, 10
    q_min = 0, 0, 0, 4, 4, 6, 6, 4, 4, 0, 0, 0, 0
    u_max = 4, 4, 2, 2, 1, 1, 1, 1, 2, 2, 4, 4, 4
    i_max = 4, 4, 2, 2, 1, 1, 1, 1, 2, 2, 4, 4, 4
    q_0 = 5
    prices = [20, 22, 25, 18, 15, 15, 20, 19, 21, 12, 22, 25, 25]
    edges, root = make_edges(terms, q_max, q_min, u_max, i_max, q_0,
   prices, 5, 0.7)
    path, length = dijkstra(edges, root, "q_13_0")
    length = (length - terms * (max(q_max) * max(prices) + 1)) * -1
    print(f"The shortest path is {path} with a length of {length}")
```

```
112
113 main()
```

A.5 Source Code for Finding an Insignificant Penalty Factor

```
1  from numpy import e as e
2  import numpy as np
3
4  def make_edges(terms, q_max, q_min, u_max, i_max, q_0, prices, goal="Null
       ", penalty=0):
5      edges = []
6      root = [f"q_0_{q_0}"]
7      vertices = root
8      vertices_units = [q_0]
9      added_constant = max(prices) * max(q_max) + 1
10     for t in range(terms):
11         closed_vertices = vertices
12         closed_vertices_units = vertices_units
13         vertices = []
14         vertices_units = []
15         for i in range(len(closed_vertices)):
16             units_to_buy_max = min((q_max[t] - closed_vertices_units[i]),
       i_max[t])
17             units_to_sell_max = min((closed_vertices_units[i] - q_min[t])
       , u_max[t])
18             if not t == terms - 1:
19                 for units_to_buy in range(1, units_to_buy_max + 1):
20                     if closed_vertices_units[i] + units_to_buy >= q_min[t
       ]:
21                         initial_vertex = closed_vertices[i]
22                         terminal_vertex = f"q_{t+1}_{units_to_buy+
       closed_vertices_units[i]}"
23                         edge_weight = e**(-0.04 * (t + 1) / (terms - 1))
       * units_to_buy * (-prices[t]) * (-1) + int(added_constant)
24                         edge = (initial_vertex, terminal_vertex,
       edge_weight)
25                         edges.append(edge)
26                         if terminal_vertex not in vertices:
27                             vertices.append(terminal_vertex)
28                             vertices_units.append(closed_vertices_units[i
       ] + units_to_buy)
29                 for units_to_sell in range(1, units_to_sell_max + 1):
30                     if closed_vertices_units[i] - units_to_sell <= q_max[
       t]:
31                         initial_vertex = closed_vertices[i]
32                         terminal_vertex = f"q_{t+1}_{
       closed_vertices_units[i]-units_to_sell}"
```

```
33                          edge_weight = e**(-0.04 * (t + 1) / (terms - 1))
    * units_to_sell * prices[t] * (-1) + int(added_constant)
34                          edge = (initial_vertex, terminal_vertex,
    edge_weight)
35                          edges.append(edge)
36                          if terminal_vertex not in vertices:
37                              vertices.append(terminal_vertex)
38                              vertices_units.append(closed_vertices_units[i
    ] - units_to_sell)
39                  initial_vertex = closed_vertices[i]
40                  terminal_vertex = f"q_{t+1}_{closed_vertices_units[i]}"
41                  edge_weight = int(added_constant)
42                  edge = (initial_vertex, terminal_vertex, edge_weight)
43                  edges.append(edge)
44                  if terminal_vertex not in vertices and q_min[t] <=
    closed_vertices_units[i] <= q_max[t]:
45                      vertices.append(terminal_vertex)
46                      vertices_units.append(closed_vertices_units[i])
47              else:
48                  initial_vertex = closed_vertices[i]
49                  terminal_vertex = f"q_{t+1}_0"
50                  if goal != "Null" and goal != closed_vertices_units[i]:
51                      edge_weight = e**(-0.04) * closed_vertices_units[i] *
     prices[t] * (1 - penalty) * (-1) + int(added_constant)
52                  else:
53                      edge_weight = e**(-0.04) * closed_vertices_units[i] *
     prices[t] * (-1) + int(added_constant)
54                  edge = (initial_vertex, terminal_vertex, edge_weight)
55                  edges.append(edge)
56      return edges, root
57
58  def dijkstra(edges, root, end):
59      start_vertex = [0, root[0], "init"]
60      open_vertices = []
61      open_vertices.append(start_vertex)
62      closed_vertices = []
63      active_vertex = start_vertex
64      while end not in active_vertex:
65          i = 0
66          for edge in edges:
67              if edge[0] == active_vertex[1]:
68                  open_vertex = [edge[2], edge[1], active_vertex[1]]
69                  open_vertex[0] += active_vertex[0]
70                  if open_vertex not in closed_vertices:
71                      already_open = False
72                      known_dist = float('inf')
73                      i = 0
74                      for vertex in open_vertices:
75                          if open_vertex[1] == vertex[1]:
76                              already_open = True
77                              known_dist = vertex[0]
```

```
78                              index = i
79                          i += 1
80                      if not already_open or known_dist > open_vertex[0]:
81                          open_vertices.append(open_vertex)
82                          if already_open:
83                              open_vertices.pop(index)
84          open_vertices.pop(0)
85          closed_vertices.append(active_vertex)
86          open_vertices = sorted(open_vertices)
87          active_vertex = open_vertices[0]
88      length = active_vertex[0]
89      path = []
90      i = 0
91      while start_vertex[1] not in path:
92          path.append(active_vertex[1])
93          parent = active_vertex[2]
94          for vertex in closed_vertices:
95              if parent == vertex[1]:
96                  active_vertex = vertex
97          i += 1
98      path.reverse()
99      return path, length
100
101 def main(a):
102     terms = 13
103     q_max = 10, 10, 10, 10, 8, 8, 8, 8, 10, 10, 10, 10, 10
104     q_min = 0, 0, 0, 4, 4, 6, 6, 4, 4, 0, 0, 0, 0
105     u_max = 4, 4, 2, 2, 1, 1, 1, 1, 2, 2, 4, 4, 4
106     i_max = 4, 4, 2, 2, 1, 1, 1, 1, 2, 2, 4, 4, 4
107     q_0 = 5
108     prices = [20, 22, 25, 18, 15, 15, 20, 19, 21, 12, 22, 25, 25]
109     edges, root = make_edges(terms, q_max, q_min, u_max, i_max, q_0,
        prices, 5, a)
110     path, length = dijkstra(edges, root, "q_13_0")
111     length = (length - terms * (max(q_max) * max(prices) + 1)) * -1
112     return f"The shortest path is {path} with a length of {length}"
113
114 def simulation(lower, upper, n):
115     arr = np.linspace(lower, upper, n)
116     for i in arr:
117         print(f"penalty factor: {i}, result: {main(i)}")
118
119 simulation(0.01951, 0.019512, 20)
```